

Pegasus and PegaRing: Efficient (Ring) Signatures from Sigma-Protocols for Power Residue PRFs with (Q)ROM Security

Xinyu Zhang¹, Ziyi Li², Ron Steinfeld¹, Raymond K. Zhao³, Joseph K. Liu¹,
and Tsz Hon Yuen¹

¹ Monash University, Melbourne, Australia

{xinyu.zhang2, ron.steinfeld, joseph.liu, john.tszhonyuen}@monash.edu

² Hong Kong Polytechnic University, Hong Kong, China

zycal.li@polyu.edu.hk

³ ExeQuantum, Melbourne, Australia

raykzhao@gmail.com

Abstract. In this work, we present a novel commit-and-open Σ -protocol based on the Legendre and power residue PRFs. Our construction leverages the oblivious linear evaluation (OLE) correlations inherent in PRF evaluations and requires only black-box access to a tree-PRG-based vector commitment. By applying the standard Fiat-Shamir transform, we obtain a post-quantum signature scheme, PEGASUS, which achieves short signature sizes (6025 to 7878 bytes) with efficient signing (3.910 to 19.438 ms) and verification times (3.942 to 18.999 ms). Furthermore, by pre-computing the commitment phase, the online response time can be reduced to as little as 0.047 to 0.721 ms. We prove the security of PEGASUS in both the classical random oracle model (ROM) and the quantum random oracle model (QROM), filling a gap left by prior PRF-based signature schemes.

We further develop a ring signature scheme, PEGARING, that preserves the three-move commit-and-open structure of PEGASUS. Compared to previous PRF-based ring signature called DualRing-PRF (ACISP 2024), PEGARING reduces the constant communication overhead by more than half and achieves significantly faster signing and verification. For a ring size of 1024, PEGARING yields signatures of 29 to 32 KB, with signing times of 8 to 44 ms, and verification times of 6 to 31 ms, depending on the parameters. Finally, we prove the security of PEGARING in both the ROM and the QROM, which is, to the best of our knowledge, the first symmetric-key primitives-based ring signature with practical performances and provable QROM security.

1 Introduction

The rapid advancement of quantum computing infrastructures has raised significant concerns within the cryptographic community, particularly due to the potential threats posed by quantum algorithms such as Shor’s polynomial time

algorithm for factoring integers and computing discrete logarithms [59]. These breakthroughs imply that many widely deployed public-key cryptosystems could be compromised by an adversary with a sufficiently powerful quantum computer. Therefore, it is imperative to develop alternative public-key cryptographic protocols that resist quantum attacks. One promising direction is to construct such schemes solely from symmetric-key primitives, as these are well-studied and less prone to unknown attacks.

Existing signature schemes based on symmetric-key primitives can be categorized into two paradigms: hash-tree-based signatures, e.g., [48, 16, 42, 11, 12] and zero-knowledge proof (including zkSNARKs⁴) based signatures, e.g., [19, 46, 44, 57, 8, 15, 56, 28, 7, 69].

Hash-Tree-based Signatures. A crucial building block of hash-tree-based signature schemes is the hash-based one-time signature (OTS), whose origin can be traced back to Lamport’s construction in the late 1970s [48]. To increase signing capacity, an OTS was combined with a Merkle tree [52], leading to hash-based many-time signatures (MTS). Such constructions, culminating in XMSS [16] and its multi-tree variant [42], are highly efficient while relying on minimal security assumptions. Nonetheless, signature schemes like XMSS suffer from stateful signing (i.e., the signer must keep track of the used one-time key pairs and ensure they are never reused). To address the limitation, Bernstein et al. proposed SPHINCS [11] and SPHINCS+ [12], which incorporated hash-based few-time signatures (FTS) into a hyper-tree (a tree of Merkle trees).

Hash-tree-based signature schemes offer the following advantages. First, their security only relies on *black-box* access to underlying hash functions, which allows for easy replacement if new vulnerabilities arise, ensuring long-term security. Second, signature verification can be extremely simple and fast, which is often dominated by the verification of authentication paths in the Merkle tree. Recently, NIST standardized SPHINCS+ for applications that require stronger security guarantees against quantum computers.

Generic ZKP-based Signatures. Zero-knowledge proof (ZKP)-based signature schemes [19, 46, 44, 57, 8, 15, 56, 28, 7, 6, 4, 34] are constructed by letting the signer prove, in zero-knowledge, that they have a secret key k corresponding to the public key (x, y) such that $y = f_k(x)$ for some symmetric one-way function f (e.g., a block cipher).

The initial building block in this line of work is the Σ -protocol for proving statements over general circuits [37, 19], which follows the MPC-in-the-head (MPCitH) paradigm [43]. By applying the Fiat-Shamir transform [35] or Unruh’s transform [60, 61, 64], one obtains signature schemes that are provably secure in the random oracle model (ROM) or quantum random oracle model (QROM) [19]. However, early Σ -protocols suffered from large proofs, motivated improvements in [46], which instantiates MPCitH in the pre-processing model, significantly reduces the proof size. Later, various signature schemes based on generic ZKPs were introduced, with different aspects of improvements:

⁴ zkSNARK stands for “zero-knowledge succinct non-interactive argument of knowledge”.

1. Reducing the size of the circuit to be proven in MPCitH for specific symmetric-key primitives, such as BBQ [57] and Banquet [8].
2. Using symmetric-key primitives with better algebraic structures, such as LegRoast [15] and Rainier [28].
3. Upgrading the proof system itself, as in VOLE-in-the-head (VOLEitH) [7] and Threshold Computation-in-the-head (TCitH) [34].

Except for the seminal Picnic signature schemes [19, 46, 44], most subsequent works improve efficiency at the cost of increased round complexity [57, 8, 15, 56, 28, 7, 6, 4, 34]. Multi-round protocols can be faster and sometimes yield smaller proofs than Σ -protocols; however, they suffer from more complicated security analysis, and the majority of them lack proofs in the QROM. In contrast, a standard three-move Σ -protocol typically enjoys tighter Fiat-Shamir reductions [2, 3], cleaner security proofs (both classical and quantum) [53, 29, 30], easy composition (e.g., for OR/threshold proofs) [23], and simpler implementations. The Σ -protocol-based Picnic signature achieves performance comparable to hash-tree-based SPHINCS+ [12], but its signature size remain significantly larger than those of state-of-the-art multi-round schemes, such as VOLEitH-based signatures [7, 6, 4]. To this end, we propose the following research question:

Is there a more efficient signature scheme with provable QROM security that can be obtained through a standard Fiat-Shamir transform of a Σ -protocol?

While our primary focus is on constructing more efficient Σ -protocol-based signature scheme, it is natural to ask whether the same three-move structure can be exploited in broader contexts. A particularly important example is **ring signatures**, which is a fundamental privacy-enhancing cryptographic primitive with wide applications in real world systems, such as blockchain, e-cash, and e-voting. In a nutshell, a ring signature allows a signer to demonstrate membership in a group without revealing its identity. To the best of our knowledge, existing symmetric-key-based ring signature constructions can be efficient, but their security has only been proven in the classical ROM [46, 27, 18, 38, 66, 70, 21], partly because the majority of them rely on multi-round protocols, which complicates the security analysis in the QROM. Nevertheless, ring signatures with provable security in QROM are limited in the literature, and often suffer from impractical performance [41]. This motivates our second research question

Is there a Σ -protocol for efficient set membership proof that can be compiled into a ring signature with provable security in QROM via the Fiat-Shamir transform?

1.1 Our Contributions

In this paper, we answer both research questions affirmatively by presenting efficient commit-and-open (C&O) Σ -protocols for Legendre and power residue

Scheme	$ \sigma $	Sign	Verify	Security	Σ -Protocol
SPHINCS ⁺ -F [12]	16976	52.90	6.26	Hash+QROM	No
SPHINCS ⁺ -S [12]	8080	861.62	2.61		
FAESTer-F [4]	5924	1.722	1.413	AES+QROM	No
FAESTer-S [4]	4506	12.787	9.783		
Picnic3 [44]	12595	18.59	14.26	Low-MC+QROM	Yes
PorcRoast [†] -F [15]	8806	3.000	≈ 3.000	PRF+ROM	No
PorcRoast [†] -S [15]	6451	19.750	≈ 19.750		
PEGASUS-F	7878	16.81	16.95	PRF+QROM	Yes
PEGASUS-S	6025	83.5	81.7		

Table 1: Comparisons of Symmetric-Key-Based Post-Quantum Signatures Targeting 128-Bit (Classical) Security. “F” (Fast) refers to parameters that trade larger signature sizes for faster running speeds; “S” (Short) refers to parameters that trade running speeds for shorter signature sizes. $|\sigma|$ refers to signature size in **bytes**. “Sign” and “Verify” times are reported in **millions of CPU cycles**. [†] is not optimized with AVX instructions. “PRF” refers to the t -th power residue PRF with $t = 254$.

PRFs. Prior works, such as LegRoast (PorcRoast) [15] and Loquat [69], rely on generic zero-knowledge proofs including MPC-in-the-head and zkSNARKs to demonstrate the correct evaluation of these PRFs, leading to multi-round protocols with security guarantees only in the classical random oracle model (ROM). We show that it is possible to design an efficient *three-move commit-and-open* Σ -protocol tailored to these PRFs, which can be compiled into a signature scheme, which we call PEGASUS (**P**ost-quantum signature**E** from the le**G**endre **A**nd power re**S**idUe prfs and **S**igma protocol), via the Fiat-Shamir transform [35]. Our construction achieves *smaller signatures* with stronger security in the quantum random oracle model (QROM), while only incurring slightly higher costs for signing and verification times.

Next, we construct a set membership proof for Legendre and power residue PRFs that preserves the three-move commit-and-open structure. To optimize its performance, we establish the β -approximate PRF relation over an *arithmetic progression list*, which may be of independent interest. To this end, we obtain an efficient ring signature scheme with signature sizes more than twice smaller than those of the same PRF-based construction in [70]. Moreover, we prove the security of our ring signature in both ROM and QROM. To the best of our knowledge, this is the *first* practical symmetric-key-based ring signature scheme with provable QROM security.

We now describe our contributions in more detail:

1. **Σ -Protocol for Legendre and Power Residue PRFs.** Our construction of the commit-and-open (C&O) Σ -Protocol leverages the oblivious linear evaluation (OLE) correlation inherent in PRF evaluations. Compared with the Σ -protocol underlying Picnic3 [44], our scheme (PEGASUS-F) achieves

Scheme	Signature Size				Sign	Verify	Security
	$n = 2^3$	2^6	2^9	2^{10}	$n = 2^{10}$		
DualRing-LB [†] [67]	3.9	5	16	28	100	21	Lattice+ROM
DualRing-PRF [†] -F [70]	43	45	52	71	123	94	PRF+ROM
DualRing-PRF [†] -S [70]	33	34	41	56	452	270	
MPCitH Ring [38]	46	50	55.5	56	0.21	0.21	Low-MC+ROM
STARK Ring [66]	25	29	29	29	0.16	0.16	Rescue-Prime+ROM
VOLEitH Ring [21]	4.78	4.86	-	5.03	0.016	0.016	AES+ROM
PEGARING-F	16	17	24	32	0.008	0.006	PRF+QROM
PEGARING-S	13	14	21	29	0.044	0.031	

Table 2: Comparisons of Symmetric-Key-based Post-Quantum Ring Signatures Targeting 128-Bit (Classical) Security. “F” (Fast) refers to parameters that trade larger signature sizes for faster running speeds; “S”(Short) refers to parameters that trade running speeds for shorter signature sizes. Signature Size is reported in **KB**, and signing/verification times are reported in **seconds**. † reports results from Python implementations. “PRF” refers to the t -th power residue PRF with $t = 254$.

signatures that are roughly **1.6×** **smaller** in size, while also offering comparable signing and verification performances.

Compared to prior PRF-based signatures such as PorcRoast [15], our construction saves about **0.5KB to 1KB** in signature size with slightly higher (but remain practical) running times.⁵ We note that PEGASUS-F also features **public keys that are eight times smaller** (0.5KB) than those of PorcRoast (4KB), while PEGASUS-S retains public key size of 4KB. Moreover, owing to the Σ -protocol structure of PEGASUS, our scheme enjoys a clean **security proof in QROM**, a property missing from *all* previous PRF-based signatures [15, 69].

However, the most recent VOLE-in-the-head-based signature scheme, called FAESTer [4], achieves smaller signatures (by about 2 KB) and faster signing and verification than PEGASUS. While FAESTer leads in concrete efficiency, our scheme is conceptually and analytically simpler. In particular, our security proofs in both ROM and QROM follow well-established Σ -protocol methodologies, making them significantly easier to analyze and verify. A detailed comparison between PEGASUS and other state-of-the-art symmetric-key-based signatures schemes is presented in Table 1.

⁵ We do not compare our scheme with the Legendre-PRF based Loquat [69] explicitly, as Loquat is optimized for SNARK-friendliness rather than concrete performances. Notably, Loquat produces signatures of 57 KB at 128-bit (classical) security level, around 10 times larger than ours.

2. **Σ -Protocol for Set Membership Proof.** We design an efficient C&O Σ -protocol for set membership proof by leveraging PEGASUS and the DualRing framework [67], leading to a ring signature (PEGARING) whose size grows linearly with the ring size, but with a small linear term of 128 bits. In particular, the ring signature size of PEGARING is $13 \text{ KB} + 128 \text{ bits} \times n$ for a ring of size n [70], where the constant term is more than **twice smaller** than DualRing-PRF, a ring signature scheme based on the Legendre and power residue PRFs by combining LegRoast [15] with DualRing [70]. Furthermore, although DualRing-PRF reports performance using a Python-based implementation, which is **at least four orders of magnitude** slower than PEGARING’s C implementation, the performance gap is not only from the choice of programming language but also from the inherent **algorithmic overhead**: DualRing-PRF has an extra cut-and-choose round, which introduced *three nested loops*, whereas PEGARING eliminates cut-and-choose entirely, leading to much faster signing and verification. Compared to DualRing-LB (lattice-based DualRing) [67], PEGARING has a **smaller linear part** (16 bytes versus 24 bytes in DualRing-LB). While the constant component of DualRing-LB is smaller (3829 bytes versus 13,772 bytes), PEGARING begins to outperform DualRing-LB when the ring size exceeds 1,243 users. Table 2 presents further comparisons, including the VOLEith-based ring signature [21], which achieves excellent efficiency but only admits ROM security. In contrast, our scheme is provably secure in both ROM and QROM. To the best of our knowledge, PEGARING is the *first* practical symmetric-key-based ring signature with provable QROM security.
3. **Theoretical Foundations for Legendre and Power Residue PRFs.** As a side contribution, we prove the hardness of β -approximate PRF relation over an *arithmetic progression list* (i.e., a list of the form $(a, a+b, a+2b, \dots)$ where $a, b \in \mathbb{F}_p$). The primary motivation for introducing this list is to improve the performance of PEGARING, as the correlations between its elements reduces the signature size by about 1.6 KB and improves the running time by about 4 times.⁶ Our result may be of independent interest as the strongly correlated list could serve as a theoretical foundation for constructing advanced signature schemes based on the Legendre and power residue PRFs.

1.2 Technical Overview

We begin by establishing the background of our work: the key identification problem (i.e., prove knowledge of a secret key) of the Legendre and power residue PRFs.

Key Identification. For an odd prime p , the Legendre PRF (and its generalization named t -th power residue PRF) is conjectured to be a pseudo-random

⁶ The improvement is based on the prime $p \approx 2^{128}$, larger fields can actually provide greater savings.

function family [26], indexed by a (secret) key $K \in \mathbb{F}_p$, which on input a field element $a \in \mathbb{F}_p$, outputs a $\log_2(t)$ -bit integer indicating the t -th power residuosity of $(K + a)$, denoted as $\mathcal{L}_K^t(a) = \mathcal{L}_0^t(K + a) \in \mathbb{Z}_t$.

To uniquely identify K , the statement (public key) must contain many PRF evaluations:

$$pk = (pk_0, \dots, pk_{L-1}) = (\mathcal{L}_K^t(I_0), \dots, \mathcal{L}_K^t(I_{L-1})),$$

where $\mathcal{I} = (I_0, \dots, I_{L-1}) \in \mathbb{F}_p^L$ is a publicly defined list. The (relaxed) key identification problem of the Legendre and power residue PRFs is then described as follows: given a public key consisting of L PRF outputs, prove the knowledge of a value K such that for $M \leq L$ uniformly random indices $(\bar{\ell}_1, \dots, \bar{\ell}_M)$, where $\bar{\ell}_e \xleftarrow{\$} [0, L-1]$, it holds that $\mathcal{L}_K^t(I_{\bar{\ell}_e}) = pk_{\bar{\ell}_e}$ for all $e \in [M]$.

Existing works [15, 69] exploit the multiplicative homomorphism of the PRFs to avoid directly evaluating them inside zero-knowledge proofs. Specifically, for values $o_e = (K + I_{\bar{\ell}_e})r_e$, where each $r_e \xleftarrow{\$} \mathbb{F}_p$ is uniformly chosen by the prover, it can be shown that

$$\begin{aligned} \mathcal{L}_0^t(o_e) &= \mathcal{L}_0^t((K + I_{\bar{\ell}_e})r_e) \\ &= \mathcal{L}_0^t(K + I_{\bar{\ell}_e}) + \mathcal{L}_0^t(r_e) \\ &= pk_{\bar{\ell}_e} + \mathcal{L}_0^t(r_e). \end{aligned}$$

Zero-knowledge proofs are then employed to ensure that for all $e \in [M]$, o_e is correctly computed using K , $I_{\bar{\ell}_e}$, and r_e . Furthermore, the values K , r_e and $\mathcal{L}_0^t(r_e)$ must be committed before the prover learns the challenge indices $(\bar{\ell}_e)_{e \in [M]}$.

Technical Overview of PEGASUS. Our key insight arises from a close inspection of the structure of $(o_e)_{e \in [M]}$. Specifically, for the verifier's challenge $\bar{\ell}_e \in [0, L-1]$, the honestly computed o_e takes the form of $(K + I_{\bar{\ell}_e})r_e$ for all $e \in [M]$. Expanding this expression yields $o_e = Kr_e + I_{\bar{\ell}_e}r_e$. This observation allows us to map $(o_e, I_{\bar{\ell}_e})$, which are known to the verifier, and (K, r_e) , which are known to the prover, to an Oblivious Linear Evaluation (OLE) correlation.

Recall that in an OLE protocol, the sender holds the coefficients (u, v) and the receiver holds the challenge c . Then, at the end of the OLE protocol, the sender learns nothing, while the receiver learns $s = uc + v$. Based on the previous observation, we let the prover be the OLE sender and the verifier be the OLE receiver. Consequently, the key identification problem of the Legendre and power residue PRFs can be solved by M invocations of an OLE protocol, where in each OLE execution, the sender's inputs are $u_e = r_e$ and $v_e = Kr_e$, and the receiver's input is $c_e = I_{\bar{\ell}_e}$. Hence, at the end of the OLE protocol, the receiver obtains $o_e = u_e c_e + v_e = r_e I_{\bar{\ell}_e} + Kr_e$, while the sender learns nothing. However, designing an OLE protocol based on symmetric key primitives is challenging, since OLE typically uses oblivious transfer (OT) as a building block, which requires public key encryption.

Our second observation is inspired by the core technique presented in Soft-SpokenOT [55] and VOLEitH [7], which shows how to realize vector OLE (VOLE)

using a tree-PRG based vector commitment scheme. However, the compiler proposed in [7] *cannot* be directly applied to our setting. In these tree-PRG based VOLE protocols, the sender holds a list $\mathbf{u} = (u_0, \dots, u_{L-1})$ and the receiver has a challenge c' that is an index in $\{1, 2, \dots, L\}$. Then, the sender defines

$$u' = \sum_{\ell=0}^{L-1} u_\ell, \quad v' = \sum_{\ell=0}^{L-1} u_\ell \cdot -(\ell + 1).$$

The receiver's VOLE output is defined as:

$$s' = u'c' + v' = \sum_{\ell=0, \ell+1 \neq c'}^{L-1} u_\ell(c' - (\ell + 1)),$$

which can be computed by having the sender reveal $(L - 1)$ -out-of- L elements in $\mathbf{u}$.

In our case, the prover has inputs $u_e = r_e$ and $v_e = Kr_e$ which obviously do not satisfy the relation required by the compiler. To address this, we first consider split r_e to additive shares such that $r_e = \sum_{\ell=0}^{L-1} r_{e,\ell}$. This unfortunately does not solve our problem since there is still a mismatch between our $v_e = Kr_e = K \sum_{\ell=0}^{L-1} r_{e,\ell}$ and $v'_e = \sum_{\ell=0}^{L-1} u_\ell \cdot -(\ell + 1)$. Moreover, the verifier's challenge $c = I_{\bar{\ell}_e}$ is a random field element in the pre-defined list $\mathcal{I}$, which does not match with $c' \in \{1, 2, \dots, L\}$ either.

We solve the problem by a novel idea, which interprets the field elements in $\mathcal{I}$ as *secret key shares*. It is obvious that the sum of elements in $\mathcal{I}$ is not equal to the secret key K . Hence, for each $e \in [M]$, the prover needs to send a key adjustment before it learns verifier's challenges. The interpretation allows us to align the prover's inputs and verifier's challenges with those of VOLE sender and receiver in the tree-PRG based protocol. As a result, the prover (signer) can commit to its secret inputs using a tree-PRG-based vector commitment, then open all-but-one shares together with the key adjustment, allowing the verifier to compute o_e and verify the relation. This leads to a standard three-move commit-and-open (C&O) Σ -protocol for key identification problem of the Legendre and power residue PRFs.

Technical Overview of PEGARING. We develop an efficient set membership proof for the Legendre and power residue PRFs using the DualRing framework [67]. DualRing can be seen as a compiler that transforms a special type of Σ -protocols, called the Type- T^* canonical identification schemes, into OR-compositions. The key requirement of DualRing is that the verification algorithm V can be decomposed into two algorithms V_1 and V_2 such that $V = V_1 \odot V_2$ for some group operation $\odot$. More importantly, V_1 must be additive or multiplicative homomorphic. The benefit of using the DualRing compiler is that the resulting ring signature consists of a single response and n challenges for a ring of size n . In the post-quantum setting, since the challenge is typically much smaller than the response, applying DualRing compiler can lead to schemes with notably short signatures for small to medium rings.

Although we have a three-move identification scheme as described in the previous subsection, the DualRing compiler cannot be directly applied to obtain a secure set membership proof, as it is not designed for C&O Σ -protocols. The core issue is that the verifier must know the real signer’s challenge to recompute the response, which reveals the signer’s identity and breaks anonymity. To solve the problem, we modify the signing protocol so that the verifier computes the response using a *mutual challenge*, defined as the sum of all ring member’s challenges. However, this creates a new problem: the response computed from the mutual challenge differs from the real signer’s actual response, violating the correctness. We propose the following solution, which preserves the three-move structure of the set membership proof.

Let n be the size of the ring. Then, the challenge of i -th ring member for each $e \in [M]$ is an index $\bar{\ell}_e^{(i)} \in [0, L - 1]$, indicating the position of the checked public key symbol $pk_{\bar{\ell}_e^{(i)}}^{(i)}$. The mutual challenge is defined as:

$$\bar{\ell}_e = \sum_{i=1}^n \bar{\ell}_e^{(i)} \pmod{L}.$$

Our key insight is that the distance between the mutual challenge $\bar{\ell}_e$ and the real signer’s challenge $\bar{\ell}_e^{(\pi)}$ is fixed in the commitment phase. In fact, it is exactly the sum of the challenges from all non-signers:

$$\bar{\ell}'_e = \bar{\ell}_e - \bar{\ell}_e^{(\pi)} = \sum_{i=1, i \neq \pi}^n \bar{\ell}_e^{(i)} \pmod{L}.$$

This observation enables the real signer to pre-compute all possible share adjustments by iterating over pairs of the mutual challenge and the real signer’s challenge. Since their distance is fixed, there are exactly L such pairs. These share adjustments are committed using a Merkle tree, together with the tree-PRG-based vector commitment in the original Σ -protocol. After receiving the verifier’s challenge, the signer opens all-but-one shares according to the mutual challenge and the corresponding share adjustment in the Merkle tree. This forms a three-move set membership proof, which can be compiled into a ring signature via the Fiat-Shamir transform.

Optimizations. We present various optimizations for PEGASUS and PEGARING.

- **Correlated All-but- τ Vector Commitment and Octopus Opening.** Tree-PRG-based vector commitment can be improved by combining multiple trees into a single large tree, a technique called batch all-but-one vector commitment (BAVC). Because intermediate tree nodes can overlap, the proof size for opening all-but- M leaves is expected to be smaller than opening all-but-one leaves per tree. A recent improvement to BAVC [17] employs the circular correlation robust (CCR) hash with the half-tree technique [40], significantly reduces commitment generation and verification time. However,

a CCR-hash is typically instantiated with fixed-key AES, which is acceptable in the classical ROM but complicates the security proof in the QROM, since AES can only be modeled as a random permutation rather than a random function.

To avoid relying on the quantum random permutation model, we modularize the commitment generation of the correlated BAVC into two separate algorithms: `BAVC.TreeGen` and `BAVC.LeafCom`. We keep the CCR-hash and half-tree techniques for `BAVC.TreeGen`, while instantiating `BAVC.LeafCom` with standard random oracles.⁷ This modularization allows us to follow the standard technique of [30], resulting in a tight bound on the QROM security of our schemes. Another improvement to the vector commitment is the use of the *Octopus opening*, originally introduced to minimize the number of nodes revealed when opening multiple leaves in a Merkle tree, which can be naturally adapted to the BAVC setting.

We note that if the classical security of PEGASUS and PEGARING suffices (as is the case for many other post-quantum signature and ring signature schemes), then correlated BAVC [17] can be fully adopted, which applies the CCR-hash also to the leaf commitment. This can lead to a significant speedup for both schemes. In particular, our experiments show that the signing and verification times of PEGASUS can be improved by 2 to 11 ms, while the running time of PEGARING can be improved by 4 to 10 ms.

- **PRF-relation with respect to arithmetic progression lists.** To further improve the overall performance of PEGARING, we introduce an optimization based on arithmetic progression lists with a fixed gap. Instead of generating the list $\mathcal{I}$ by selecting L uniformly random field elements, we partition it into z sub-lists, each containing c elements, such that $L = z \times c$. For each sub-list, we sample a uniformly random element $a_j \xleftarrow{\$} \mathbb{F}_p$ and construct the arithmetic progression

$$\mathcal{I}_j = (a_j, a_j + b, \dots, a_j + b \cdot (c - 1)),$$

for $j \in [0, z - 1]$, where $b \xleftarrow{\$} \mathbb{F}_p^*$ is a fixed non-zero gap shared across all sub-lists. By leveraging the fixed gap b , we reduce the size of the Merkle tree in PEGARING from L to z , resulting in faster signing and verification, as well as smaller signature sizes. The key challenge we solve here is the β -approximate PRF relation with respect to the arithmetic progression list, which is technically non-trivial as the proof technique in LegRoast [15] does not apply in this setting. In particular, due to the elements in each sub-list are correlated, the character sum can only be taken over a very short sequence, making classical bounds like the Weil bound inapplicable. To overcome the problem, we first show that the output distribution of t -th power residue RPFs is nearly uniform, even when evaluated over the arithmetic progression lists with fixed gaps (Lemma 11). We then establish the β -approximate PRF

⁷ This is eventually similar to the tree-PRG-based vector commitment used in [24], but we use batched version (BAVC), while they use the single tree.

relation using Rényi divergence [54] The result is formally stated in Theorem 2.

- **Online-Offline Signing.** Similar to many other Σ -protocols, the first round (commitment generation) can be performed offline, since it does not involve any message. For PEGASUS and PEGARING, this means that if the signer pre-computes the commitments offline, the response can be generated extremely quickly, where PEGASUS online time ranges from 0.047 ms (1% of total time) to 0.721 ms (3% of total time) and PEGARING online time ranges from 1.298 ms (15% of total time) to 10.175 ms (22% of total time). This is particularly useful in scenarios requiring lightweight signature generation. For example, a resource-constrained device with limited battery life can generate commitments while charging. Later, once on-the-fly, it can compute the Fiat-Shamir challenge and generate the response efficiently.

1.3 Related Work

Legendre and Power Residue PRFs based Post-Quantum Signature.

The first post-quantum signature scheme based on Legendre and power residue PRFs was introduced by Beullens et al. [15], building on prior work by Grassi et al. [39], which highlighted the MPC-friendliness of these PRFs. As a result, MPC-in-the-head (MPCitH) [43] was applied to construct signature schemes based on the Legendre PRF (LegRoast) and the power residue PRF (PorcRoast) in [15]. Both LegRoast and PorcRoast demonstrated good performance among symmetric-key-based post-quantum signatures, with signature sizes of 12.2 KB for LegRoast and 6.3 KB for PorcRoast. More recently, Zhang et al. [69] introduced a new approach for constructing the Legendre PRF based signature scheme by leveraging the univariate sumcheck protocol, a core technique in the hash-based zkSNARK called Aurora [10]. This innovation led to the SNARK-friendly post-quantum signature, but with relatively large signature sizes (59 KB) and slow signing times (over 4 seconds in their proof-of-concept Python implementation). While PEGASUS already achieves concrete improvements in signature size, it also establishes the security proofs in the QROM, a feature missing from prior works [15, 69].

Post-Quantum Ring Signatures. Symmetric-key-based post-quantum ring signatures were first proposed in [27, 46], which leverage the accumulator with one-way domain. The core idea is to let the signer prove to the verifier its knowledge of a pre-image of an accumulated value in zero-knowledge. Consequently, these constructions rely heavily on Merkle trees as accumulators and zero-knowledge proofs for generic circuits. Subsequent works improved upon this framework by either shifting the Merkle tree membership proof verification outside the circuit [38] or replacing the MPC-in-the-head with zkSTARKs [66]. Recently, Zhang et al. [70] proposed a ring signature scheme based on the Legendre and power residue PRFs, which is constructed by compiling the LegRoast signature [15] into a ring signature using the DualRing framework [67]. To ensure

anonymity, their construction introduces an additional cut-and-choose round, which results in a relatively large constant (33 KB) in the signature sizes and slow signing/verification times.

Lattice-based ring signature schemes such as [33, 49, 14, 50] are highly efficient in terms of signature size. For instance, MatRiCT+ [33] achieves a 9 KB signature for a ring of size 11, which is approximately 1.4 to $1.7\times$ smaller than that of PEGARING. However, MatRiCT+ is designed for threshold ring signatures and reports performance based on transaction generation and verification rather than standard ring signature signing and verification. Even so, the reported timings show that PEGARING with anonymity level $1/1024$ is already up to $12\times$ faster than MatRiCT+ at anonymity level $1/10$. Furthermore, MatRiCT+ incurs a growing public key size with the number of users (e.g., 3 KB per user for a ring of size 1024) whereas PEGARING has constant public key size of 0.5 KB up to 4 KB, depending on the parameter settings. Compared to another lattice-based scheme, Falafel [14], which produces 30 KB signatures and requires 90 ms signing time for a ring of size 8, PEGARING achieves half the signature size and $4.5\times$ faster signing. Bloom [49], designed for large ring sizes, provides about $1.23\times$ smaller signature sizes than PEGARING for a ring of size 64, but signing and verification times are not reported in the paper, making a direct comparison difficult. More importantly, none of the above schemes establish the security proof in the QROM, making PEGARING a strong candidate for practical applications with stricter post-quantum security requirements.

2 Preliminaries

Notations. We denote security parameter by λ and a set of negligible functions by $\text{negl}(\lambda)$ where a function $f(\lambda)$ is negligible if $f(\lambda) = \lambda^{-\omega(1)}$. We use $\mathbb{F}_p$ to denote the prime field for a large prime p where $p = O(2^\lambda)$. For positive integer N and real $q \in [0, 1]$, we denote by $\mathfrak{B}(N, q)$ the binomial distribution with N trials and success probability q . We define $[n] = \{1, \dots, n\}$ and $[0, n-1] = \{0, 1, \dots, n-1\}$. For a bit string h and an integer l , $h[:l]$ denotes the first l -bit of x . We use bold lower case letters (e.g., $\mathbf{v}$) to represent vectors and bold upper case letters (e.g., $\mathbf{V}$) to represent vector of vectors. Finally, ‘PPT’ stands for probabilistic polynomial time.

Let B be a Boolean statement, $\|B\|$ denotes a bit, which is 1 if B is true, and 0 otherwise. We use $\otimes$ to represent tensor product. For a finite set S , the cardinality of S is denoted by $|S|$, and $x \xleftarrow{\$} S$ means that x is uniformly random sampled from S . For two sets S_1 and S_2 , $S_1 \times S_2$ denotes the Cartesian product of them. Let D be a distribution, and let $x \leftarrow D$ denote sampling x from D . We use $\text{Dis}(D_1, D_2)$ to denote the statistical distance between two distributions D_1 and D_2 . For an algorithm $\mathcal{A}$, given an input string inp , we write $\text{out} \leftarrow \mathcal{A}(\text{inp})$ (or equivalently $\mathcal{A}(\text{out}) \rightarrow \text{inp}$) to denote that out is the output of $\mathcal{A}$ on input inp . If $\mathcal{A}$ is deterministic, then out is fixed and denoted by $\text{out} := \mathcal{A}(\text{inp})$. We may also use the notation $\text{out} \leftarrow \mathcal{A}(\text{inp}; r)$ to make the randomness r explicit for a probabilistic algorithm $\mathcal{A}$. When $\mathcal{A}$ has oracle access to O , we write $\mathcal{A}^O$.

2.1 Security Models

The Random Oracle Model (ROM), introduced by Bellare and Rogaway [9], idealizes a hash function as a truly random function accessible only through black-box queries. Depending on how the adversary interacts with the oracle, the model is classified into the classical ROM and the quantum ROM.

Random Oracle Model In the classical Random Oracle Model (ROM), algorithms can only access the random oracle through classical queries. The oracle is typically simulated via lazy sampling, where an adversary cannot distinguish between interactions with the true random oracle and with the lazy sampling simulation (Lemma 1).

Lemma 1. *There exists a PPT algorithm $\mathcal{S}$ that simulates the classical random oracle for a hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$ by maintaining a hash list L_H , generated via lazy sampling. For any PPT adversary $\mathcal{A}$, we have*

$$\Pr[\mathcal{A}^{O_H}(1^\lambda) \rightarrow 1] = \Pr[\mathcal{A}^{\mathcal{S}}(1^\lambda) \rightarrow 1],$$

where $\mathcal{S}$ is formally described as follows:

- Initialize $L_H = \emptyset$;
- If $\exists (x, y) \in L_H$: **return** y ;
- Otherwise, sample $y \xleftarrow{\$} Y$, update $L_H \leftarrow L_H \cup (x, y)$, and **return** y .

During the simulation, the simulator records query-response pairs in a transcript, which it can later inspect to extract useful information (e.g., witness).

Quantum Random Oracle Model In the Quantum Random Oracle Model (QROM), the hash function is idealized as a publicly and quantum-accessible random oracle. Generally, the quantum computation corresponding to any function $H : X \rightarrow Y$ can be represented as a unitary transformation U_H :

$$|x\rangle_X |y\rangle_Y |H\rangle_H \rightarrow |x\rangle_X |y \oplus H(x)\rangle_Y |H\rangle_H,$$

where $x \in X$, $y \in Y$ and X , Y , and H denote the registers that store x , y , and H , respectively.

Let $\mathcal{H} := \{H | H : X \rightarrow Y\}$. A QRO can be defined analogously to the classical RO by selecting a function $H \xleftarrow{\$} \mathcal{H}$ uniformly at random from the family of functions. For each query $\sum_x \alpha_x |x, y_x\rangle$, the QRO performs the following operation:

$$O_H : \sum_x \alpha_x |x, y_x\rangle \otimes |H\rangle \rightarrow \sum_x \alpha_x |x, y_x \oplus H(x)\rangle \otimes |H\rangle.$$

and returns $\sum_x \alpha_x |x, y_x \oplus H(x)\rangle$.

In addition, we may consider another type of QRO, referred to as the Phase Oracle (PhO). For each basis $|x\rangle$ and $|y\rangle$, the PhO performs the following operation:

$$\text{PhO}_H : |x\rangle |y\rangle \otimes |H\rangle \rightarrow (-1)^{y \cdot H(x)} |x\rangle |y\rangle \otimes |H\rangle.$$

Lemma 2 ($O_H \Leftrightarrow \text{PhO}_H$ (Lemma 3. in [68])). *For any adversary $\mathcal{A}$ makes queries to O_H , let $\mathcal{B}$ be the adversary that is identical to $\mathcal{A}$, except it performs the Hadamard transformation $H^{\otimes n}$ to the response registers before and after each query. Then $\Pr[\mathcal{A}^{O_H}() \rightarrow 1] = \Pr[\mathcal{B}^{PhO_H}() \rightarrow 1]$.*

Remark 1. We introduce the Hadamard transform, which maps computational basis states to the Fourier basis. Let $Y = \{0, 1\}^n$. For each $y \in Y$, the n -bit Hadamard transform is defined as

$$H^{\otimes n} : |y\rangle \rightarrow |\hat{y}\rangle,$$

where $|\hat{y}\rangle = \frac{1}{\sqrt{2^n}} \sum_{y' \in \{0,1\}^n} (-1)^{y \cdot y'} |y'\rangle$. It is obvious that $H^{\otimes n}(|0\rangle) = |\tau\rangle$, where τ is the uniform superposition state.

From an equivalent perspective, the QRO can be initialized as a uniform superposition over all functions $\sum_H \frac{1}{\sqrt{|\mathcal{H}|}} |H\rangle$, where $\mathcal{H} := \{H : X \rightarrow Y\}$. This type of QRO is referred to as the Standard Oracle (StO), as formalized in Definition 1.

Definition 1 (StO [68]). *The StO oracle is initialized in a uniform superposition over all functions $\mathcal{H}$ with domain X and range Y . For any basis state $|x\rangle$ and $|y\rangle$, the oracle performs the following unitary operation:*

$$|x\rangle |y\rangle \otimes \sum_H \frac{1}{\sqrt{|\mathcal{H}|}} |H\rangle \rightarrow \sum_H \frac{1}{\sqrt{|\mathcal{H}|}} |x\rangle |y \oplus H(x)\rangle |H\rangle,$$

where $x \in X$, $y \in Y$, and $\mathcal{H}$ denotes the set of all functions with domain X and range Y .

For a function $H : X \rightarrow Y$, it can be represented as

$$H(x_0) \| H(x_1) \| \cdots \| H(x_{2^m-1}),$$

where $X := \{0, 1\}^m$ and $x_i \in X$ for $i \in \{0, 1, \dots, 2^m - 1\}$.

Based on this perspective, the internal state of the quantum oracle corresponding to such a function can be expressed as $\otimes_{x \in X} |H(x)\rangle$. In particular, the internal state of the StO can be represented by $\otimes_{x \in X} |\tau\rangle$. We redefine the unitary operation $\text{StO} := I \otimes \text{CNOT}_{Y, D_x} \otimes I^{\otimes 2^m}$:

$$\text{StO} : |x\rangle |y\rangle \otimes |\tau\rangle^{\otimes 2^m} \rightarrow \sum_{y_x \in \{0,1\}^n} |x\rangle |y \oplus y_x\rangle |y_x\rangle_{D_x} \otimes |\tau\rangle^{\otimes 2^m-1},$$

where D_x the register that stores the output corresponding to x , for each $x \in X$.

Remark 2. CNOT_{Y, D_x} is a unitary acting on Y and D_x defined as:

$$\text{CNOT}_{Y D_x} : |y, y_x\rangle_{Y D_x} \rightarrow |y \oplus y_x, y_x\rangle_{Y D_x}.$$

Compressed Random Oracle It can be observed that a query on $|x\rangle$ affects the register D_x , thereby entangling D_x with X and Y into a joint system. For simplicity, we therefore focus only on properties of the transformation $U_1 := I \otimes \text{CNOT}_{Y, D_x}$, where

$$U_1 : |x\rangle |y\rangle |y_x\rangle \rightarrow |x\rangle |y \oplus y_x\rangle |y_x\rangle.$$

Let $|\widehat{y}\rangle = H^{\otimes n} |y\rangle$ for each $y \in Y$. Next, we consider the following process:

$$\begin{aligned} |x\rangle |\widehat{y}\rangle |\widehat{y}_x\rangle &\xrightarrow{I \otimes I \otimes H^{\otimes n}} |x\rangle |\widehat{y}\rangle \otimes \sum_{\widehat{y}'_x} \frac{1}{\sqrt{2^n}} (-1)^{\widehat{y}_x \cdot \widehat{y}'_x} |\widehat{y}'_x\rangle \\ &\xrightarrow{\text{PhO}} |x\rangle (-1)^{\widehat{y} \cdot \widehat{y}'_x} |\widehat{y}\rangle \otimes \sum_{\widehat{y}'_x} \frac{1}{\sqrt{2^n}} (-1)^{\widehat{y}_x \cdot \widehat{y}'_x} |\widehat{y}'_x\rangle = |x\rangle |\widehat{y}\rangle \otimes \sum_{\widehat{y}'_x} (-1)^{(\widehat{y} \oplus \widehat{y}_x) \cdot \widehat{y}'_x} |\widehat{y}'_x\rangle \\ &\xrightarrow{I \otimes I \otimes H^{\otimes n}} |x\rangle |\widehat{y}\rangle \otimes \sum_{\widehat{y}'_x} \sum_{\widehat{y}''_x} (-1)^{(\widehat{y} \oplus \widehat{y}_x) \cdot \widehat{y}'_x} \cdot (-1)^{\widehat{y}'_x \cdot \widehat{y}''_x} |\widehat{y}''_x\rangle = |x\rangle |\widehat{y}\rangle |\widehat{y}_x \oplus \widehat{y}\rangle. \end{aligned} \quad (1)$$

Therefore, in the Fourier basis, we can define the unitary operation

$$\text{FStO} : |x\rangle |\widehat{y}\rangle |\widehat{y}_0\rangle \cdots |\widehat{y}_{2^m-1}\rangle \rightarrow |x\rangle |\widehat{y}\rangle |\widehat{y}_0\rangle \cdots |\widehat{y}_x \oplus \widehat{y}\rangle \cdots |\widehat{y}_{2^m-1}\rangle.$$

If D_x is initialized as $|\tau\rangle$, then its representation in the Fourier basis corresponds to $|\widehat{0}\rangle$. Then

$$\text{FStO} : |x\rangle |\widehat{y}\rangle |\widehat{0}\rangle \cdots |\widehat{0}\rangle \rightarrow |x\rangle |\widehat{y}\rangle |\widehat{0}\rangle \cdots |\widehat{0} \oplus \widehat{y}\rangle \cdots |\widehat{0}\rangle.$$

The above result shows that once the adversary makes quantum queries, the inputs and outputs become entangled with the oracle's state. This entanglement enables the recording of queries and responses. Building on this observation, Zhandry [68] formalized the idea through the Compressed Standard Random Oracle (CStO), which provides a framework for simulating the QRO.

Let us define

$$\text{Comp}_x := |\perp\rangle \langle \widehat{0}| + \sum_{\widehat{z} \neq 0} |\widehat{z}\rangle \langle \widehat{z}|, |y\rangle \rightarrow \begin{cases} |\perp\rangle & \text{if } \widehat{y} = \widehat{0} \\ |\widehat{y}\rangle & \text{if } \widehat{y} \neq \widehat{0}, \end{cases} \quad (2)$$

and $\text{Comp} = \otimes_x \text{Comp}_x$.

Definition 2 (CFStO [68, 22]). *The Compressed StO on the Fourier basis (CFStO) can be represented as follows:*

$$\text{CFStO} := \text{Comp} \circ \text{FStO} \circ \text{Comp}. \quad (3)$$

Due to the exponential storage space required, the above representation is inefficient. The StO can be efficiently represented by the techniques in [68].

Definition 3 (CStO [68, 22, 32]). Let $Y' := Y \cup \{\perp\}$. Consider the standard classical sparse encoding function **Encode**, which compresses any database $\mathbf{y} = \{y_x\}_{x \in X}$ with at most q non- $\perp$ entries to the $L_q = \{(x_1, y_1), \dots, (x_l, y_l), \perp\}$. Here, $y_x \in \{y_x\}_{x \in X}$ is sorted in L_q as (x, y_x) in the order of x if $y_x \neq \perp$. Then,

$$\text{Encode} : |\mathbf{y}\rangle \rightarrow |x_1, y_{x_1}\rangle \cdots |x_s, y_{x_s}\rangle |\perp\rangle$$

for any basis state $|\mathbf{y}\rangle$. The corresponding inverse operation is defined as **Decode**.

Let $D = \otimes_{x \in X} D_x$ be the state of oracle. The state space of D_x is given by $\mathcal{H}_{D_x} = \mathbb{C}[\{0, 1\}^n \cup \{\perp\}]$. The CStO can be defined as follow:

$$\text{CStO} := \text{Encode} \circ \text{Comp} \circ \mathbf{F} \circ \text{Comp} \circ \text{Decode},$$

where $\mathbf{F}$ is an unitary acting on X , Y , and D defined as:

$$\mathbf{F} : |x, y\rangle |D\rangle \rightarrow |x, y \oplus y_x\rangle |D\rangle, (x, y_x) \in D.$$

The CStO technique provides a perfect simulation of the QRO.

Lemma 3 (CStO $\Leftrightarrow$ StO [68]). CStO and StO are perfectly indistinguishable.

Lemma 4 (Lemma 2.6 in [32]). Let $\mathcal{A}$ be an oracle quantum algorithm that outputs $\mathbf{x} = (x_1, \dots, x_n) \in X^n$ and $z \in Z$. Let $\tilde{A}$ be an oracle quantum algorithm that runs $\mathcal{A}$, makes n classical queries on the outputs x_i to obtain $\mathbf{y} = H(\mathbf{x})$ then obtains $(\mathbf{x}, \mathbf{y}, z)$. When $\tilde{A}$ interacts with the compressed oracle instead, and at the end D is obtained by measuring the internal state of the compressed oracle, then, conditioned on $\tilde{A}$'s output $(\mathbf{x}, \mathbf{y}, z)$,

$$\Pr[\mathbf{y} = D(\mathbf{x}) | (\mathbf{x}, \mathbf{y}, z)] \geq 1 - \frac{2n}{|Y|}.$$

2.2 Database Property for QROM

For the CStO introduced above, we further define the following notations. The set of internal states of the RO during the query process is referred to as the database, denoted by D . We define $\mathcal{D} := \{D : X \rightarrow Y \cup \{\perp\}\}$.

Definition 4 (Database Property (Def 5.1 in [22])). A database property on $\mathcal{D}$ is a subset $\mathcal{P} \subseteq \mathcal{D}$ of the set of databases D .

Additionally, we introduce the following notations for database properties:

- For any database $D \in \mathcal{D}$, we define $D[x \rightarrow y] \in \mathcal{D}$ to be the database

$$D[x \rightarrow y](x') = D(x'), x' \neq x$$

and

$$D[x \rightarrow y](x) = y.$$

- The set of the database that coincide with D outside of x :

$$D|_x := \{D[x \rightarrow y] | y \in Y\} \subseteq \mathcal{D}.$$

- The restriction of database property $P \subseteq D$ to the databases in $D|_x$:

$$P|_{D|_x} := P \cap D|_x.$$

Definition 5 (Quantum Transition Capacity (Def 5.5 [22])). Let P, P' be two database properties. Then, the quantum transition capacity is defined as

$$\llbracket P \rightarrow P' \rrbracket := \max_{x, y, D} \|P'|_{D|_x} \text{FStO}_{x, y} P|_{D|_x}\|.$$

Furthermore,

$$\llbracket P \xrightarrow{q} P' \rrbracket := \sup_{U_1, \dots, U_{q-1}} \|P' \cdot \text{CStO} \cdot U_{q-1} \text{FStO} \cdots \text{cStO} \cdot U_1 \cdot \text{FStO} \cdot P\|.$$

Lemma 5 (Database Properties Sequence (Lemma 5.6 [32])). For any sequence of database properties $P_0, P_1, \dots, P_q$,

$$\llbracket \neg P_0 \xrightarrow{\$} P_q \rrbracket \leq \sum_{d=1}^q \llbracket \neg P_{s-1} \xrightarrow{\$} P_s \rrbracket.$$

We have the following theorem to bound quantum transition capacities.

Theorem 1 ([22, 31]). Let P and P' be database properties with trivial intersection, i.e., $P \cap P' = \emptyset$, and for every $D \in \mathcal{D}$ and $x \in X$ let

$$\mathbb{L}^{x, D} : \begin{cases} P|_{D|_x} & \text{if } \perp \in P'|_{D|_x} \\ P'|_{D|_x} & \text{if } \perp \in P|_{D|_x}, \end{cases} \quad (4)$$

with $\mathbb{L}^{x, D}$ being either of the two if $\perp \notin P|_{D|_x} \cup P'|_{D|_x}$. Then

$$\llbracket P \xrightarrow{\$} P' \rrbracket \leq \max_{x, D} \sqrt{10 \Pr[U \in \mathbb{L}^{x, D}]},$$

where U is uniform over Y , and the maximization can be restricted to $D \in \mathcal{D}$ and $x \in X$ for which both $P|_{D|_x}$ and $P'|_{D|_x}$ are non-empty.

Corollary 1 ([31]). Denote the database $\text{CL} := \{D : \exists x \neq x' \text{ s.t. } D(x) = D(x')\}$ for $\mathcal{D} : X \rightarrow Y$. Then,

$$\llbracket \text{SZ}_{\leq i} \setminus \text{CL} \rightarrow \text{CL} \rrbracket \leq 2e \sqrt{\frac{(i+1)}{|Y|}}.$$

2.3 Programming of Quantum Random Oracle

Based on the original OW2H lemma proposed by Unruh [62], some versions of OW2H are proposed to reduce the reduction loss. In this part, we introduce the SC-OW2H [1].

Lemma 6 (Semi-classical O2H, [1]). *Let $S \subseteq X$ be random. Let $G, H : X \rightarrow Y$ be random functions satisfying $\forall x \notin S, G(x)=H(x)$. Let z be a random bitstring. (S, G, H, z may have arbitrary joint distribution.) Let $\mathcal{A}$ be an oracle algorithm of query depth d . Let $\mathcal{B}^H$ be an oracle algorithm that on input z does the following: pick $i \xleftarrow{\$} \{1, \dots, d\}$, run $\mathcal{A}^H(z)$ untill (just before) the i -th query, measure all query input registers in the computations basis, output the set T of measurement outcomes. Let*

$$P_{\text{left}} := \Pr[b \leftarrow \mathcal{A}^H(z) : b = 1],$$

$$P_{\text{right}} := \Pr[b \leftarrow \mathcal{A}^G(z) : b = 1],$$

and

$$P_{\text{guess}} := \Pr[S \cap T \neq \emptyset : T \leftarrow \mathcal{B}^H(z)].$$

Then

$$|P_{\text{left}} - P_{\text{right}}| \leq 2d\sqrt{P_{\text{guess}}},$$

and

$$|\sqrt{P_{\text{left}}} - \sqrt{P_{\text{right}}}| \leq 2d\sqrt{P_{\text{guess}}}.$$

The same result holds with $\mathcal{B}^G$ instead of $\mathcal{B}^H$ in the definition of P_{guess} .

2.4 Legendre and Power Residue PRFs

The *Legendre PRF* relies on the conjectured pseudo-randomness of the Legendre symbol with a secret shift [13]. Specifically, on input $a \in \mathbb{F}_p$, the Legendre symbol outputs $\{-1, 0, 1\}$, depending on whether a is a square modulo p :

$$\left(\frac{a}{p}\right) = \begin{cases} 1, & \text{if } a = b^2 \pmod{p} \text{ for some } b \in \mathbb{F}_p^* \\ 0, & \text{if } a = 0 \pmod{p} \\ -1, & \text{otherwise} \end{cases}$$

Then, the Legendre PRF is defined as:

$$\mathcal{L}_0^2(a) = \left\lfloor \frac{1}{2} \left(1 - \left(\frac{a}{p}\right) \right) \right\rfloor \in \mathbb{Z}_2$$

Namely, the PRF outputs 1 if a is a quadratic non-residue (QNR) modulo p and 0 otherwise.

The *t -th power residue PRF* generalizes the Legendre PRF, which uses higher power residue symbols instead of quadratic residue symbols to increase the PRF throughput [26]. Specifically, for an integer t and a prime p such that $t|(p-1)$

1), the computation of the t -th power residue symbol $a \mapsto a^{(p-1)/t} \pmod{p}$ yields an output of $\log_2(t)$ -bits, in contrast to the single-bit output obtained by the Legendre PRF. Formally, the t -th power residue PRF is defined as follows (Definition 6).

Definition 6 (t -th Power Residue PRF). *Let p be an odd prime and t be an integer such that $p \equiv 1 \pmod{t}$. Let $g \in \mathbb{F}_p^*$ be the generator. We define the t -th power residue symbol $\mathcal{L}_K^t(\cdot) : \mathbb{F}_p \rightarrow \mathbb{Z}_t$ with the hidden shift K as*

$$\mathcal{L}_K^t(a) = \begin{cases} i, & \text{if } (a + K)/g^i \equiv z^t \pmod{p} \text{ for some } z \in \mathbb{F}_p^* \\ 0, & \text{if } (a + K) \equiv 0 \pmod{p} \end{cases}$$

Both Legendre and power residue PRFs have multiplicative homomorphism, that is $\mathcal{L}_0^t(a \cdot b) = \mathcal{L}_0^t(a) + \mathcal{L}_0^t(b)$ for non-zero $a, b \in \mathbb{F}_p^*$.

To simplify the notation, we extend the definition of the t -th power residue PRF to be ‘list-wise’. Namely, given a list $\mathcal{I} = \{I_0, \dots, I_{L-1}\} \in \mathbb{F}_p^L$, we denote the PRF evaluations as $\mathcal{L}_K^t(\mathcal{I}) = (\mathcal{L}_K^t(I_0), \dots, \mathcal{L}_K^t(I_{L-1})) \in \mathbb{Z}_t^L$. We define the NP-relations for the t -th power residue PRFs with respect to two different lists. The first list $\mathcal{I} = (I_0, \dots, I_{L-1})$ is defined the same as in LegRoast [15], where each element is uniformly random chosen from $\mathbb{F}_p$. The second list $\mathcal{I}' = (\mathcal{I}'_0, \dots, \mathcal{I}'_{z-1})$ is defined as, for each $j \in [0, z-1]$, $\mathcal{I}'_j = (a_j, a_j + b, \dots, a_j + (c-1)b)$, where $b \in \mathbb{F}_p^*$ is a fixed non-zero element, a_j is chosen uniformly random from the field, and $L = z \times c$.

Definition 7 (t -th Power Residue PRF Relation). *For an odd prime p , a positive integer t with $t|(p-1)$, and a list $\tilde{\mathcal{I}} \in \{\mathcal{I}, \mathcal{I}'\}$ of L field elements, we define the t -th power residue PRF relation $R_{\mathcal{L}^t}$ as*

$$R_{\mathcal{L}^t} = \{(\mathcal{L}_K^t(\tilde{\mathcal{I}}), K) \in \mathbb{Z}_t^L \times \mathbb{F}_p \mid K \in \mathbb{F}_p\}.$$

In order to enhance the efficiency of Legendre and t -th power residue PRFs based signature schemes, Beullens et al. [15] introduced a relaxed PRF relation, which can be extended with respect to the arithmetic progression list $\mathcal{I}'$ as follows:

Definition 8 (β -approximate PRF relation). *Given $\beta \in [0, 1]$, an odd prime p , a positive integer $t|(p-1)$, and a list $\tilde{\mathcal{I}} \in \{\mathcal{I}, \mathcal{I}'\}$ of L field elements, we define the β -approximate PRF relation $R_{\beta\mathcal{L}^t}$ as*

$$R_{\beta\mathcal{L}^t} = \{(s, K) \in \mathbb{Z}_t^L \times \mathbb{F}_p \mid \exists a \in \mathbb{Z}_t : d_H(s + (a, \dots, a), \mathcal{L}_K^t(\tilde{\mathcal{I}})) \leq \beta L\}$$

where $d_H(\cdot, \cdot)$ denotes the Hamming distance.

For uniformly random list $\mathcal{I}$, the β -approximate PRF relation is as hard as the t -th power residue PRF relation, which is shown in [15, Theorem 1]. We present and prove similar results for arithmetic progression list $\mathcal{I}'$.

Theorem 2. Let $\mathfrak{B}(n, q)$ denote the binomial distribution with n samples and the success probability q . Take $K \in \mathbb{F}_p$ and $s = \mathcal{L}_K^t(\mathcal{T})$. Then with probability at least

$$1 - 2.72 \cdot t(p-1) \cdot \Pr[\mathfrak{B}(L, 1/t) > (1-\beta)L]$$

over the choice of $(a_0, \dots, a_{z-1})$, there exists only one witness for $s \in R_{\beta \mathcal{L}^t}$, namely K , which is also a witness for the exact relation $R_{\mathcal{L}^t}$.

To prove the theorem, we first show that for a fixed non-zero $b \in \mathbb{F}_p^*$ and a uniformly random chosen $a \in \mathbb{F}_p$, the probability of

$$\mathcal{L}_0^t(a) = s_0, \mathcal{L}_0^t(a+b) = s_1, \dots, \mathcal{L}_0^t(a+(L-1)b) = s_{L-1}$$

for any $(s_0, \dots, s_{L-1}) \in \mathbb{Z}_t^{\mathcal{L}}$ is at most $\frac{1}{t^c} + O(\frac{1}{\sqrt{p}})$ (see Lemma 11). This establishes the *real* distribution of the PRF sequence over arithmetic progression lists. Then, we prove Theorem 2 by analyzing the probability under the *ideal* distribution of the sequences and switch from the ideal case to the real case using the Rényi divergence. The full proof is deferred to Appendix B.2.

In our signature scheme, we *only* use Legendre and power residue PRFs for *key generation*. The security of the signing and verification relies solely on the random oracle assumption. Currently, no quantum algorithm is known that can recover the secret key from public key symbols derived from these PRFs. For completeness, we present a short survey of existing attacks against these PRFs in Appendix A.

2.5 Vector Commitment Schemes

Batch all-but-one vector commitment (BAVC). Batch all-but-one vector commitment (BAVC) is a generalization of the tree-PRG-based vector commitment [7], which allows all-but- M commitments to be opened efficiently. At a high-level, tree-PRG-based vector commitment expands a single master seed $sd_0 \in \{0, 1\}^\lambda$ into $L = 2^d$ seeds using a tree-like structure. Concretely, given a length-doubling pseudorandom generator $\text{PRG} : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{2\lambda}$, the master seed sd_0 (layer 0) is expanded into two children seeds (layer 1):

$$(sd_{1,0}, sd_{1,1}) \leftarrow \text{PRG}(sd_0).$$

These layer 1 seeds are then used to generate next level of seeds in layer 2:

$$\begin{aligned} (sd_{1,0,0}, sd_{1,0,1}) &\leftarrow \text{PRG}(sd_{1,0}), \\ (sd_{1,1,0}, sd_{1,1,1}) &\leftarrow \text{PRG}(sd_{1,1}). \end{aligned}$$

The process continues until the bottom level with L leaf nodes is reached. Each leaf node is then committed using a PRG and collision-resistant hash function (CRHF) $H_T : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{3\lambda}$, which generates λ -bit message and 2λ -bit commitment. Finally, the commitments are aggregated using another CRHF

$H_{\text{com}} : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$, where both hashes are modeled as independent random oracles. To avoid cumbersome notation, we use the generic symbol $\mathcal{H}(\cdot)$ when referring to random oracles in BAVC.

In the original tree-PRG based vector commitment, revealing $L - 1$ messages requires only $d = \log(L)$ nodes, similar to the authentication path (with respect to the unopened leaf) in a Merkle tree. Since a signature protocol typically needs multiple trees (say M), BAVC combines them into one big tree and interleaves their leaf nodes, hence creating overlaps when opening all-but- M leaves and reducing the overall opening size. In this work, we separate the tree generation and leaf commitment for BAVC to facilitate cleaner security proofs of our scheme in the (Q)ROM. We begin by presenting the syntax of the BAVC, followed by the definition of its security properties.

Definition 9 (BAVC Syntax). *Let PRG be a length-doubling pseudorandom generator and $\mathcal{H}$ be a random oracle. The batch all-but-one vector commitment (BAVC) with message space $\mathcal{M}$ in the random oracle model is defined by the following PPT algorithms:*

- $\text{BAVC.pp} \leftarrow \text{BAVC.Setup}(1^\lambda)$ *On input the security parameter λ in unary, the algorithm outputs the public parameters BAVC.pp.*
- $(\text{decom}, \mathbf{L}) \leftarrow \text{BAVC.TreeGen}^{\text{PRG}}(\text{BAVC.pp})$ *On input public parameters BAVC.pp, the algorithm outputs the opening information decom and leaf nodes $\mathbf{L} = (l_{e,0}, \dots, l_{e,L-1})_{e \in [M]}$.*
- $(\text{com}, \mathbf{R}) \leftarrow \text{BAVC.LeafCom}^{\mathcal{H}}(\mathbf{L}, \text{BAVC.pp})$ *On input the list of leaf nodes $\mathbf{L} = (l_{e,0}, \dots, l_{e,L-1})_{e \in [M]}$ and public parameters BAVC.pp, the algorithm outputs a commitment com and messages $\mathbf{R} = (r_{e,0}, \dots, r_{e,L-1})_{e \in [M]}$ where $r_{e,\ell} \in \mathcal{M}$ for all $e \in [M]$ and $\ell \in [0, L - 1]$.*
- $(\perp \vee \text{decom}_S) \leftarrow \text{BAVC.Open}^{\text{PRG}, \mathcal{H}}(\text{decom}, S, \text{BAVC.pp})$ *On input the opening information decom and a set of indices $S := \{\bar{\ell}_1, \dots, \bar{\ell}_M\}$, where $\bar{\ell}_e \in [0, L - 1]$ for all $e \in [M]$, the algorithm either outputs $\perp$ or the opening decom_S for all-but- M messages.*
- $(\perp \vee (\text{com}, \bar{\mathbf{R}})) \leftarrow \text{BAVC.Verify}^{\text{PRG}, \mathcal{H}}(\text{decom}_S, S, \text{BAVC.pp})$ *On input the opening decom_S and the indices $S := \{\bar{\ell}_1, \dots, \bar{\ell}_M\}$, the algorithm either outputs the commitment com and messages $\bar{\mathbf{R}} := ((r_{e,\ell})_{\ell \in [0, L-1], \ell \neq \bar{\ell}_e})_{e \in [M]}$ (accept the opening) or $\perp$ (reject the opening).*

Remark 3 (Commitment Generation). In $\text{BAVC.LeafCom}^{\mathcal{H}}(\mathbf{L}, \text{BAVC.pp})$, each leaf node in $\mathbf{L}$ is first hashed with H_T to produce λ -bit message and 2λ -bit commitment. After interleaving, this yields the tuple $(\text{com}_{e,0}, \dots, \text{com}_{e,L-1})_{e \in [M]}$. All these commitments are then aggregated into a single commitment by hashing with H_{com} : $\text{com} \leftarrow H_{\text{com}}((\text{com}_{e,0}, \dots, \text{com}_{e,L-1})_{e \in [M]})$. This differs slightly from the original BAVC definition, where the commitments for each $e \in [M]$ are first compressed via $\text{com}_e \leftarrow H_{\text{com}}^{(e)}(\text{com}_{e,0}, \dots, \text{com}_{e,L-1})$. Then the M commitments are aggregated: $\text{com} \leftarrow H_{\text{com}}(\text{com}_1, \dots, \text{com}_M)$. While the intermediate hash ($H_{\text{com}}^{(e)}$) yields a cleaner construction, it complicates the security proof in the QROM. To allow a cleaner analysis, we eliminate the intermediate hashing step.

Following the definition in [6], correctness allows the sender to abort on certain inputs S during BAVC.Open (e.g., when the opening size exceeds a threshold). In our scheme, the abort probability remains small so that it does not pose any problem.

Definition 10 (Correctness with aborts (adapted from [6])). *BAVC is correct with aborts if for all $\text{BAVC.pp} = \text{BAVC.Setup}(1^\lambda)$ which is an implicit input to all algorithms below, $S = \{\ell_e\}_{e \in [M]}$ where $\ell_e \in [0, L - 1]$, and $\lambda \in \mathbb{N}$, the following holds:*

$$\Pr \left[\begin{array}{l} \text{decom}_S = \perp \vee (\text{com}, \bar{\mathbf{R}}) \\ = \text{BAVC.Verify}^{\text{PRG}, \mathcal{H}}(\text{decom}_S, S) \end{array} \middle| \begin{array}{l} (\text{decom}, \mathbf{L}) \leftarrow \text{BAVC.TreeGen}^{\text{PRG}}() \\ (\text{com}, \mathbf{R}) \leftarrow \text{BAVC.LeafCom}^{\mathcal{H}}(\mathbf{L}) \\ \text{decom}_S \leftarrow \text{BAVC.Open}(\text{decom}, S) \end{array} \right] = 1 - \text{negl}(\lambda).$$

Remark 4 (Handling the Abort of BAVC). In the BAVC-based commitment-and-open protocol, the algorithm may abort (i.e., $\text{decom}_S = \perp$) if the opening size induced by the challenge S exceeds the threshold. To handle this, we let the verifier resample the challenge set until it obtains an S such that $\text{decom}_S \neq \perp$. Formally, this restricts the challenge distribution from the full uniform distribution over $[0, L - 1]^M$ to the uniform distribution over the non-aborting subset. The statistical distance between these two distributions is at most the abort probability $\alpha(\lambda)$. For sufficiently large L and M , $\alpha(\lambda)$ is negligible. Thus, in our proofs, we may assume without loss of generality that $\text{decom}_S \neq \perp$, and absorbs $\alpha(\lambda)$ into the negligible error terms of BAVC.

A commitment scheme is extractable binding (Definition 11) if there exists an extractor such that for any commitment opening, the extracted messages is equal to the opened message.

Definition 11 (Extractable Binding). *Let BAVC be a vector commitment defined in Definition 9. We say that BAVC satisfies extractable binding in the (Q)ROM if the following holds. For any $\text{BAVC.pp} \leftarrow \text{BAVC.Setup}(1^\lambda)$ (implicitly given as input to the following algorithms), there exists a (quantum) PPT extractor $\mathcal{E}_{\text{BAVC}}$ such that for any (quantum) PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ with shared state, the following properties are satisfied:*

- **Simulation Indistinguishability.** *There exists a negligible function $\epsilon_{\text{sim}}(\cdot)$ such that*

$$\text{Dis}(\{(\text{com}_0, \bar{\mathbf{R}}_0, \text{decom}_{S_0}, S_0)\}, \{(\text{com}_1, \bar{\mathbf{R}}_1, \text{decom}_{S_1}, S_1)\}) \leq \epsilon_{\text{sim}}(\lambda),$$

where

$$\text{com}_0 \leftarrow \mathcal{A}_1^{\mathcal{H}}(1^\lambda), \quad (\bar{\mathbf{R}}_0, \text{decom}_{S_0}, S_0) \leftarrow \mathcal{A}_2^{\mathcal{H}}(\text{com}_0),$$

and

$$\text{com}_1 \leftarrow \mathcal{A}_1^{\mathcal{E}_{\text{BAVC}}}(1^\lambda), \quad (\bar{\mathbf{R}}_1, \text{decom}_{S_1}, S_1) \leftarrow \mathcal{A}_2^{\mathcal{E}_{\text{BAVC}}}(\text{com}_1).$$

– **Extractability.** There exists a negligible function $\epsilon_{ext}(\cdot)$ such that

$$\Pr[\text{com}' = \text{com} \wedge \bar{\mathbf{R}}' = \bar{\mathbf{R}} \wedge \text{rela}(\mathbf{R}^*, \bar{\mathbf{R}}', S) = 1] \leq \epsilon_{ext}(\lambda).$$

Here, $\text{com}, \mathbf{R}^* = ((r_{e,\ell})_{\ell \in [0, L-1]})_{e \in [M]}$, $\bar{\mathbf{R}} = ((r_{e,\ell})_{\ell \in [0, L-1] \setminus \{\bar{\ell}_e\}})_{e \in [M]}$, and $S = (\bar{\ell}_e)_{e \in [M]}$ are generated as follows:

$$\text{com} \leftarrow \mathcal{A}_1^{\mathcal{E}_{\text{BAVC}}}(1^\lambda), \quad \mathbf{R}^* \leftarrow \mathcal{E}_{\text{BAVC}}(\text{com}), \quad (\bar{\mathbf{R}}, \text{decom}_S, S) \leftarrow \mathcal{A}_2^{\mathcal{E}_{\text{BAVC}}}(\text{com}),$$

and

$$(\text{com}', \bar{\mathbf{R}}') \leftarrow \text{BAVC.Verify}(\text{decom}_S, S),$$

with

$$\bar{\mathbf{R}}' = ((r'_{e,\ell})_{\ell \in [0, L-1] \setminus \{\bar{\ell}_e\}})_{e \in [M]}.$$

Finally, the predicate rela is defined as

$$\text{rela}(\mathbf{R}^*, \bar{\mathbf{R}}', S) = \|\exists e \in [M], \ell \in [0, L-1] \setminus \{\bar{\ell}_e\} : r_{e,\ell} \neq r'_{e,\ell}\|.$$

In the selective hiding game, the adversary has to guess if claimed committed values at the hidden index are randomly generated or not.

Definition 12 (Selective Hiding). Let PRG be a length-doubling pseudorandom generator and $\mathcal{H}$ be the (quantum) random oracle. The selective hiding experiment for BAVC with $M, L = \text{poly}(\lambda)$ is defined as follows:

1. $\text{BAVC.pp} \leftarrow \text{BAVC.Setup}(1^\lambda)$, we assume BAVC.pp is implicitly input to all following algorithms.
2. $b \xleftarrow{\$} \{0, 1\}$.
3. $(\text{decom}, \mathbf{L}) \leftarrow \text{BAVC.TreeGen}^{\text{PRG}}()$ where $\mathbf{L} = (l_{e,0}, \dots, l_{e,L-1})_{e \in [M]}$.
4. $(\text{com}, \mathbf{R}) \leftarrow \text{BAVC.LeafCom}^{\mathcal{H}}(\mathbf{L})$, where $\mathbf{R} = (r_{e,0}^*, \dots, r_{e,L-1}^*)_{e \in [M]}$.
5. $S \leftarrow \mathcal{A}^{\text{PRG}, \mathcal{H}}(\text{com})$, where $S = \{\bar{\ell}_1, \dots, \bar{\ell}_M\}$ for $\bar{\ell}_e \in [0, L-1]$.
6. $\text{decom}_S \leftarrow \text{BAVC.Open}^{\text{PRG}, \mathcal{H}}(\text{decom}, S)$.
7. $r_{e,\ell} \leftarrow r_{e,\ell}^*$ for all $\ell \in [0, L-1], \ell \neq \bar{\ell}_e$ and $e \in [M]$.
8. Set $r_{e,\bar{\ell}_e} \xleftarrow{\$} \mathcal{M}$ if $b = 0$. Otherwise ($b = 1$), set $r_{e,\bar{\ell}_e} \leftarrow r_{e,\bar{\ell}_e}^*$.
9. $b' \leftarrow \mathcal{A}(((r_{e,\ell})_{\ell \in [0, L-1]})_{e \in [M]}, \text{decom}_S)$.
10. Output 1 (success) if $b' = b$, else 0 (failure).

The advantage of $\mathcal{A}$ ($\text{AdvHD}_{\mathcal{A}}^{\text{BAVC}}$) is defined as the $\Pr[\mathcal{A} \text{ wins}] - \frac{1}{2}$ in the hiding experiment. We say BAVC satisfies selective hiding if for any (quantum) PPT adversary $\mathcal{A}$, the advantage $\text{AdvHD}_{\mathcal{A}}^{\text{BAVC}}$ is negligible.

In this paper, we prove the extractable binding and selective hiding of BAVC in the (Q)ROM, which is deferred to Appendix B.1.

Merkle-Tree-based Vector Commitment We use the Merkle-tree-based vector commitment [52] in our construction of PEGARING. Below, we present a generalized Merkle-tree-based vector commitment that supports the opening of M leaf nodes, rather than just a single leaf node.

Definition 13 (Merkle-Tree Vector Commitment). *Let $\mathcal{H}$ be a random oracle. The Merkle-tree-based vector commitment with message space $\mathcal{M}$ is defined by the following PPT algorithms:*

- $\text{com}_{\text{MT}} \leftarrow \text{MT.Commit}(\mathbf{V})$: On input a vector $\mathbf{V} = (v_{e,0}, \dots, v_{e,z-1})_{e \in [M]}$, where $v_{e,i} \in \mathcal{M}$ for all $i \in [0, z-1]$ and $e \in [M]$, the algorithm outputs a short commitment com_{MT} .
- $(\mathbf{V}_I, \text{path}_I) \leftarrow \text{MT.Open}(I, \mathbf{V})$: On input a set of indices $I := \{\bar{i}_1, \dots, \bar{i}_M\}$, where $\bar{i}_e \in [0, z-1]$ and the vector $\mathbf{V}$ that was committed using MT.Commit , the deterministic algorithm outputs M leaf nodes $\mathbf{V}_I = (v_{e,\bar{i}_e})_{e \in [M]}$ and authentication paths path_I .
- $(\perp \vee \text{com}_{\text{MT}}) \leftarrow \text{MT.Verify}(\mathbf{V}_I, \text{path}_I, I)$: On input leaf nodes $\mathbf{V}_I = (v_{e,\bar{i}_e})_{e \in [M]}$, the corresponding authentication paths path_I , and indices $I = \{\bar{i}_1, \dots, \bar{i}_M\}$, where $\bar{i}_e \in [0, z-1]$, the deterministic algorithm outputs com_{MT} (accept the opening) if and only if $\mathbf{V}_I$ contains valid leaf nodes with respect to the path path_I . Otherwise, outputs $\perp$ (reject the opening).

Similar to BAVC, Merkle-tree based vector commitment can be combined to one tree such that open multiple leaves result in shorter proofs. The proof can abort if the opening exceeds a certain threshold. The binding of Merkle tree follows the collision-resistant property of the hash function. In this work, we use the Merkle tree only as a binding aggregator, meaning that we neither require extraction from the nodes nor rely on its hiding property.

2.6 Commit-and-Open Σ -Protocol

We construct commit-and-open (C&O) Σ -protocol for β -approximate relation $\mathcal{R}_{\beta\mathcal{L}^t}$ as defined in Definition 8 and set membership proof of the relation $\mathcal{R}_{\beta\mathcal{L}^t}^{\text{OR}}$ defined as follows:

$$\mathcal{R}_{\beta\mathcal{L}^t}^{\text{OR}} = \{(pk_1, \dots, pk_n; sk) : \exists \pi \in [n], (pk_\pi, sk) \in \mathcal{R}_{\beta\mathcal{L}^t}\}.$$

In this section, we recall the syntax of C&O Σ -protocol, along with the security definitions, including completeness, proof-of-knowledge with online extraction (PoK-OE), and special honest verifier zero-knowledge (SHVZK).

Definition 14 (Commit-and-Open Σ -Protocol). *Let $\mathcal{R} = \{R_\lambda\}$ be a set of binary relations and let $\mathcal{H} : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$ denote a random oracle. A commit-and-open (C&O) Σ -protocol $\Pi^{\mathcal{H}}$ for $\mathcal{R}$ is a tuple of PPT algorithms $(\text{Setup}, \text{KeyGen}, \mathcal{P}, \mathcal{V})$ defined as follows:*

- $\text{pp} \leftarrow \text{Setup}(1^\lambda)$: On input the security parameter 1^λ , output public parameters pp .

- $(pk, sk) \leftarrow \text{KeyGen}(\text{pp})$: On input pp , output a public-secret key pair $(pk, sk) \in R_\lambda$.
- $\mathcal{P}^\mathcal{H} = (P_1^\mathcal{H}, P_2^\mathcal{H})$: The prover is a two-phase, stateful oracle algorithm that may query $\mathcal{H}$.
 - $((A, A_o), \text{st}_P) \leftarrow P_1^\mathcal{H}(pk, sk)$: On input (pk, sk) , output commitments A , auxiliary information A_o , and prover state st_P .
 - $Z \leftarrow P_2^\mathcal{H}(\text{st}_P, (A, A_o), S)$: On input $(\text{st}_P, (A, A_o), S)$, output a response Z that opens the commitments in A with respect to challenge S .
- $\mathcal{V}^\mathcal{H} = (V_1^\mathcal{H}, V_2^\mathcal{H})$: The verifier is a two-phase, stateful oracle algorithm that may query $\mathcal{H}$.
 - $(S, \text{st}_V) \leftarrow V_1^\mathcal{H}(pk, (A, A_o))$: On input $(pk, (A, A_o))$, output a challenge S and verifier state st_V .
 - $b \leftarrow V_2^\mathcal{H}(\text{st}_V, pk, (A, A_o), S, Z)$: On input $(\text{st}_V, pk, (A, A_o), S, Z)$, output $b \in \{0, 1\}$ indicating accept or reject.

As in the classical case, when the challenge S is selected uniformly at random, the verifier can be simplified to a single-stage PPT oracle algorithm $\mathcal{V}^\mathcal{H}$. For any $(pk, sk) \leftarrow \text{KeyGen}(\text{pp})$, the transcript

$$(b, (A, A_o), S, Z) = [\mathcal{P}^\mathcal{H}(pk, sk) \leftrightarrow \mathcal{V}^\mathcal{H}(pk)]$$

denotes the information exchanged between the prover and verifier throughout the protocol in the presence of the random oracle $\mathcal{H}$.

Definition 15 (Completeness). Let $\Pi^\mathcal{H} = (\text{Setup}, \text{KeyGen}, \mathcal{P}^\mathcal{H}, \mathcal{V}^\mathcal{H})$ be a Commit-and-Open Σ -protocol in the (Q) ROM for $\mathcal{R} = \{R_\lambda\}_{\lambda \in \mathbb{N}}$. $\Pi^\mathcal{H}$ is correct if for any $\text{pp} \leftarrow \text{Setup}(1^\lambda)$ and any (quantum) PPT algorithm $\mathcal{A}$,

$$\Pr[(pk, sk) \in R_\lambda \wedge b = 0] \leq \text{negl}(\lambda),$$

where $(pk, sk) \leftarrow \mathcal{A}(\text{pp})$ and $(b, (\text{com}, A_o), S, Z) = [\mathcal{P}^\mathcal{H}(pk, sk) \leftrightarrow \mathcal{V}^\mathcal{H}(pk)]$.

Definition 16 (PoK-OE). We say that the protocol $\Pi^\mathcal{H}$ satisfies the proof of knowledge with online extraction (PoK-OE) property in the (Q) ROM if for any $\text{pp} \leftarrow \text{Setup}(1^\lambda)$ and $(pk, sk) \leftarrow \text{KeyGen}(\text{pp})$, there exists a (quantum) PPT extractor $\mathcal{E}$ such that for any (quantum) PPT algorithm $\mathcal{P}^*$:

- **(Simulation Indistinguishability).** The extractor $\mathcal{E}$ can simulate $\mathcal{H}$, and this simulation is statistically indistinguishable from the real one.

$$\text{Dis}(\{(b, (\text{com}, A_o), S, Z)\}, \{(b', (\text{com}', A'_o), S', Z')\}) \leq \epsilon_{\text{sim}}(\lambda),$$

where

$$(b, (\text{com}, A_o), S, Z) = [\mathcal{P}^{*\mathcal{H}}(pk) \leftrightarrow \mathcal{V}^\mathcal{H}(pk)],$$

and

$$(b', (\text{com}', A'_o), S', Z') = [\mathcal{P}^{*\mathcal{E}}(pk) \leftrightarrow \mathcal{V}^\mathcal{E}(pk)].$$

- **(Extractability)**. In the interaction with the adversary $\mathcal{P}^*$ under the simulated random oracle $\mathcal{H}$, the extractor relies on this interaction together with the valid transcript generated by $\mathcal{P}^*$ to extract a witness.

$$\Pr[(pk, sk) \notin R_\lambda \wedge b = 1] \leq \epsilon_{ext}(\lambda),$$

where

$$(b, (\text{com}, A_o), S, Z) = [\mathcal{P}^{*\mathcal{E}}(pk) \leftrightarrow \mathcal{V}^{\mathcal{E}}(pk)]$$

and

$$sk \leftarrow \mathcal{E}(pk, (\text{com}, A_o), S, Z).$$

Here, ϵ_{sim} and ϵ_{ex} are negligible in λ .⁸

Definition 17 (SHVZK). Let $\Pi^{\mathcal{H}} = (\text{Setup}, \text{KeyGen}, \mathcal{P}^{\mathcal{H}}, \mathcal{V}^{\mathcal{H}})$ be a CStO Σ -protocol in the (Q)ROM for $\mathcal{R} = \{R_\lambda\}_{\lambda \in \mathbb{N}}$. $\Pi^{\mathcal{H}}$ is special honest verifier zero-knowledge (SHVZK) in the (Q)ROM, if there exists a (quantum) PPT simulator Sim such that for any (quantum) PPT algorithm $\mathcal{A}$,

$$\text{Adv}_{\Pi^{\mathcal{H}}, \mathcal{A}}^{\text{SHVZK}}(1^\lambda) := \left| \Pr[\text{SHVZK}(1^\lambda) \rightarrow 1] - \frac{1}{2} \right| \leq \text{negl}(\lambda),$$

where the SHVZK game is defined in Fig. 1.

Game SHVZK(1^λ)	getTransCh(sk, pk, S)
1 : $\text{pp} \leftarrow \text{Setup}(1^\lambda)$	1 : $((A, A_o), \text{state}_P) \leftarrow P_1^{\mathcal{H}}(pk, sk)$
2 : $(pk, sk) \leftarrow \text{KeyGen}(\text{pp})$	2 : $Z \leftarrow P_2^{\mathcal{H}}(\text{state}_P, (A, A_o), S)$
3 : $S \xleftarrow{\$} \mathcal{CH}, b \xleftarrow{\$} \{0, 1\}$	3 : return $((A, A_o), S, Z)$
4 : $\text{tr}^0 \leftarrow \text{getTransCh}(sk, pk, S)$	
5 : $\text{tr}^1 \leftarrow \text{Sim}^{\mathcal{H}}(pk, S)$	
6 : $b' \leftarrow \mathcal{A}^{\mathcal{H}}(pk, \text{tr}^b)$	
7 : return $\ b' = b\ $	

Fig. 1: SHVZK Game

2.7 Digital Signature

Definition 18 (Syntax). A signature scheme consists of the following algorithms $(\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Verify})$:

⁸ In this paper, we use lazy sampling and CStO to simulate classical random oracles and quantum respectively, which is perfect. Hence $\epsilon_{sim} = 0$, and will be omitted in the following analysis.

1. $(pp) \leftarrow \text{Setup}(1^\lambda)$. On input a security parameter λ in unary, the algorithm generates a set of public parameters denoted as pp .
2. $(sk, pk) \leftarrow \text{KeyGen}(pp)$: On input public parameters pp generated in the setup phase, the algorithm outputs a secret and public key pair (sk, pk) .
3. $\sigma \leftarrow \text{Sign}(sk, \text{msg}, pp)$: On input a secret key sk , a message msg , and public parameters pp , the (randomized) algorithm outputs a signature σ .
4. $(0/1) := \text{Verify}(pk, \text{msg}, \sigma, pp)$: On input a public key pk , a message msg , a signature σ , and public parameters pp , the (deterministic) algorithm outputs 1 (accept) or 0 (reject).

We define the perfect correctness of the signature scheme.

Definition 19 (Perfect Correctness). For any message msg and any $pp \leftarrow \text{Setup}(1^\lambda)$,

$$\Pr \left[\text{Verify}(pk, \text{msg}, \sigma, pp) = 1 \mid \begin{array}{l} (pk, sk) \leftarrow \text{KeyGen}(pp) \\ \sigma \leftarrow \text{Sign}(sk, \text{msg}, pp) \end{array} \right] = 1$$

In our security proof of PEGASUS, we first show that our signature scheme achieves existential unforgeability in the key only model (EUF-KO), where the adversary is limited to using only the public key to attempt a forgery.

Definition 20 (EUF-KO). Given a security parameter λ , public parameters $pp \leftarrow \text{Setup}(1^\lambda)$, and a random oracle $\mathcal{H}$, we say that the signature scheme is EUF-KO-secure in the random oracle model, if any PPT algorithm $\mathcal{A}$ has negligible advantage $\text{Adv}_{\mathcal{A}}^{\text{EUF-KO}}$ in the EUF-KO game, where $\text{Adv}_{\mathcal{A}}^{\text{EUF-KO}}$ is defined as

$$\Pr \left[\text{Verify}(pk, \text{msg}^*, \sigma^*, pp) = 1 \mid \begin{array}{l} (sk, pk) \leftarrow \text{KeyGen}(pp) \\ (\text{msg}^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{H}}(pk, pp) \end{array} \right]$$

where $\mathcal{A}^{\mathcal{H}}$ denotes $\mathcal{A}$'s access to the random oracle.

We then extend our proof to existential unforgeability against chosen message attack (EUF-CMA) by allowing the adversary to make signing oracle queries, and showing the reduction from EUF-CMA to EUF-KO.

Definition 21 (EUF-CMA). Given a security parameter λ , public parameters $pp \leftarrow \text{Setup}(1^\lambda)$, and a random oracle $\mathcal{H}$, we say that the signature scheme is EUF-CMA-secure in the random oracle model if any PPT algorithm $\mathcal{A}$ has negligible advantage $\text{Adv}_{\mathcal{A}}^{\text{EUF-CMA}}$ in the EUF-CMA game, where $\text{Adv}_{\mathcal{A}}^{\text{EUF-CMA}}$ is defined as

$$\Pr \left[\begin{array}{l} \text{Verify}(pk, \text{msg}^*, \sigma^*, pp) = 1 \\ \wedge \text{msg}^* \notin Q_s \end{array} \mid \begin{array}{l} (sk, pk) \leftarrow \text{KeyGen}(pp) \\ (\text{msg}^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{H}, \text{Sign}}(pk, pp) \end{array} \right]$$

where $\mathcal{A}^{\mathcal{H}, \text{Sign}}$ denotes $\mathcal{A}$'s access to the random oracle and signing oracle, respectively. Furthermore, we define Q_s as the set of messages that are queried to the signing oracle by $\mathcal{A}$.

2.8 Ring Signature

Let $\mathbb{PK} = (pk_1, pk_2, pk_3, \dots)$ denote the set of all public keys added to the system and a ring of size n is denoted as $\mathcal{PK} \subseteq \mathbb{PK}$ such that $|\mathcal{PK}| = n$.

Definition 22 (Syntax). A ring signature scheme is a tuple of PPT algorithms (R-Setup, R-KeyGen, R-Sign, R-Verify) that are described as follows:

- $\text{R-pp} \leftarrow \text{R-Setup}(1^\lambda)$: On input a security parameter λ in unary, outputs a set of public parameters denoted as R-pp .
- $(pk, sk) \leftarrow \text{R-KeyGen}(\text{R-pp})$: On input the public parameters generated from Setup, outputs a public key pk and the corresponding secret key sk .
- $\sigma \leftarrow \text{R-Sign}(\mathcal{PK}, \text{msg}, sk_\pi, \text{R-pp})$: On input a list $\mathcal{PK} \subseteq \mathbb{PK}$ of n public keys, a message msg , and a secret key sk_π corresponds to pk_π where $pk_\pi \in \mathcal{PK}$, the (randomized) algorithm outputs a signature σ .
- $0/1 := \text{R-Verify}(\mathcal{PK}, m, \sigma)$: On input a list $\mathcal{PK}$ of n public keys, a message msg and a ring signature σ , the (deterministic) algorithm outputs 0 (reject) if the ring signature is invalid with respect to the message msg and the ring $\mathcal{PK}$. Otherwise, output 1 (accept).

We require the standard correctness condition, where for any security parameter λ , public parameters $\text{R-pp} \leftarrow \text{R-Setup}(1^\lambda)$, a list of secret and public keys $\{(sk_i, pk_i)\}_{i \in [n]}$ output by $\text{R-KeyGen}(\text{R-pp})$, signer index $\pi \in [n]$, and message msg , we have $1 = \text{R-Verify}(\mathcal{PK}, \text{msg}, \text{R-Sign}(\mathcal{PK}, \text{msg}, sk_\pi, \text{R-pp}), \text{R-pp})$, where $\mathcal{PK} = (pk_1, \dots, pk_n)$.

We assume the adversary has (classical) access to corruption and signing oracles, following the assumption in [41]. We leave the quantum access to signing and corruption oracles (such as those formalized in [20]) as future works.

Definition 23 (Unforgeability w.r.t. Insider Corruption). For any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$, a ring signature scheme is said to satisfy unforgeability with respect to insider corruption if $\mathcal{A}$ has at most negligible advantage ($\text{Adv}_{\mathcal{A}}^{\text{EUF}}(1^\lambda)$) in winning the following game:

- **Setup.** The challenger runs $\text{R-pp} \leftarrow \text{R-Setup}(1^\lambda)$ and generates key pairs $(sk_i, pk_i) \leftarrow \text{R-KeyGen}(\text{R-pp})$ for all $i \in [N]$. It sends the set of public keys $\mathbb{PK} = (pk_1, \dots, pk_N)$ to $\mathcal{A}$ and initializes two empty lists, Q_{SO} and Q_{CO} .
- **Oracle queries.** The adversary $\mathcal{A}$ may adaptively issue polynomially many queries to the signing oracle O_{sig} and the corruption oracle O_{cor} :
 - For each signing query $(\text{msg}, \mathcal{PK}, pk_i)$, if $pk_i \notin \mathcal{PK}$ or $\mathcal{PK} \not\subseteq \mathbb{PK}$, the oracle O_{sig} returns $\perp$. Otherwise, it computes $\sigma \leftarrow \text{R-Sign}(\mathcal{PK}, \text{msg}, sk_i, \text{R-pp})$ and returns σ . The challenger then records $(pk_i, \mathcal{PK}, \text{msg})$ in Q_{SO} .
 - For each corruption query i , if $pk_i \notin \mathbb{PK}$, the oracle O_{cor} returns $\perp$. Otherwise, it returns sk_i . The challenger then records pk_i in Q_{CO} .
- **Forgery.** Finally, $\mathcal{A}$ outputs $(\mathcal{PK}^*, \text{msg}^*, \sigma^*)$. The adversary wins if the following hold simultaneously:
 - $\mathcal{PK}^* \subseteq \mathbb{PK} \setminus Q_{CO}$ (the ring only contains uncorrupted public keys),

Algorithm 1: Setup

```

1   $\text{pp} \leftarrow \text{Setup}(1^\lambda)$ 
2  On input a security parameter  $\lambda$ , the algorithm computes the
   following public parameters:
3    - A large odd prime  $p$ ,
4    - An integer  $t \geq 2$  such that  $t|(p-1)$ ,
5    - An integer  $L$  (the number of PRF symbols in the public key),
6    - A list  $\mathcal{I} = (I_0, \dots, I_{L-1})$  where  $I_\ell$  are randomly chosen from  $\mathbb{F}_p$ 
   for all  $\ell \in [0, L-1]$ ,
7    - An integer  $M$  (the number of parallel executions).
8    - Parameters  $\text{BAVC.pp}$  generated by executing  $\text{BAVC.Setup}(1^\lambda)$ .
9  Output  $\text{pp} = (p, t, L, M, \mathcal{I}, \text{BAVC.pp})$ 

```

- $(\cdot, \mathcal{PK}^*, \text{msg}^*) \notin Q_{SO}$ (the message-ring pair was never queried to the signing oracle), and
- $1 = \text{R-Verify}(\mathcal{PK}^*, \text{msg}^*, \sigma^*, \text{R-pp})$.

Definition 24 (Anonymity against Full Key Exposure). For any polynomial time adversary $\mathcal{A}$, a ring signature is anonymous against full key exposure if any PPT adversary $\mathcal{A}$ has at most negligible advantage ($\text{Adv}_{\mathcal{A}}^{\text{EUF}}$) to win the following game:

- The challenger runs $\text{R-pp} \leftarrow \text{R-Setup}(1^\lambda)$ and generates key pairs $(sk_i, pk_i) \leftarrow \text{R-KeyGen}(\text{R-pp})$ for all $i \in [N]$. The challenger samples a challenge bit $b^* \xleftarrow{\$} \{0, 1\}$ and sends R-pp , $\mathbb{PK} = \{pk_i\}_{i \in [N]}$, and $\{sk_i\}_{i \in [N]}$ to $\mathcal{A}$.
- $\mathcal{A}$ computes a challenge $(\mathcal{PK}, \text{msg}, i_0, i_1)$, where $\mathcal{PK}$ contains pk_{i_0}, pk_{i_1} and sends it to the challenger. Note that we do not assume that $\mathcal{PK} \subseteq \mathbb{PK}$.
- The challenger runs $\sigma \leftarrow \text{R-Sign}(\mathcal{PK}, \text{msg}, sk_{i_b}, \text{R-pp})$ and sends σ to $\mathcal{A}$.
- $\mathcal{A}$ outputs b , and we say $\mathcal{A}$ wins the anonymity game if $b = b^*$.

3 Commit-and-Open Σ -Protocol for Legendre and Power Residue PRFs

3.1 Protocol Description

We first present the setup and key generation of base Σ -protocol in Algorithms 1 and 2, respectively. The C&O Σ -protocol is then shown in Algorithm 3.

3.2 Security Analysis of the Σ -Protocol

Let $\mathcal{R}_{\beta\mathcal{L}^t}$ denote the β -approximate PRF relation from Definition 8 and $L_{\mathcal{R}}$ be the language induced by $\mathcal{R}_{\beta\mathcal{L}^t}$ (i.e., $L_{\mathcal{R}} = \{pk : \exists sk \text{ such that } (pk, sk) \in \mathcal{R}_{\beta\mathcal{L}^t}\}$). The construction in Algorithm 3 satisfies completeness, proof-of-knowledge with online extraction (PoK-OE) and special honest verifier zero-knowledge (SHVZK) in the (quantum) random oracle model.

Algorithm 2: Key Generation

```

1   $(sk, pk) \leftarrow \text{KeyGen}(\text{pp})$ 
2  On input public parameters  $\text{pp}$ , the algorithm computes the secret
   key and the public key as follows:
3    - Randomly select  $K$  from  $\mathbb{F}_p^*$  and set  $sk = K$ ,
4    - Compute  $pk = \mathcal{L}_K^t(\mathcal{I}) = (\mathcal{L}_K^t(I_0), \dots, \mathcal{L}_K^t(I_{L-1}))$ .
5  Output  $(pk, sk)$ .
```

Theorem 3 (Completeness). *Assume the completeness (with negligible abort probability) of BAVC, if the witness-statement pair $(pk, sk) \in \mathcal{R}_{\beta\mathcal{L}^t}$ and both parties are honest, then the verifier always accepts the transcript.*

Proof Sketch. The key to the completeness proof is that by revealing all-but-one shares of r_e together with the key share adjustment, the verifier can reconstruct $o_e = (K + I_{\bar{\ell}_e})r_e$ for all $e \in [M]$. We defer the proof to Section B.3.

Now, we show the PoK-OE of the Σ -protocol, assuming the extractability of BAVC in the (quantum) random oracle model.

Theorem 4 (PoK-OE). *Let $\beta \in [0, 1]$ be fixed and let $\mathcal{H}$ denote a $(Q)RO$. If the BAVC protocol is extractably binding in the $(Q)ROM$, then the commit-and-open Σ -protocol $\Pi^{\mathcal{H}}$ is a PoK-OE in the $(Q)ROM$.*

Proof Sketch. For any PPT adversary $\mathcal{P}^*$, we construct a PPT extractor $\mathcal{E}$ that relies on the BAVC extractor. In the simulation of $\mathcal{H}$, if $\mathcal{P}^*$ produces a valid transcript through interaction with $\mathcal{E}$, then $\mathcal{E}$ extracts a valid secret key sk . In particular, $\mathcal{E}$ invokes the BAVC extractor to get all committed messages $\mathbf{R} = (r_{e,0}, \dots, r_{e,L-1})_{e \in [M]}$, which allows it to compute the candidate witness K^* from the key share adjustments $\Delta K_e = Kr_e + \sum_{\ell=0}^{L-1} I_{\ell} r_{e,\ell}$. For an accepting transcript, the value o_e must satisfy $\mathcal{L}_0^t(o_e) = T_e + pk_{\bar{\ell}_e}$, where $(T_e)_{e \in [M]}$ were fixed and sent in the first round. Thus, the check can be seen as verifying PRF outputs with a fixed public shift. According to the definition of the β -approximate PRF relation (Definition 8), for any incorrect $K^* \neq K$, there are at most $(1 - \beta) \cdot L$ values in $\mathcal{I}$ that can lead to matching Legendre/power-residue symbols. Hence, the probability that all M challenges fall in these “bad” values is at most $(1 - \beta)^M$. Together with the BAVC extraction error ϵ_{BAVC} in the $(Q)ROM$, we can bound the extraction error ϵ_{ext} of $\mathcal{E}$ by $\epsilon_{\text{BAVC}} + (1 - \beta)^M$. We defer the formal proof to Section B.4.

We now show the special honest verifier zero-knowledge (SHVZK) of the Σ -protocol.

Theorem 5 (SHVZK). *Let $\mathcal{H}$ denote a $(Q)RO$. If the BAVC protocol satisfies selective hiding in the $(Q)ROM$, then the commit-and-open Σ -protocol $\Pi^{\mathcal{H}}$ has SHVZK in the $(Q)ROM$.*

Proof Sketch. We construct the simulator which samples the key share adjustments $(\Delta K_e^*)_{e \in [M]}$ uniformly from $\mathbb{F}_p$. For every $e \in [M]$, given the known

Algorithm 3: Σ -Protocol based on Legendre and Power Residue PRFs

```

1 Procedure Commitment( $sk, pp$ ):
2   The prover  $\mathcal{P}$  generates the BAVC tree by running the tree
   generation algorithm:  $(\text{decom}, \mathbf{L}) \leftarrow \text{BAVC.TreeGen}(\text{BAVC.pp})$ .
3   Next,  $\mathcal{P}$  derives the commitment and the message by running the
   leaf commitment algorithm:  $(\text{com}, \mathbf{R}) \leftarrow \text{BAVC.LeafCom}(\mathbf{L}, \text{BAVC.pp})$ ,
   where  $\mathbf{R} = (r_{e,0}, \dots, r_{e,L-1})_{e \in [M]}$ .
4   for  $e = 1$  to  $M$  do
5     Compute  $r_e = \sum_{\ell=0}^{L-1} r_{e,\ell}$  and  $T_e = \mathcal{L}_0^t(r_e)$ .
6     Compute key adjustment  $\Delta K_e = Kr_e + \sum_{\ell=0}^{L-1} I_\ell r_{e,\ell}$ , where
        $K = sk$ .
7    $\mathcal{P}$  sends  $A = \text{com}$  and  $A_o = (T_e, \Delta K_e)_{e \in [M]}$  to  $\mathcal{V}$ .

8 Procedure Challenge( $A, A_o$ ):
9   On receive the commitment  $A$  and the auxiliary information  $A_o$ , the
   verifier  $\mathcal{V}$  randomly samples the challenge set  $S := \{\bar{\ell}_e\}_{e \in [M]}$  where
    $\bar{\ell}_e \xleftarrow{\$} [0, L-1]$ , and sends  $S$  to  $\mathcal{P}$ .

10 Procedure Response( $\text{decom}, S$ ):
11   On receive the challenge  $S$ ,  $\mathcal{P}$  computes
    $\text{decom}_S \leftarrow \text{BAVC.Open}(\text{decom}, S)$  and sends the response  $Z = \text{decom}_S$ 
   to  $\mathcal{V}$ .

12 Procedure Verify( $pk, ((A, A_o), S, Z), pp$ ):
13    $\mathcal{V}$  parses  $A = \text{com}$ ,  $A_o = (T_e, \Delta K_e)_{e \in [M]}$ ,  $S = \{\bar{\ell}_e\}_{e \in [M]}$ , and
    $Z = \text{decom}_S$ .
14    $\mathcal{V}$  re-computes commitment and messages by running
    $(\text{com}', \bar{\mathbf{R}})_{e \in [M]} \leftarrow \text{BAVC.Verify}(\text{decom}_S, S)$ , where
    $\bar{\mathbf{R}} = (r_{e,\ell})_{\ell \in [0, L-1] \setminus \{\bar{\ell}_e\}, e \in [M]}$ .
15    $\mathcal{V}$  outputs 0 (reject the signature) if  $\text{com}' \neq \text{com}$ . Otherwise,  $\mathcal{V}$  runs
   the following predicate function  $V(pk, S, \bar{\mathbf{R}}, A_o)$ , which is defined as:
16   for  $e = 1$  to  $M$  do
17     Compute  $o_e = I_{\bar{\ell}_e} \sum_{\ell=0, \ell \neq \bar{\ell}_e}^{L-1} r_{e,\ell} - \sum_{\ell=0, \ell \neq \bar{\ell}_e}^{L-1} r_{e,\ell} I_\ell + \Delta K_e$ .
18     Check if  $T_e \stackrel{?}{=} \mathcal{L}_0^t(o_e) - pk_{\bar{\ell}_e}$ .
19   If any previous check fails,  $\mathcal{V}$  outputs 0. Otherwise,  $\mathcal{V}$  accepts the
   signature and outputs 1.

```

challenge $\bar{\ell}_e$, it computes o_e^* exactly as the honest verifier would, and sets $T_e^* = \mathcal{L}_0^t(o_e^*) - pk_{\bar{\ell}_e}$. Since ΔK_e^* is uniform, the input o_e^* is also uniform in $\mathbb{F}_p$. Therefore, T_e^* is the PRF evaluation on a uniform field element. Indistinguishability between the real and simulated transcripts is shown via a simple hybrid argument. In the real protocol, ΔK_e and T_e are derived from the committed messages and the witness K . In the first hybrid, we replace unopened $r_{e,\bar{\ell}_e}$ by an independent uniform sample. Then, by hiding property of BAVC, this change is

indistinguishable to the verifier. Since $r_{e, \bar{\ell}_e}$ is uniform, the derived $\Delta K'_e$ is uniformly distributed (like ΔK_e^*) and T'_e is the PRF evaluation of a uniform value (like T_e^*), which proves that indistinguishability between the simulated and real transcripts. The full proof is shown in Section B.5

3.3 PEGASUS: Post-Quantum Signature based on Legendre and Power Residue PRFs

By applying the Fiat-Shamir transform [35] to the Σ -protocol described in Section 3.1, we obtain a post-quantum signature scheme for Legendre and power residue PRFs. The construction is shown in Algorithm 4. In addition to the public parameters generated in Algorithm 1, we define a hash function $\mathcal{H}_{\text{FS}} : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$ for the Fiat-Shamir transform, and an expand function $\text{expand} : \{0, 1\}^\lambda \rightarrow \{0, 1\}^*$ to derive verifier challenges.

3.4 Security Analysis of PEGASUS in (Q)ROM

EU-F-KO Security in (Q)ROM. We first show existential unforgeability of PEGASUS in the key-only model. The argument builds on the PoK-OE property of the underlying Σ -protocol, and follows the general structure of the Fiat-Shamir security proof. However, since our construction of PEGASUS introduces slight modifications to the FS transform, the proof must additionally account for these changes.

Theorem 6 (EU-F-KO of PEGASUS in the ROM). *Let $\beta \in [0, 1]$ be fixed, and let $\mathcal{H}$ and $\mathcal{H}_{\text{FS}}$ be random oracles. If the commit-and-open protocol $\Pi^{\mathcal{H}}$ is a PoK-OE in the ROM, and the β -approximate PRF relation $\mathcal{R}_{\beta\mathcal{L}^t}$ is hard, then the signature scheme $\text{PEGASUS}^{\mathcal{H}_{\text{FS}}, \mathcal{H}} := \text{FS}^{\mathcal{H}_{\text{FS}}}[\Pi^{\mathcal{H}}]$ achieves EU-F-KO security in the ROM.*

Proof Sketch. At a high level, if $\mathcal{A}$ outputs a valid message-signature pair (m, σ) , then, with overwhelming probability, it must have queried $\mathcal{H}_{\text{FS}}$ on some input correlated with the random oracle output used in the forgery. Consequently, the PPT extractor $\mathcal{E}$ (as defined in the PoK-OE notion) can leverage the forged signature together with $\mathcal{A}$'s correlated queries to extract the secret key sk from the public key pk . This extraction would contradict the hardness of the underlying NP relation $\mathcal{R}_{\beta\mathcal{L}^t}$ associated with (pk, sk) . We defer the full proof of Theorem 6 to Section B.6.

We further analyze the EU-F-KO security of PEGASUS in the QROM. Unlike the classical ROM, in the QROM an adversary can issue quantum queries to the QRO and receive superposition responses. In this setting, the classical lazy-sampling technique is inapplicable, as we cannot maintain a classical query list to extract information. Fortunately, several proof techniques have been developed for the QROM. In particular, the CStO technique [68] enables both the simulation of the QRO and the extraction of classical information, which we employ in our analysis.

Algorithm 4: PEGASUS Signing and Verification

```

1 Procedure Sign(msg, sk, pp):
2   Sample a random salt: salt  $\xleftarrow{\$} \{0, 1\}^{2\lambda}$ .
3   Compute the BAVC commitment:
      (decom, L)  $\leftarrow$  BAVC.TreeGen(BAVC.pp) and
      (com, R)  $\leftarrow$  BAVC.LeafCom(BAVC.pp, L), where
      R =  $(r_{e,0}, \dots, r_{e,L-1})_{e \in [M]}$ .
4   for  $e = 1$  to  $M$  do
5     Compute  $r_e = \sum_{\ell=0}^{L-1} r_{e,\ell}$  and  $T_e = \mathcal{L}_0^t(r_e)$ .
6     Compute key share adjustment  $\Delta K_e = Kr_e + \sum_{\ell=0}^{L-1} I_\ell r_{e,\ell}$ .
7   Define  $\sigma_1 = (\text{com}, T_1, \dots, T_M, \Delta K_1, \dots, \Delta K_M)$ .
8   Initiate the counter: ctr = 0.
9   Compute the challenge  $h = \mathcal{H}_{\text{FS}}(\text{salt}, \sigma_1, \text{msg}, \text{ctr})$ .
10  Expand  $h$ :  $S \leftarrow \text{expand}(h[128])$  where  $S := \{\bar{\ell}_e\}_{e \in [M]}$  and for every
       $e \in [M]$ ,  $\bar{\ell}_e \in [0, L-1]$ 
11  if  $\perp \leftarrow \text{BAVC.Open}(\text{decom}, S)$  then
12    Return to Step 9 with ctr = ctr + 1.
13  else
14     $\text{decom}_S \leftarrow \text{BAVC.Open}(\text{decom}, S)$ 
15  Output the signature  $\sigma = (\text{salt}, h, \text{ctr}, \text{decom}_S, (\Delta K_e)_{e \in [M]})$ .

16 Procedure Verify(msg, pk,  $\sigma$ , pp):
17  Parse  $\sigma = (\text{salt}, h, \text{ctr}, \text{decom}_S, (\Delta K_e)_{e \in [M]})$ .
18  Expand the challenge  $h$ :  $S \leftarrow \text{expand}(h[128])$  where  $S := \{\bar{\ell}_e\}_{e \in [M]}$ 
      and for every  $e \in [M]$ ,  $\bar{\ell}_e \in [0, L-1]$ 
19  Recompute  $(\text{com}', \bar{\mathbf{R}}) \leftarrow \text{BAVC.Verify}(\text{decom}_S, S)$ , where
       $\bar{\mathbf{R}} = (r_{e,\ell})_{\ell \in [0, L-1], e \in [M]}$ .
20  for  $e = 1$  to  $M$  do
21    Compute  $o_e = I_{\bar{\ell}_e} \sum_{\ell=0, \ell \neq \bar{\ell}_e}^{L-1} r_{e,\ell} - \sum_{\ell=0, \ell \neq \bar{\ell}_e}^{L-1} r_{e,\ell} I_\ell + \Delta K_e$ .
22    Recompute residuosity symbols:  $T'_e = \mathcal{L}_0^t(o_e) - pk_{\bar{\ell}_e}$ .
23  Check if  $h \stackrel{?}{=} \mathcal{H}(\text{salt}, \sigma'_1, \text{msg}, \text{ctr})$  where
       $\sigma'_1 = (\text{com}', T'_1, \dots, T'_M, \Delta K_1, \dots, \Delta K_M)$ .

```

Theorem 7 (EUF-KO of PEGASUS in the QROM). *Let $\beta \in [0, 1]$ be fixed, and let $\mathcal{H}$ and $\mathcal{G}$ be quantum random oracles. If the commit-and-open protocol $\Pi^{\mathcal{H}}$ is a PoK-OE in the QROM, and the β -approximate PRF relation $\mathcal{R}_{\beta\mathcal{L}^t}$ is hard, then the signature scheme $\text{PEGASUS}^{\mathcal{H}_{\text{FS}}, \mathcal{H}} := \text{FS}^{\mathcal{H}_{\text{FS}}}[\Pi^{\mathcal{H}}]$ achieves EUF-KO security in the QROM.*

Proof Sketch. The proof strategy is analogous to that in the classical ROM, where we construct a quantum PPT algorithm $\mathcal{B}$ that simulates $\mathcal{H}_{\text{FS}}$ via CStO

and extracts transcripts of the underlying C&O Σ -protocol. We define two database properties: SUC, which captures the case where the adversary queries the QRO and outputs a valid signature while the extracted Σ -protocol transcript is invalid, and CL, which corresponds to database collisions. Our analysis bounds the probability of quantum transition capacity $[\perp \xrightarrow{q} \text{SUC} \cup \text{CL}]$, where q is the number of quantum queries the adversary makes to the QRO $\mathcal{H}_{\text{FS}}$. We present the full security proof of EUF-KO in the QROM in Section B.7

EUF-CMA Security in the (Q)ROM. We now reduce the existential unforgeability of PEGASUS under chosen-message attacks (EUF-CMA) to EUF-KO.

Theorem 8 (EUF-CMA of PEGASUS in the ROM). *Let $\mathcal{H}$ and $\mathcal{H}_{\text{FS}}$ be random oracles. If the commit-and-open protocol $\Pi^{\mathcal{H}}$ is a PoK-OE and has SHVZK in the ROM, and the β -approximate PRF relation $\mathcal{R}_{\beta\mathcal{L}^\dagger}$ is hard, then the signature scheme $\text{PEGASUS}^{\mathcal{H}_{\text{FS}}, \mathcal{H}} := \text{FS}^{\mathcal{H}_{\text{FS}}}[\Pi^{\mathcal{H}}]$ achieves EUF-CMA security in the ROM.*

Proof Sketch. In the EUF-CMA game, the adversary is granted access to a signing oracle, which is simulated by the reduction algorithm $\mathcal{B}$. The simulation leverages the SHVZK property of the underlying protocol. Moreover, $\mathcal{B}$ needs to program the random oracle so that the adversary’s query is mapped to the pre-defined challenge. We defer the full proof to Section B.8

Theorem 9 (EUF-CMA of PEGASUS Signature in the QROM). *Let $\mathcal{H}$ and $\mathcal{H}_{\text{FS}}$ be quantum random oracles. If the commit-and-open protocol $\Pi^{\mathcal{H}}$ is a PoK-OE and has SHVZK in the QROM, and the β -approximate PRF relation $\mathcal{R}_{\beta\mathcal{L}^\dagger}$ is hard, then the signature scheme $\text{PEGASUS}^{\mathcal{H}_{\text{FS}}, \mathcal{H}} := \text{FS}^{\mathcal{H}_{\text{FS}}}[\Pi^{\mathcal{H}}]$ achieves EUF-CMA security in the QROM.*

Proof Sketch. The proof idea is similar to that of Theorem 8. In particular, $\mathcal{B}$ simulates $\mathcal{H}_{\text{FS}}$ using the CStO technique [68] in the RO Simulation Phase. Additionally, in the Signing Simulation Phase, $\mathcal{B}$ reprograms the QRO rather than aborts. The probability that $\mathcal{A}$ detects such reprogramming is bounded using Lemma 6. The full security proof is shown in Section B.9

4 PEGARING: Post-Quantum Ring Signature

4.1 Protocol Description

We begin by introducing a generalized list structure using arithmetic progressions, which is applied to improve the efficiency of PEGARING. Let $\mathcal{I}'$ be a list which includes z sub-lists $\mathcal{I}'_0, \dots, \mathcal{I}'_{z-1}$, where each sub-list $\mathcal{I}'_j = (a_j, a_j + b, \dots, a_j + b(c-1))$ for $j \in [0, z-1]$. Each a_j is sampled uniformly from $\mathbb{F}_p$, and the fixed gap $b \in \mathbb{F}_p^*$ is shared across all sub-lists. By setting $L = c \times z$, we can “stretch” the sub-lists and get a list of L field elements. In the set membership

proof, we interpret each index $\ell \in [0, L-1]$ as $\ell = z'c + c'$ where $z' = \lfloor \frac{\ell}{c} \rfloor$ and $c' = \ell \pmod{c}$. Hence, $z' \in [0, z-1]$ denotes the index of the sub-list in $\mathcal{I}'$, while $c' \in [0, c-1]$ denotes the position of the element in the sub-list $\mathcal{I}'_{z'}$. Note that $I_\ell = I_{z'c+c'} = a_{z'} + bc'$. This arithmetic progression list generalizes the uniformly random list. That is, by setting $c = 1$, we obtain a list that contains L uniformly random elements.

We define the setup for PEGARING, which outputs R-pp that includes parameters pp generated by Algorithm 1, replacing $\mathcal{I}$ with $\mathcal{I}'$ and appending the following parameters:

- Integer z and c such that $z \times c = L$,
- $\mathcal{H}_{\text{FS}} : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$ for Fiat-Shamir transform,
- $\mathcal{H}_{\text{MT}} : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$ for computing the leaf nodes of Merkle tree,
- $\mathcal{H}_{\text{PK}} : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$ for hashing the public keys,
- **expand** : $\{0, 1\}^\lambda \rightarrow \{0, 1\}^*$ for generating the challenge indices.

The key generation is defined the same as that in Algorithm 2. Let us first recall the relation of the set membership proof based on Legendre and power residue PRFs:

$$\mathcal{R}_{\beta\mathcal{L}^t}^{\text{OR}} = \{(\mathcal{PK}, sk) : \exists pk_\pi \in \mathcal{PK}, (pk_\pi, sk) \in \mathcal{R}_{\beta\mathcal{L}^t}\},$$

where $\mathcal{PK} = (pk_1, \dots, pk_n)$. We present the C&O Σ -protocol for $\mathcal{R}_{\beta\mathcal{L}^t}^{\text{OR}}$ in Algorithms 5, 6, and 7.

4.2 Security Analysis of PEGARING in (Q)ROM

We begin by showing the correctness of PEGARING. In the verification, to make sure

$$T'_e = \mathcal{L}_0^t(o_e) - \sum_{i=1}^n pk_{\bar{\ell}_e^{(i)}}^{(i)} = \mathcal{L}_0^t(r_e) - \sum_{i \neq \pi} pk_{\bar{\ell}_e^{(i)}}^{(i)},$$

the value o_e must be reconstructed as

$$o_e = (K + I_{\bar{\ell}_e^{(\pi)}})r_e.$$

However, opening BAVC at position $\bar{\ell}_e$ only allows the verifier to recover $(K + I_{\bar{\ell}_e})r_e$ (see the correctness of PEGASUS). To bridge this gap, we adjust the key share by setting

$$\Delta K_e = (K - I_{\bar{\ell}_e} + I_{\bar{\ell}_e^{(\pi)}})r_e + \sum_{\ell=0}^{L-1} I_\ell r_{e,\ell},$$

which cancels the term $I_{\bar{\ell}_e} r_e$ obtained from the opening and injects the required $I_{\bar{\ell}_e^{(\pi)}} r_e$, so the verifier recovers o_e with respect to the real signer's challenge.

Another subtlety for correctness is the modular reduction hidden in the key share adjustment. Recall that we have

$$I_{\bar{\ell}_e} = a_{\bar{z}_e} + b\bar{c}_e \quad \text{and} \quad I_{\bar{\ell}_e^{(\pi)}} = a_{\bar{z}_e^{(\pi)}} + b\bar{c}_e^{(\pi)}.$$

Algorithm 5: PegaRing Signing (Part I)

```

1 Procedure R-Sign(msg, sk, PK, R-pp):
2   Sample a random salt: salt  $\xleftarrow{\$} \{0, 1\}^{2\lambda}$ .
3   for  $i \in [n], i \neq \pi$  do
4     Generate random challenges for non-signers:  $h_i \leftarrow \{0, 1\}^\lambda$ .
5     Expand challenges to get  $\{\bar{\ell}_e^{(i)}\}_{e \in [M]} \leftarrow \text{expand}(h_i)$ , where each
       $\bar{\ell}_e^{(i)} \in [0, L-1]$ .
6     Let  $(\bar{z}_e^{(i)}, \bar{c}_e^{(i)}) := \left( \left\lfloor \frac{\bar{\ell}_e^{(i)}}{c} \right\rfloor, \bar{\ell}_e^{(i)} \pmod{c} \right)$  for each  $e \in [M]$ .
7   Compute (decom, L)  $\leftarrow$  BAVC.TreeGen(BAVC.pp) and
      (com, R)  $\leftarrow$  BAVC.LeafCom(L), where  $\mathbf{R} = (r_{e,0}, \dots, r_{e,L-1})_{e \in [M]}$ .
8   for  $e = 1$  to  $M$  do
9     Compute  $r_e = \sum_{\ell=0}^{L-1} r_{e,\ell}$ .
10    Compute  $\bar{z}'_e = \sum_{i \neq \pi} \bar{z}_e^{(i)} \pmod{z}$  and  $\bar{c}'_e = \sum_{i \neq \pi} \bar{c}_e^{(i)} \pmod{c}$ .
11    for  $j = 0$  to  $z-1$  do
12      Let  $\bar{j} = j - \bar{z}'_e \pmod{z}$ .
13      Compute key share adjustments
           $\Delta K_{e,j}^0 = (K - (a_j + b\bar{c}'_e) + a_{\bar{j}})r_e + \sum_{\ell=0}^{L-1} I_\ell r_{e,\ell}$  and
           $\Delta K_{e,j}^1 = (K - (a_j + b\bar{c}'_e) + bc + a_{\bar{j}})r_e + \sum_{\ell=0}^{L-1} I_\ell r_{e,\ell}$ .
14      Let  $b_e \xleftarrow{\$} \{0, 1\}$ , compute the leaf node as
           $v_{e,j}^0 = \mathcal{H}_{\text{MT}}(\text{salt}, e, j, 0, \Delta K_{e,j}^{b_e})$ ,
           $v_{e,j}^1 = \mathcal{H}_{\text{MT}}(\text{salt}, e, j, 1, \Delta K_{e,j}^{1-b_e})$ , and
           $v_{e,j} = \mathcal{H}_{\text{MT}}(\text{salt}, e, j, v_{e,j}^0, v_{e,j}^1)$ .
15      Compute  $T_e = \mathcal{L}_0^t(r_e) - \sum_{i=1, i \neq \pi}^n pk_{\bar{\ell}_e^{(i)}}^{(i)}$ .
16      Compute comMT = MT.Commit(V), where
           $\mathbf{V} = ((v_{1,0}, \dots, v_{M,0}), \dots, (v_{1,z-1}, \dots, v_{M,z-1}))$ .
17      Compute  $h_{\text{PK}} = \mathcal{H}_{\text{PK}}(\text{PK})$  and define  $\sigma_1 = (\text{com}, \text{com}_{\text{MT}}, (T_e)_{e \in [M]})$ .

```

Algorithm 6: PegaRing Signing (Part II)

```

18 Procedure R-Sign:
19   Initiate the counter ctr = 0.
20    $h = \mathcal{H}_{\text{FS}}(\text{salt}, \sigma_1, \text{msg}, \text{ctr}, h_{\mathcal{PK}})$  and  $h_\pi = h[:128] \oplus \bigoplus_{i=1, i \neq \pi}^n h_i$ .
21   Expand  $h_\pi$ :  $(\bar{\ell}_e^{(\pi)})_{e \in [M]} \leftarrow \text{expand}(h_\pi)$  where for every  $e \in [M]$ ,
       $\bar{\ell}_e^{(\pi)} \in [0, L-1]$ .
22   for  $e = 1$  to  $M$  do
23     Compute  $(\bar{z}_e^{(\pi)}, \bar{c}_e^{(\pi)})$  from  $\bar{\ell}_e^{(\pi)}$ ,  $\bar{z}_e = \sum_{i=1}^n \bar{z}_e^{(i)} \pmod{z}$ ,
       $\bar{c}_e = \sum_{i=1}^n \bar{c}_e^{(i)} \pmod{c}$ , and  $\bar{\ell}_e = \bar{z}_e c + \bar{c}_e$ .
24     If  $\bar{c}_e \geq \bar{c}'_e$ , set  $\Delta K_e = \Delta K_{e, \bar{z}_e}^0$  and  $v_e$  to be the hash of  $\Delta K_{e, \bar{z}_e}^1$ .
      Otherwise, set  $\Delta K_e = \Delta K_{e, \bar{z}_e}^1$  and  $v_e$  to be the hash of  $\Delta K_{e, \bar{z}_e}^0$ .
25     Let  $d_e \in \{0, 1\}$  indicates the hash sequence of  $\Delta K_e$  and  $v_e$ .
26   Define  $S = \{\bar{\ell}_e\}_{e \in [M]}$  and  $I = \{\bar{z}_e\}_{e \in [M]}$ .
27   if  $(\perp \leftarrow \text{BAVC.Open}(\text{decom}, S))$  OR  $(\perp \leftarrow \text{MT.Open}(I, \mathbf{V}))$  then
28     Return to Step 19 with ctr = ctr + 1.
29   else
30     Compute  $\text{decom}_S \leftarrow \text{BAVC.Open}(\text{decom}, S)$  and
       $(\mathbf{V}_I, \text{path}_I) \leftarrow \text{MT.Open}(I, \mathbf{V})$ .
31   Let  $\sigma = (\text{salt}, \text{ctr}, \text{decom}_S, \text{path}_I, (\Delta K_e, v_e, d_e)_{e \in [M]}, (h_i)_{i \in [n]})$ .

```

In the formation of key share adjustment, we let the real signer iterate the pair of values $(a_{\bar{z}_e}, a_{\bar{z}_e^{(\pi)}})$ with

$$\bar{z}_e - \bar{z}_e^{(\pi)} = \sum_{i \neq \pi} \bar{z}_e^{(i)}$$

for every $\bar{z}_e = 0, 1, \dots, z-1$ (note that $\sum_{i \neq \pi} \bar{z}_e^{(i)}$ is a fixed value since all non-signers' challenges are generated in the first round). For the pair of $\bar{c}_e$ and $\bar{c}_e^{(\pi)}$, we use the fixed gap $\bar{c}'_e = \sum_{i \neq \pi} \bar{c}_e^{(i)}$ rather than enumerating since these values are in the same additively cyclic group $\mathbb{Z}_c$. However, the gap is defined over modulo c . Concretely, we have

$$\bar{c}_e^{(\pi)} = \bar{c}_e - \bar{c}'_e \pmod{c} = \begin{cases} \bar{c}_e - \bar{c}'_e & \text{if } \bar{c}_e \geq \bar{c}'_e, \\ \bar{c}_e - \bar{c}'_e + c & \text{otherwise.} \end{cases}$$

To hide which case holds, we prepare two key share adjustments, $\Delta K_{e,j}^0$ (without wraparound) and $\Delta K_{e,j}^1$ (with wraparound, including the extra $bc \cdot r_e$), and commit to them in random order. With this issue handled, the remaining correctness follows directly from PEGASUS and DualRing.

Below, we present PEGARING's unforgeability with respect to insider corruption and anonymity against full key exposure in the (Q)ROM.

Algorithm 7: PegaRing Verification

```

1 Procedure R-Verify(msg,  $\mathcal{PK}$ ,  $\sigma$ , R-pp):
2   Parse  $\sigma = (\text{salt}, \text{ctr}, \text{decom}_S, \text{path}_I, (\Delta K_e, v_e, d_e)_{e \in [M]}, (h_i)_{i \in [n]})$ .
3   for  $i = 1$  to  $n$  do
4     Expand  $h_i$ :  $(\bar{\ell}_e^{(i)})_{e \in [M]} \leftarrow \text{expand}(h_i)$  where for every  $e \in [M]$ ,
        $\bar{\ell}_e^{(i)} \in [0, L-1]$ .
5     Compute  $\bar{z}_e^{(i)} = \lfloor \frac{\bar{\ell}_e^{(i)}}{c} \rfloor$  and  $\bar{c}_e^{(i)} = \bar{\ell}_e^{(i)} \pmod{c}$  for every  $e \in [M]$ .
6   for  $e = 1$  to  $M$  do
7     Compute  $\bar{z}_e = \sum_{i=1}^n \bar{z}_e^{(i)} \pmod{z}$ ,  $\bar{c}_e = \sum_{i=1}^n \bar{c}_e^{(i)} \pmod{c}$ , and
        $\bar{\ell}_e = \bar{z}_e c + \bar{c}_e$ .
8     Define  $S := \{\bar{\ell}_e\}_{e \in [M]}$  and  $I = \{\bar{z}_e\}_{e \in [M]}$ .
9     Recompute Merkle tree leaf based on  $d_e$ : If  $d_e = 0$ , compute
        $v_{e, \bar{z}_e} = \mathcal{H}_{\text{MT}}(\text{salt}, e, j, \mathcal{H}_{\text{MT}}(\text{salt}, e, j, 0, \Delta K_e), v_e)$ . Otherwise
       ( $d_e = 1$ ), compute
        $v_{e, \bar{z}_e} = \mathcal{H}_{\text{MT}}(\text{salt}, e, j, v_e, \mathcal{H}_{\text{MT}}(\text{salt}, e, j, 1, \Delta K_e))$ .
10    Define  $\mathbf{V}_I = (v_{e, \bar{z}_e})_{e \in [M]}$ .
11    Recompute  $(\text{com}', \mathbf{R})_{e \in [M]} \leftarrow \text{BAVC.verify}(\text{decom}_S, S)$ , where
        $\mathbf{R} = ((r_{e, \ell})_{\ell \in [0, L-1] \setminus \{\bar{\ell}_e\}})$  and  $\text{com}'_{\text{MT}} \leftarrow \text{MT.Verify}(\mathbf{V}_I, \text{path}_I, I)$ .
12    for  $e \in [M]$  do
13      Compute  $o_e = (a\bar{z}_e + b\bar{c}_e) \sum_{\ell=0, \ell \neq \bar{\ell}_e}^{L-1} r_{e, \ell} - \sum_{\ell=0, \ell \neq \bar{\ell}_e}^{L-1} r_{e, \ell} I_\ell + \Delta K_e$ .
14       $T'_e = \mathcal{L}_0^t(o_e) - \sum_{i=1}^n pk_{\bar{\ell}_e^{(i)}}^{(i)}$ .
15    Check if  $\bigoplus_{i=1}^n h_i \stackrel{?}{=} \mathcal{H}_{\text{FS}}(\text{salt}, \sigma'_1, \text{msg}, \text{ctr}, \mathcal{H}_{\mathcal{PK}}(\mathcal{PK}))[:128]$  where
        $\sigma'_1 = (\text{com}', \text{com}'_{\text{MT}}, (T'_e)_{e \in [M]})$ .

```

Theorem 10. Let $\beta \in [0, 1]$ be fixed, $\mathcal{H}_{\text{FS}}$ and $\mathcal{H}_{\text{MT}}$ be simulated as random oracles $\mathcal{G}_{\text{FS}}$ and $\mathcal{G}_{\text{MT}}$, respectively. If the commit-and-open protocol $\Pi^{\mathcal{H}}$ is a PoK-OE in the ROM and β -approximate PRF relation $\mathcal{R}_{\beta \mathcal{L}^t}$ is hard, then the ring signature scheme PEGARING is unforgeable with respect to the insider corruption in the ROM.

Proof Sketch. Similar to the reduction from PEGASUS to PoK-OE, if an adversary $\mathcal{A}$ produces a valid forge (m^*, σ^*) targeting the challenge public key $pk^* \in \mathcal{PK}$, then it must have queried the random oracles $\mathcal{H}_{\text{FS}}$ and $\mathcal{H}_{\text{MT}}$. Hence, we can extract a transcript for the underlying Σ -protocol (Section 3.1). Then, by the PPT extractor from PoK-OE, we can recover the secret key sk^* , thereby breaking the underlying NP relation $\mathcal{R}_{\beta \mathcal{L}^t}$ associated with (pk^*, sk^*) . The full proof is shown in Section B.10.

Theorem 11. Let $\beta \in [0, 1]$ be fixed, $\mathcal{H}_{\text{FS}}$ and $\mathcal{H}_{\text{MT}}$ be simulated as quantum random oracles $\mathcal{G}_{\text{FS}}$ and $\mathcal{G}_{\text{MT}}$, respectively. If the commit-and-open protocol $\Pi^{\mathcal{H}}$ is a PoK-OE in the QROM and β -approximate PRF relation $\mathcal{R}_{\beta \mathcal{L}^t}$ is hard, then

the ring signature scheme *PEGARING* is unforgeable with respect to the insider corruption in the *QROM*.

Proof Sketch. The overall proof strategy is similar to that of Theorem 10, except that the random oracles are simulated using the CStO technique [68]. In the proof, we assume classical access to the signing and corruption oracles, following the assumption in [41]. This is justified since the adversary is modeled as quantum-capable, whereas honest users implement the schemes on classical machines. We again define the database properties *SUC* and *CL* similarly to that in the proof of Theorem 7 and bound the quantum transition capacity. We defer the full proof to Section B.11.

Theorem 12. *Our ring signature scheme (PegaRing) is anonymous against full key exposure in the random oracle model if BAVC is hiding. Concretely, an adversary makes q_h random oracle queries and q_s signing oracle queries can win the anonymity game with advantage $\text{Adv}_{\mathcal{A}}^{\text{anon}}(\lambda) \leq \frac{q_s(q_s+q_h)}{2^{2\lambda}}$.*

Proof Sketch. We prove the anonymity by simulating the signature without using any secret key. The simulation is basically similar to that of PEGASUS, which is done by programming the Fiat-Shamir random oracle to a pre-defined challenge. The indistinguishability between the simulated ring signature and the real ring signature is then argued based on the hiding property of BAVC. We move the full proof to Section B.12.

Theorem 13. *Our ring signature scheme (PegaRing) is anonymous against full key exposure in the quantum random oracle model if BAVC is hiding. Concretely, an adversary makes q_h quantum random oracle queries and q_s signing oracle queries can win the anonymity game with advantage $\text{Adv}_{\mathcal{A}}^{\text{anon}}(\lambda) \leq \frac{q_s(q_s+q_h)}{2^\lambda}$.*

Proof. The proof is similar to that of Theorem 12, except that $\mathcal{B}$ needs to program the QRO. By the OW2H lemma, the probability that any quantum PPT adversary distinguishes this programming is bounded above by $\frac{q_s(q_s+q_h)}{2^\lambda}$. $\square$

5 Performances

5.1 Signature Size Analysis

Each PEGASUS signature consists of the following components:

- A 2λ bits salt `salt`;
- A 2λ bits h that is the output of the hash function $\mathcal{H}$;
- Tree-PRG based vector commitment openings `decomS` with T_{open} intermediate nodes, each of length λ -bits, and M commitments to unrevealed leaf, each of length 2λ bits.
- M share adjustments $(\Delta K_e)_{e \in [M]}$, where $\Delta K_e \in \mathbb{F}_p$.

Overall, we have total signature sizes⁹:

$$\begin{aligned} |\sigma| &= 4\lambda + T_{\text{open}} \cdot \lambda + M \cdot 2\lambda + M \cdot \lambda \\ &= (4 + T_{\text{open}} + 3M)\lambda \end{aligned}$$

Each PEGARING signature consists of the components included in the PEGASUS (except for FS output h as it can be computed from ring members' challenge seeds) plus the opening of the Merkle tree and the hash of sibling key share adjustments $(v_e)_{e \in [M]}$. Hence, the total signature size for n ring members is

$$\begin{aligned} |\sigma| &= (2 + T_{\text{open}} + 3M + n)\lambda + (T_{\text{open}}^{\text{MT}} + M) \cdot 2\lambda + n \cdot \lambda \\ &= (2 + T_{\text{open}} + 5M + 2T_{\text{open}}^{\text{MT}} + n) \cdot \lambda, \end{aligned}$$

where $T_{\text{open}}^{\text{MT}}$ includes the nodes for opening M leaves in a Merkle tree.

5.2 Choice of Parameters and Implementations

To achieve 128-bit classical security (about 100-bits of quantum security according to the NIST standard¹⁰), we choose the prime $p = 2^{127} - 1$ as in LegRoast [15] and Loquat [69]. In our experiments, we set the power residue PRF parameters to $t = 254$. The trade-off between signature size and signing/verification time is controlled by the public key length L . Specifically, increasing L results in larger public keys and higher signing and verification costs, but yields smaller signatures. We evaluate performance across $L = 2^9$ to 2^{12} , corresponding to public key sizes ranging from 0.5 KB to 4 KB. The BAVC parameter is set as a constant of $T_{\text{open}} = 0.9 \times (M \times \log_2(L))$ and the Merkle tree parameter is set to $T_{\text{open}}^{\text{MT}} = 0.9 \times (M \times \log_2(z))$.

Note that when $t = 2$ and $L = 2^{15}$, the construction relies on the security of the Legendre PRF. However, our experimental results show that the signature scheme performs better with higher-order power residue PRFs. For instance, when PEGASUS is instantiated with the Legendre PRF, the resulting signature size is 39 KB, with signing and verification time of 845.73 ms and 866.35 ms, respectively. The large overhead is because the BAVC tree is substantially larger than in the case $t = 254$.

We also benchmark PEGARING under various values of ring sizes n . For the arithmetic progression list construction, we ensure that the bound derived from Rényi divergence $\frac{t^c}{\sqrt{p}} \leq \frac{1}{Lt}$ is satisfied. Given $p \approx 2^{128}$, we use $c = 4$ and $z = L/c$. Our implementation is written in C (with the AVX optimization for hash functions), and all experiments are conducted on a Linux system equipped with an AMD Ryzen 5 7640HS CPU at 4.3 GHz and 32 GB of RAM. During benchmarking, Hyperthreading and Turbo Boost are disabled.

⁹ The size of the counter $|\text{ctr}|$ is negligible, as it is typically a small integer significantly smaller than λ bits.

¹⁰ Assume the maximum depth of quantum circuit is 2^{64} .

6 Conclusion

In this paper, we introduced a commit-and-open Σ -protocol for the key identification problem (i.e., proving knowledge of a secret key) based on the Legendre and power residue PRFs. Applying the standard Fiat-Shamir transform, we obtained an efficient post-quantum signature scheme, PEGASUS, which achieves smaller signature sizes than prior schemes based on these PRFs [15, 69], while also enjoying stronger security guarantees in the quantum random oracle model.

We further constructed an efficient set-membership proof that preserves the three-move commit-and-open structure for relations induced by the Legendre and power residue PRFs, and compiled it into a ring signature scheme, PEGARING, via Fiat-Shamir. Compared to the prior ring signature scheme based on the same PRF relation [70], PEGARING reduces the signature size by more than half and achieves significantly faster performance. Finally, we proved the security of our ring signature in both the classical random oracle model and the quantum random oracle model, which is, to the best of our knowledge, the first symmetric-key-based ring signature with provable QROM security.

Future Works. The current PEGARING scheme has a signature size that grows linearly with the number of ring members. Constructing a variant with logarithmic-size signatures is non-trivial. The main challenge lies in the lack of an efficient, tailored sum argument (based on symmetric-key primitives) that can simultaneously prove the relation between users' challenges and the mutual challenge, while also demonstrating that the public key components used in verification correspond to the claimed challenge indices.

Acknowledgments.

We sincerely thank Muhammed Esgin, Dongxi Liu and Sushmita Ruj for their help with the early draft of this paper.

References

1. Ambainis, A., Hamburg, M., Unruh, D.: Quantum security proofs using semi-classical oracles. In: *Advances in Cryptology - CRYPTO 2019 - 39th Annual International Cryptology Conference*, Santa Barbara, CA, USA, August 18-22, 2019, Proceedings, Part II. pp. 269–295 (2019)
2. Attema, T., Fehr, S., Klooß, M.: Fiat-shamir transformation of multi-round interactive proofs. In: *Theory of Cryptography Conference*. pp. 113–142. Springer (2022)
3. Attema, T., Fehr, S., Klooß, M.: Fiat-shamir transformation of multi-round interactive proofs (extended version). *Journal of Cryptology* **36**(4), 36 (2023)
4. Baum, C., Beullens, W., Braun, L., Delpech de Saint Guilhem, C., Klooß, M., Majenz, C., Mukherjee, S., Orsini, E., Ramacher, S., Rechberger, C., et al.: Shorter, tighter, faester: Optimizations and improved (qrom) analysis for vole-in-the-head signatures. In: *Annual International Cryptology Conference*. pp. 124–156. Springer (2025)
5. Baum, C., Beullens, W., Mukherjee, S., Orsini, E., Ramacher, S., Rechberger, C., Roy, L., Scholl, P.: One tree to rule them all: Optimizing GGM trees and owfs for post-quantum signatures. In: *ASIACRYPT (1)*. *Lecture Notes in Computer Science*, vol. 15484, pp. 463–493. Springer (2024)
6. Baum, C., Beullens, W., Mukherjee, S., Orsini, E., Ramacher, S., Rechberger, C., Roy, L., Scholl, P.: One tree to rule them all: Optimizing ggm trees and owfs for post-quantum signatures. In: *International Conference on the Theory and Application of Cryptology and Information Security*. pp. 463–493. Springer (2025)
7. Baum, C., Braun, L., de Saint Guilhem, C.D., Klooß, M., Orsini, E., Roy, L., Scholl, P.: Publicly verifiable zero-knowledge and post-quantum signatures from vole-in-the-head. In: *Annual International Cryptology Conference*. pp. 581–615. Springer (2023)
8. Baum, C., de Saint Guilhem, C.D., Kales, D., Orsini, E., Scholl, P., Zaverucha, G.: Banquet: Short and fast signatures from aes. In: *PKC*. pp. 266–297 (2021)
9. Bellare, M., Rogaway, P.: Random oracles are practical: A paradigm for designing efficient protocols. In: *CCS*. pp. 62–73. ACM (1993)
10. Ben-Sasson, E., Chiesa, A., Riabzev, M., Spooner, N., Virza, M., Ward, N.P.: Aurora: Transparent succinct arguments for r1cs. In: *Eurocrypt*. pp. 103–128. Springer (2019)
11. Bernstein, D.J., Hopwood, D., Hülsing, A., Lange, T., Niederhagen, R., Papachristodoulou, L., Schneider, M., Schwabe, P., Wilcox-O’Hearn, Z.: Sphincs: practical stateless hash-based signatures. In: *Eurocrypt*. pp. 368–397. Springer (2015)
12. Bernstein, D.J., Hülsing, A., Kölbl, S., Niederhagen, R., Rijneveld, J., Schwabe, P.: The sphincs+ signature framework. In: *Proceedings of the 2019 ACM CCS*. pp. 2129–2146 (2019)
13. Beullens, W., Beyne, T., Udovenko, A., Vitto, G.: Cryptanalysis of the legendre prf and generalizations. *IACR Transactions on Symmetric Cryptology* pp. 313–330 (2020)
14. Beullens, W., Katsumata, S., Pintore, F.: Calamari and falaffl: logarithmic (linkable) ring signatures from isogenies and lattices. In: *ASIACRYPT*. pp. 464–492. Springer (2020)
15. Beullens, W., de Saint Guilhem, C.D.: Legroast: Efficient post-quantum signatures from the legendre prf. In: *PQCrypto*. pp. 130–150. Springer (2020)

16. Buchmann, J., Dahmen, E., Hülsing, A.: Xmss-a practical forward secure signature scheme based on minimal security assumptions. In: PQCrypto. pp. 117–129. Springer (2011)
17. Bui, D., Cong, K., de Saint Guilhem, C.D.: Faster voleith signatures from all-but-one vector commitment and half-tree. In: Australasian Conference on Information Security and Privacy. pp. 205–223. Springer (2025)
18. Buser, M., Liu, J.K., Steinfeld, R., Sakzad, A.: Post-quantum id-based ring signatures from symmetric-key primitives. In: ACNS. pp. 892–912. Springer (2022)
19. Chase, M., Derler, D., Goldfeder, S., Orlandi, C., Ramacher, S., Rechberger, C., Slamanig, D., Zaverucha, G.: Post-quantum zero-knowledge and signatures from symmetric-key primitives. In: Proceedings of the 2017 ACM CCS. pp. 1825–1842 (2017)
20. Chatterjee, R., Chung, K.M., Liang, X., Malavolta, G.: A note on the post-quantum security of (ring) signatures. In: IACR International Conference on Public-Key Cryptography. pp. 407–436. Springer (2022)
21. Chiang, J.H.Y., Damgård, I., Duro, W.R., Engan, S., Kolby, S., Scholl, P.: Post-quantum threshold ring signature applications from vole-in-the-head. Cryptology ePrint Archive (2025)
22. Chung, K., Fehr, S., Huang, Y., Liao, T.: On the compressed-oracle technique, and post-quantum security of proofs of sequential work. In: EUROCRYPT (2). Lecture Notes in Computer Science, vol. 12697, pp. 598–629. Springer (2021)
23. Cramer, R., Damgård, I., Schoenmakers, B.: Proofs of partial knowledge and simplified design of witness hiding protocols. In: Annual International Cryptology Conference. pp. 174–187. Springer (1994)
24. Cui, H., Liu, H., Yan, D., Yang, K., Yu, Y., Zhang, K.: Resolved: Shorter signatures from regular syndrome decoding and vole-in-the-head. In: IACR International Conference on Public-Key Cryptography. pp. 229–258. Springer (2024)
25. van Dam, W., Hallgren, S.: Efficient quantum algorithms for shifted quadratic character problems. arXiv preprint quant-ph/0011067 (2000)
26. Damgård, I.B.: On the randomness of legendre and jacobi sequences. In: ASIACRYPT. pp. 163–172. Springer (1988)
27. Derler, D., Ramacher, S., Slamanig, D.: Post-quantum zero-knowledge proofs for accumulators with applications to ring signatures from symmetric-key primitives. In: PQCrypto. pp. 419–440. Springer (2018)
28. Dobraunig, C., Kales, D., Rechberger, C., Schafneger, M., Zaverucha, G.: Shorter signatures based on tailor-made minimalist symmetric-key crypto. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. pp. 843–857 (2022)
29. Don, J., Fehr, S., Majenz, C., Schaffner, C.: Security of the fiat-shamir transformation in the quantum random-oracle model. In: Annual International Cryptology Conference. pp. 356–383. Springer (2019)
30. Don, J., Fehr, S., Majenz, C., Schaffner, C.: Efficient nizks and signatures from commit-and-open protocols in the qrom. In: Annual International Cryptology Conference. pp. 729–757. Springer (2022)
31. Don, J., Fehr, S., Majenz, C., Schaffner, C.: Efficient nizks and signatures from commit-and-open protocols in the QROM. In: CRYPTO (2). Lecture Notes in Computer Science, vol. 13508, pp. 729–757. Springer (2022)
32. Don, J., Fehr, S., Majenz, C., Schaffner, C.: Online-extractability in the quantum random-oracle model. In: EUROCRYPT (3). Lecture Notes in Computer Science, vol. 13277, pp. 677–706. Springer (2022)

33. Esgin, M.F., Steinfeld, R., Zhao, R.K.: Matricot+: More efficient post-quantum private blockchain payments. In: IEEE S&P. pp. 1281–1298. IEEE (2022)
34. Feneuil, T., Rivain, M.: Threshold computation in the head: Improved framework for post-quantum signatures and zero-knowledge arguments. *Journal of Cryptology* **38**(3), 28 (2025)
35. Fiat, A., Shamir, A.: How to prove yourself: Practical solutions to identification and signature problems. In: Conference on the theory and application of cryptographic techniques. pp. 186–194. Springer (1986)
36. Frixons, P., Schrottenloher, A.: Quantum security of the legendre prf. *Mathematical Cryptology* **1**(2), 52–69 (2021)
37. Giacomelli, I., Madsen, J., Orlandi, C.: {ZKBoo}: faster {Zero-Knowledge} for boolean circuits. In: 25th usenix security symposium (usenix security 16). pp. 1069–1083 (2016)
38. Goel, A., Green, M., Hall-Andersen, M., Kaptchuk, G.: Efficient set membership proofs using mpc-in-the-head. *Proceedings on Privacy Enhancing Technologies* **2**, 304–324 (2022)
39. Grassi, L., Rechberger, C., Rotaru, D., Scholl, P., Smart, N.P.: Mpc-friendly symmetric key primitives. In: Proceedings of the 2016 ACM CCS. pp. 430–443 (2016)
40. Guo, X., Yang, K., Wang, X., Zhang, W., Xie, X., Zhang, J., Liu, Z.: Half-tree: Halving the cost of tree expansion in cot and dpf. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 330–362. Springer (2023)
41. Haque, A., Scafuro, A.: Threshold ring signatures: new definitions and post-quantum security. In: IACR International Conference on Public-Key Cryptography. pp. 423–452. Springer (2020)
42. Hülsing, A., Rausch, L., Buchmann, J.: Optimal parameters for xmss mt. In: International conference on availability, reliability, and security. pp. 194–208. Springer (2013)
43. Ishai, Y., Kushilevitz, E., Ostrovsky, R., Sahai, A.: Zero-knowledge from secure multiparty computation. In: Proceedings of the thirty-ninth annual ACM ToC. pp. 21–30 (2007)
44. Kales, D., Zaverucha, G.: Improving the performance of the picnic signature scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems* pp. 154–188 (2020)
45. Kaludjerović, N., Kleinjung, T., Kostic, D.: Improved key recovery on the legendre prf. *Cryptology ePrint Archive* (2020)
46. Katz, J., Kolesnikov, V., Wang, X.: Improved non-interactive zero knowledge with applications to post-quantum signatures. In: Proceedings of the 2018 ACM CCS. pp. 525–537 (2018)
47. Khovratovich, D.: Key recovery attacks on the legendre prfs within the birthday bound. *Cryptology ePrint Archive* (2019)
48. Lamport, L.: Constructing digital signatures from a one way function (1979)
49. Lyubashevsky, V., Nguyen, N.K.: Bloom: Bimodal lattice one-out-of-many proofs and applications. In: ASIACRYPT. pp. 95–125. Springer (2022)
50. Lyubashevsky, V., Nguyen, N.K., Seiler, G.: Smile: set membership from ideal lattices with applications to ring signatures and confidential transactions. In: CRYPTO. pp. 611–640. Springer (2021)
51. May, A., Zwyedinger, F.: Legendre prf (multiple) key attacks and the power of preprocessing. In: 2022 IEEE 35th Computer Security Foundations Symposium (CSF). pp. 428–438. IEEE (2022)

52. Merkle, R.C.: A certified digital signature. In: *Asiacrypt*. pp. 218–238. Springer (1989)
53. Pointcheval, D., Stern, J.: Security proofs for signature schemes. In: *International conference on the theory and applications of cryptographic techniques*. pp. 387–398. Springer (1996)
54. Rényi, A.: On measures of entropy and information. In: *Proceedings of the fourth Berkeley symposium on mathematical statistics and probability*, volume 1: contributions to the theory of statistics. vol. 4, pp. 547–562. University of California Press (1961)
55. Roy, L.: Softspokenot: Quieter ot extension from small-field silent vole in the minicrypt model. In: *Annual International Cryptology Conference*. pp. 657–687. Springer (2022)
56. Delpech de Saint Guilhem, C., Orsini, E., Tanguy, T.: Limbo: Efficient zero-knowledge mpcith-based arguments. In: *ACM CCS*. pp. 3022–3036 (2021)
57. de Saint Guilhem, C.D., De Meyer, L., Orsini, E., Smart, N.P.: Bbq: using aes in picnic signatures. In: *SAC*. pp. 669–692. Springer (2019)
58. Seres, I.A., Horváth, M., Burcsi, P.: The legendre pseudorandom function as a multivariate quadratic cryptosystem: security and applications. *Applicable Algebra in Engineering, Communication and Computing* pp. 1–31 (2023)
59. Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM review* **41**(2), 303–332 (1999)
60. Unruh, D.: Quantum proofs of knowledge. In: *Annual international conference on the theory and applications of cryptographic techniques*. pp. 135–152. Springer (2012)
61. Unruh, D.: Non-interactive zero-knowledge proofs in the quantum random oracle model. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. pp. 755–784. Springer (2015)
62. Unruh, D.: Revocable quantum timed-release encryption. *Journal of the ACM (JACM)* **62**(6), 1–76 (2015)
63. Unruh, D.: Revocable quantum timed-release encryption. *J. ACM* **62**(6), 49:1–49:76 (2015)
64. Unruh, D.: Computationally binding quantum commitments. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. pp. 497–527. Springer (2016)
65. Van Dam, W., Hallgren, S., Ip, L.: Quantum algorithms for some hidden shift problems. *SIAM Journal on Computing* **36**(3), 763–778 (2006)
66. Xue, Y., Lu, X., Au, M.H., Zhang, C.: Efficient linkable ring signatures: new framework and post-quantum instantiations. In: *European Symposium on Research in Computer Security*. pp. 435–456. Springer (2024)
67. Yuen, T.H., Esgin, M.F., Liu, J.K., Au, M.H., Ding, Z.: Dualring: generic construction of ring signatures with efficient instantiations. In: *Annual International Cryptology Conference*. pp. 251–281. Springer (2021)
68. Zhandry, M.: How to record quantum queries, and applications to quantum indistinguishability. In: *CRYPTO (2)*. *Lecture Notes in Computer Science*, vol. 11693, pp. 239–268. Springer (2019)
69. Zhang, X., Steinfeld, R., Esgin, M.F., Liu, J.K., Liu, D., Ruj, S.: Loquat: A snark-friendly post-quantum signature based on the legendre prf with applications in ring and aggregate signatures. In: *Annual International Cryptology Conference*. pp. 3–38. Springer (2024)

70. Zhang, X., Steinfeld, R., Liu, J.K., Esgin, M.F., Liu, D., Ruj, S.: Dualring-prf: Post-quantum (linkable) ring signatures from legendre and power residue prfs. In: Australasian Conference on Information Security and Privacy. pp. 124–143. Springer (2024)

Appendix

A Attacks against Legendre and Power Residue PRFs

The security of Legendre and power residue PRFs has been extensively studied over the past several decades.

Attacks on Shifted Legendre Symbol Problem (SLSP). Let p be an odd prime and K be a uniformly random value in $\mathbb{F}_p$. The SLSP requires an adversary to find the hidden shift K with *oracle access* to the PRF, which on the adversarial input $a \in \mathbb{F}_p$, outputs the Legendre symbol $\left(\frac{K+a}{p}\right)$.

It is known that if the adversary is allowed to query the PRF in a quantum mechanism superposition, then a polynomial-time quantum algorithm can determine K and break the pseudo-randomness of the PRF, using just a single query [25, 65]. However, if the oracle can only be queried classically, Khovratovich [47] showed a memoryless collision search algorithm with $O(\sqrt{p} \log p)$ Legendre symbol evaluations when $\sqrt{p} \log p$ queries are available. Recent works concurrently improved the attack presented in [47], either by exploiting the multiplicative property of the Legendre function [13, 45] or by allowing input-independent pre-computations [51].

In particular, if the number of oracle queries is bounded by $L \leq \sqrt[4]{p}$, [13] presented a table-based collision search algorithm that recovers the hidden shift with a time complexity of $O\left(\frac{p \log^2 p}{L^2}\right)$ Legendre symbol evaluations and a memory cost of $O(L^2)$. [45] shows a similar table-based collision attack that further reduces the run time complexity to $O(\sqrt{p} \log \log p)$ on a $\Theta(\log p)$ -bit machine by using $\sqrt[4]{p \log^2 p \log \log p}$ queries and $O(\sqrt{p} \log \log p \log p)$ memory. By allowing pre-computation [51], the query complexity can be brought down (from $\sqrt{p} \log p$) to $O(\sqrt[3]{p})$, with $O\left(\sqrt[3]{p^2}\right)$ (offline) run time and $O(\sqrt[3]{p})$ memory. There also exists a multiple-key attack against the Legendre PRF with $O(\sqrt{mp})$ time/query complexity to recover m hidden shifts [51]. Another work [58] shows that recovering the hidden shift of the Legendre PRF is equivalent to solving a specific family of multivariate quadratic (MQ) equation systems, and algebraic cryptanalysis does not improve previous results.

Furthermore, [45] shows that if we allow the classical memory with quantum random access (QRACM) model, the (classical) query complexity, computation complexity and memory complexity (in QRACM) can be improved to $O(\sqrt[6]{p})$, $O(\sqrt[3]{p})$, and $O(\sqrt[3]{p})$, respectively. As demonstrated in [36], there exists a quantum attack on the SLSP with $2^{O(\sqrt{\log_2 p})} \sqrt[3]{p}$ time complexity, while using $\sqrt[3]{p}$ classical queries and a subexponential number of qubits memory. The key advantage of the attack in [36] is that they removed the requirement of QRACM, as there is no practical and scalable implementation of QRACM at the moment.

Attacks on Key Recovery Let us first define the adversary's advantage in breaking the NP relation $\mathcal{R}$.

Definition 25 (Advantage in Breaking NP relation $\mathcal{R}$). Let $\mathcal{A}$ be a PPT adversary. We define the advantage of $\mathcal{A}$ in breaking the relation $\mathcal{R} := \{R_\lambda\}_{\lambda \in \mathbb{N}}$

under distribution D as

$$\text{Adv}_{\mathcal{A}}^{\mathcal{R}}(1^\lambda) := \Pr \left[(x, w') \in R_\lambda : (x, w) \leftarrow D(R_\lambda); w' \leftarrow \mathcal{A}(x) \right].$$

We say that the relation $\mathcal{R}$ is hard with respect to distribution D if, for any PPT adversary $\mathcal{A}$, the advantage satisfies $\text{Adv}_{\mathcal{A}}^{\mathcal{R}}(1^\lambda) \leq \text{negl}(\lambda)$.

Let $\mathcal{R} = \mathcal{R}_{\mathcal{L}^t}$, we have $\text{Adv}_{\mathcal{A}}^{\mathcal{R}_{\mathcal{L}^t}}(1^\lambda)$ denote the advantage of $\mathcal{A}$ in the key recovery problem for the Legendre and power residue PRFs. Unlike the SLSP, in the key recovery problem, the adversary loses the ability to query the PRF. Namely, the adversary is required to find the hidden shift K with only a small amount of random, uncontrolled PRF evaluations (i.e., public key).

Notably, the polynomial-time quantum attacks introduced in [25, 65] become infeasible in the context of the key recovery since the adversary is not able to perform quantum superposition queries to the PRF. Moreover, if the number of oracle calls is bounded by L , the complexities of key recovery attacks in [47, 13, 45] become $O\left(\frac{p \log p}{L}\right)$, $O\left(\frac{p \log^2 p}{L^2}\right)$ and $O\left(\frac{p \log p \log \log p}{L^2}\right)$, respectively. These complexities are exponential in relation to the security parameter, especially when L is sufficiently small. It is crucial to emphasize that the enhancements proposed in [13, 45] are reliant on the sequential characteristics of Legendre PRF queries. Consequently, it remains an open question whether these attacks can be extended to random point queries. In addition, it is worth noting that the attacks described in [36, 51] are not applicable, as both of them rely on a substantial number of oracle queries.

For **power residue symbols**, the best known attack is due to [13], in which the number of queries is bounded by $L \leq \sqrt[4]{p}$, and the time complexity is $O\left(\frac{p \log^2 p}{L^2 \log^2 t}\right)$ power residue symbol evaluations with $O(L^2 \log t)$ memory, especially for small t . A better attack when t is a large value has a run time of $O\left(\frac{p \log^2 p}{L t \log^2 t}\right)$ with $O(L \log t)$ memory. Overall, for both Legendre and power residue symbols, we have $\text{Adv}_{\mathcal{A}}^{\mathcal{R}_{\mathcal{L}^t}}(1^\lambda) \leq \text{negl}(\lambda)$.

B Proofs

B.1 Security Proofs of BAVC

Extractable Binding. We first prove the extractable binding of BAVC in the (Q)ROM used in this paper.

Lemma 7 (Extractable Binding in the ROM). *The BAVC protocol is extractable binding in the ROM.*

Proof. For any PPT adversary $\mathcal{A}$, we construct a PPT extractor $\mathcal{E}_{\text{BAVC}}$ that simulates the random oracle $\mathcal{H}$ via lazy sampling while maintaining a query list L_H . By the property of the classical random oracle, this simulation is perfect, and hence $\epsilon_{\text{sim}} = 0$.

When $\mathcal{A}_1$ outputs com , the extractor $\mathcal{E}_{\text{BAVC}}$ checks the query list L_H to extract $\mathbf{L}^* = ((l_{e,\ell})_{\ell \in [0, L-1]})_{e \in [M]}$ such that there exists $\mathbf{R}^c = ((r_{e,\ell}^c)_{\ell \in [0, L-1]})_{e \in [M]}$ satisfying

$$(1\|l_{e,\ell}, r_{e,\ell}^c) \in L_H \text{ and } (2\|\mathbf{R}^c, \text{com}) \in L_H.$$

If the extraction fails, Ext_{BAVC} aborts.

Then $\mathcal{E}_{\text{BAVC}}$ computes $\mathbf{R}^* = ((r_{e,\ell}^*)_{\ell \in [0, L-1]})_{e \in [M]}$, where $r_{e,\ell}^* = H(0\|l_{e,\ell})$, and outputs $\mathbf{R}^*$.

Assume $\mathcal{A}$ and BAVC.Verify do not find a collision in $\mathcal{H}$. Denote this event as **Event**. By the randomness of the RO, we have $\Pr[\text{Event}] \leq \frac{q^2}{2^{2\lambda}}$, where q is the total number of RO queries. According to the definition of extractable binding, we have

$$\epsilon_{\text{ext}}(\lambda) \leq \Pr[\text{rela}(\mathbf{R}^*, \bar{\mathbf{R}}', S) | \text{com} = \text{com}', \bar{\mathbf{R}} = \bar{\mathbf{R}}'],$$

where

$$(\bar{\mathbf{R}}, \text{decom}_S, S) \leftarrow \mathcal{A}_2^{\mathcal{E}_{\text{BAVC}}}(\text{com}), (\text{com}', \bar{\mathbf{R}}') \leftarrow \text{BAVC.Verify}^{\mathcal{E}_{\text{BAVC}}}(\text{decom}_S, S).$$

Denote $\mathbf{R}'^c := ((r_{e,\ell}'^c)_{\ell \in [0, L-1]})_{e \in [M]}$ and $\bar{\mathbf{R}}' = ((r_{e,\ell}')_{\ell \in [0, L-1] \setminus \{\bar{\ell}_e\}})_{e \in [M]}$. According to the construction of BAVC , we have $\text{com}' = H(1\|\mathbf{R}'^c)$. Thus, $\mathbf{R}^c = \mathbf{R}'^c$ since there is no collision. If $\mathbf{R}^c = \mathbf{R}'^c$, then $r_{e,\ell}^c = r_{e,\ell}'^c$ and hence $r_{e,\ell}' = r_{e,\ell}^* = H(0\|l_{e,\ell})$. In general, we obtain

$$\epsilon_{\text{ext}}(\lambda) \leq \Pr[\mathcal{B} \text{ aborts}] + \Pr[\text{Event}] \leq \Pr[\mathcal{E}_{\text{BAVC}} \text{ aborts}] + \frac{q^2}{2^{2\lambda}}.$$

Next, we analyze the probability that $\mathcal{E}_{\text{BAVC}}$ aborts:

- If there is no $\mathbf{R}^c$ such that $(2\|\mathbf{R}^c, \text{com}) \in L_H$, then $\mathcal{A}$ did not query the RO to obtain com . In this case, when BAVC.Vf generates com by querying the RO, the output will be uniformly random. Hence, $\Pr[\text{com}' = \text{com}] = \frac{1}{2^\lambda}$.
- If there exists $\mathbf{R}^c$ such that $(2\|\mathbf{R}^c, \text{com}) \in L_H$ but, for some $e, \ell \in [M] \times [0, L-1]$, we have $(1\|l_{e,\ell}, r_{e,\ell}^c) \notin L_H$, then the value $H(1\|l_{e,\ell})$ is not fixed yet and will be generated uniformly at random only when $l_{e,\ell}$ is queried later. Under the condition that $\text{com} = \text{com}'$ and $\mathbf{R} = \mathbf{R}'$, this freshly generated value must coincidentally equal $r_{e,\ell}^c$. Hence, in this case the extractor aborts with probability at most $\frac{ML}{2^{2\lambda}}$.

Therefore, in general we conclude

$$\epsilon_{\text{ext}}(\lambda) \leq \frac{q^2 + ML + 1}{2^{2\lambda}}.$$

□

Lemma 8 (Extractable Binding in the QROM). *The BAVC protocol is extractable binding in the QROM.*

Proof. The proof idea is similar to the proof in the ROM. Specially, the PPT extractor simulates the quantum random oracle $\mathcal{H}$ via CStO and generates the database D_H . According to the property of classical RO, the simulation is perfect. Thus, $\epsilon_{sim} = 0$. Since $\mathcal{P}^*$ and $\mathcal{E}_{\text{BAVC}}$ are quantum PPT algorithms. To simple proof, $\mathcal{E}_{\text{BAVC}}$ will not check the databased and abort same as in the ROM. After $\mathcal{P}^*$ outputs **com**, $\mathcal{E}_{\text{BAVC}}$ apply quantum measure to the database D_H , to extract $\mathbf{L}^* = ((l_{e,\ell})_{\ell \in [0, L-1]})_{e \in [M]}$.

Let $q = q_1 + q_2$ be the total number of $\mathcal{P}^*$'s quantum queries to $\mathcal{H}$, and q_1 is the number of P_1^* 's quantum queries to $\mathcal{H}$ and q_2 is the number of P_2^* 's quantum queries to $\mathcal{H}$. Let q_V be the number of **BAVC.Verify**'s classical queries to $\mathcal{H}$. To analyze the upper bound of extraction success. We define the database properties SUC and CL as follows:

$$\text{SUC} := \left\{ \begin{array}{l} \exists \mathbf{L}^* \text{ s.t. } (1 \| l_{e,\ell}, \mathbf{R}) \in D \text{ satisfies} \\ D : \text{BAVC.Verify}(\text{decom}_S, S) = (\text{com}, \bar{\mathbf{R}}) \text{ for } \text{com} = D(2 \| \mathbf{R}) \\ \text{and } (\mathbf{R}^*, \bar{\mathbf{R}}, S) \notin R \text{ for } r_{e,\ell}^* = D(0 \| l_{e,\ell}) \end{array} \right\}$$

and

$$\text{CL} := \{ D : \exists x \neq x' \text{ s.t. } D(x) = D(x') \neq \perp \}.$$

We have $\epsilon_{ext} \leq \max_{\mathcal{A}} \Pr[D_H \in \text{SUC} \cup \text{CL}] + \frac{q_v^2 + ML + 1}{2^{2\lambda}}$.

According to basic properties of the quantum transition capacity and Lemma 5 (Lemma 5.6 in [32]), we have

$$\llbracket \perp \xrightarrow{q_1} \text{CL} \rrbracket \leq \sum_{i=0}^{q_1-1} \llbracket \text{SZ}_{\leq i} \setminus \text{CL} \rightarrow \text{CL} \rrbracket \leq 2eq_1 \sqrt{\frac{q_1}{2^{2\lambda}}},$$

and

$$\llbracket \perp \xrightarrow{q_1} \text{SUC} \rrbracket \leq \sum_{i=0}^{q_1-1} \llbracket \text{SZ}_{\leq i} \setminus \text{SUC} \rightarrow \text{SUC} \rrbracket.$$

We analyze the upper bound of $\llbracket \text{SZ}_{\leq i} \setminus \text{SUC} \rightarrow \text{SUC} \rrbracket$ as follows:

- $\llbracket \text{SZ}_{\leq i} \setminus \text{SUC} \rightarrow \text{SUC} \rrbracket$. We consider arbitrary but fixed $D \in \mathcal{D}$ and input $x \in X$. We assume that $D(x) = \perp$, x is fresh. Define $\mathbf{P} := \text{SZ}_{\leq s} \setminus \text{SUC}$ and $\mathbf{P}' := \text{SUC}$.

- Case 1: $D \in \text{SUC}$ and $\perp \in \text{SUC}|_{D|x} = \mathbf{P}'|_{D|x}$. Set $\mathbf{L} := \mathbf{P}|_{D|x}$. Furthermore,

$$u \in \mathbf{L} \leftrightarrow D[x \rightarrow u] \in \mathbf{P} \rightarrow D[x \rightarrow u] \notin \text{SUC} \rightarrow u = \text{com}.$$

Thus, we can bound

$$\Pr[U \in \mathbf{L}] \leq \Pr[U = \text{com}] \leq \frac{1}{2^{2\lambda}}.$$

- Case 2: $D \notin \text{SUC}$ and the prefix of x is 1. Because $D(x) = \perp$, $\perp \notin \mathbf{P}'|_{D|x}$. Set $\mathbf{L} := \mathbf{P}'|_{D|x}$. Therefore,

$$u \in \mathbf{L} \leftrightarrow D[x \rightarrow u] \in \mathbf{P}' \rightarrow u = \text{com}.$$

Thus, we can bound

$$\Pr[U \in \mathbb{L}] \leq \frac{1}{2^{2\lambda}}.$$

- Case 3: $D \notin \text{SUC}$ and $[x]^1 = 2$. Because $D(x) = \perp$, $\perp \notin P'|_{D|x}$. Set $\mathbb{L} := P'|_{D|x}$. Then,

$$u \in \mathbb{L} \leftrightarrow D[x \rightarrow u] \in P' \rightarrow u \in \{r_{e,\ell}^c\}_{e \in [M], \ell \in [0, L-1]}.$$

Thus, we can bound

$$\Pr[U \in \mathbb{L}] \leq \frac{ML}{2^{2\lambda}}.$$

We have

$$\llbracket \text{SZ}_{\leq i} \setminus \text{SUC} \rightarrow \text{SUC} \rrbracket \leq \sqrt{10 \max\left\{\frac{1}{2^{2\lambda}}, \frac{ML}{2^{2\lambda}}\right\}}.$$

In general,

$$\epsilon_{ext} \leq \frac{60q^3 + 10MLq^2}{2^{2\lambda}} + \frac{q_v^2 + ML + 1}{2^{2\lambda}}.$$

□

Selective Hiding. We prove the selective hiding property of BAVC in the (Q)ROM considered in this work.

Lemma 9 (Selective Hiding in the ROM [5]). *The BAVC protocol satisfies selective hiding in the ROM.*

Lemma 10 (Selective Hiding in the QROM). *The BAVC protocol satisfies selective hiding in the QROM.*

Proof. The proof is nearly identical to the classical ROM case. The only distinction arises in handling the adversary's queries to the QRO: when reprogramming the QRO's responses at $x \in \{l_{e,\ell}\}_{e \in [M]}$ to random values, we must employ programming techniques specific to the QROM. In particular, we rely on the OW2H lemma [63] to bound the adversary's probability of detecting such reprogramming. □

B.2 Proof of Theorem 2

We first show the distribution of t -th power residue PRFs with respect to the arithmetic progression list is nearly uniformly random.

Lemma 11. *For c integers $s_0, \dots, s_{c-1} \in \mathbb{Z}_t$, a fixed $b \in \mathbb{F}_p^*$, and an odd prime $p > c$ with $t|p-1$,*

$$\left| N_{p,b}(s_0, \dots, s_{c-1}) - \frac{p}{tc} \right| < O_{c,t}(\sqrt{p}) + \frac{c}{t}$$

where $N_{p,b}(s_0, \dots, s_{c-1}) := \#\{a \in \mathbb{F}_p : \mathcal{L}_0^t(a + \ell b) = s_\ell \text{ for } \ell = 0, \dots, c-1\}$ and $O_{c,t}(\sqrt{p}) = ct\sqrt{p}$.

Proof. Let $\xi = e^{2\pi i/t}$ be a primitive t -th root of unity, and let $\mu_t := \{\xi^k | 0 \leq k < t\} \subset \mathbb{C}$ denote the set of all complex t -th roots of unity. It is obvious that μ_t is isomorphic to $\mathbb{Z}_t$. That is, there exists a bijection $\tau : \mathbb{Z}_t \rightarrow \mu_t$, defined by $\tau(k) = \xi^k$ for $k \in \mathbb{Z}_t$.

Let $\chi : \mathbb{F}_p^* \rightarrow \mu_t$ be a multiplicative character of order t derived from the t -th power residue symbol. We can define $\chi(x) = \xi^{\mathcal{L}_0^t(x)}$ for $x \in \mathbb{F}_p^*$, which is indeed a group homomorphism since $\chi(xy) = \xi^{\mathcal{L}_0^t(xy)} = \xi^{\mathcal{L}_0^t(x) + \mathcal{L}_0^t(y)} = \xi^{\mathcal{L}_0^t(x)} \cdot \xi^{\mathcal{L}_0^t(y)} = \chi(x)\chi(y)$. We extend χ to $\mathbb{F}_p$ by defining $\chi(0) = 0$.

The target vector $(s_0, \dots, s_{c-1}) \in \mathbb{Z}_t^c$ is mapped to $(\omega_0, \dots, \omega_{c-1}) \in \mu_t^c$ via the isomorphism τ (i.e., $\omega_\ell = \tau(s_\ell)$ for all $\ell \in [0, c-1]$). Hence, it is equivalent to prove the lemma for $N_{p,b}(\omega_0, \dots, \omega_{c-1}) := \{\#a \in \mathbb{F}_p : \chi(a + b\ell) = \omega_\ell \text{ for all } 0 \leq \ell \leq c-1\}$.

The key fact we use is, for some $\omega, \eta \in \mu_t$, if $\eta = \omega$, then $(\omega^{-1}\eta) = \xi^0 = 1$, hence $\sum_{j=0}^{t-1} (\omega^{-1}\eta)^j = \sum_{j=0}^{t-1} 1^j = t$. On the other hand, if $\eta \neq \omega$, $(\omega^{-1}\eta) = \xi^k$ for some $0 < k < t$. Let $r = \xi^k \neq 1$, then we have the finite geometric sum $\sum_{j=0}^{t-1} r^j = \frac{1-r^t}{1-r}$. Furthermore, since $r \in \mu_t$, we have $r^t = 1$. Hence, when $\eta \neq \omega$, we have $\sum_{j=0}^{t-1} (\omega^{-1}\eta)^j = 0$.

We define the following equation, where for $\omega \in \mu_t$ and $x \in \mathbb{F}_p^*$, we have

$$\frac{1}{t} \left(\sum_{j=0}^{t-1} (\omega^{-1}\chi(x))^j \right) = \begin{cases} 1 & \text{if } \omega = \chi(x) \\ 0 & \text{if } \omega \neq \chi(x) \end{cases}$$

By replacing x with $(a+b\ell)$ under the condition that $a+b\ell \neq 0$ for $\ell \in [0, c-1]$, we define the function $F_b(a)$ as

$$F_b(a) = \prod_{\ell=0}^{c-1} \frac{1}{t} \left(\sum_{j=0}^{t-1} (\omega_\ell^{-1}\chi(a+b\ell))^j \right)$$

Note that $F_b(a) = 1$ if and only if for all $\ell \in [0, c-1]$, $\omega_\ell = \chi(a+b\ell)$. Otherwise (i.e., there exists some $\ell \in [0, L-1]$ such that $\omega_\ell \neq \chi(a+b\ell)$), $F_b(a) = 0$.

Hence, we have

$$\begin{aligned} N_{p,b}(\omega_0, \dots, \omega_{c-1}) &= \sum_{\substack{a \in \mathbb{F}_p \\ a, a+b, \dots, a+(c-1)b \neq 0}} F_b(a) \\ &= \sum_{\substack{a \in \mathbb{F}_p \\ a, a+b, \dots, a+(c-1)b \neq 0}} \prod_{\ell=0}^{c-1} \frac{1}{t} \left(\sum_{j=0}^{t-1} (\omega_\ell^{-1}\chi(a+b\ell))^j \right) \end{aligned}$$

Let us first consider the case when $(a+b\ell) = 0$ for some $\ell \in [0, c-1]$. By convention, let us define $0^0 = 1$. Observe that if $(a+b\ell) = 0$, we have $\frac{1}{t} \sum_{j=0}^{t-1} (\omega_\ell^{-1} \cdot 0)^j = \frac{1}{t}$. Furthermore, there exists at most one $(a+b\ell) = 0$ in

each sequence (i.e., when $a = -b\ell$). Hence, for any sequence with $(a + b\ell) = 0$ we have

$$\left| \prod_{\ell=0}^{c-1} \frac{1}{t} \left(\sum_{j=0}^{t-1} (\omega_\ell^{-1} \chi(a + b\ell))^j \right) \right| \leq \frac{1}{t}.$$

For a fixed $b \in \mathbb{F}_p^*$, there are exactly c such numbers $a = 0, a = -b, a = -2b, \dots, a = -(c-1)b$ in $\mathbb{F}_p$. Let $|e_c| \leq c$, then we have

$$\begin{aligned} N_{p,b}(\omega_0, \dots, \omega_{L-1}) &= \sum_{a \in \mathbb{F}_p} \prod_{\ell=0}^{c-1} \frac{1}{t} \left(\sum_{j=0}^{t-1} (\omega_\ell^{-1} \chi(a + b\ell))^j \right) + \frac{e_c}{t} \\ &= \frac{1}{t^c} \sum_{a \in \mathbb{F}_p} \left(\prod_{\ell=0}^{c-1} \left(\sum_{j=0}^{t-1} (\omega_\ell^{-1} \chi(a + b\ell))^j \right) \right) + \frac{e_c}{t} \end{aligned}$$

We now expand the product inside the sum: for each $a \in \mathbb{F}_p$, let $F'_b(a) = \prod_{\ell=0}^{c-1} \left(\sum_{j=0}^{t-1} (\omega_\ell^{-1} \chi(a + b\ell))^j \right)$

$$\begin{aligned} F'_b(a) &= \prod_{\ell=0}^{c-1} \left(1 + \sum_{j=1}^{t-1} (\omega_\ell^{-1} \chi(a + b\ell))^j \right) \\ &= 1 + \sum_{S \subset \{0, \dots, c-1\} / \emptyset} \left(\prod_{\ell \in S} \left(\sum_{j=1}^{t-1} (\omega_\ell^{-1} \chi(a + b\ell))^j \right) \right) \\ &= 1 + \sum_{S \subset \{0, \dots, c-1\} / \emptyset} \left(\sum_{\substack{(j_\ell), \ell \in S \\ 1 \leq j_\ell \leq t-1}} \left(\prod_{\ell \in S} \omega_\ell^{-j_\ell} \right) \left(\prod_{\ell \in S} \chi(a + b\ell)^{j_\ell} \right) \right) \end{aligned}$$

Let $\omega_{S,j} = \prod_{\ell \in S} \omega_\ell^{-j_\ell} \in \mu_t$. We have

$$\begin{aligned} &N_{p,b}(\omega_0, \dots, \omega_{c-1}) \\ &= \frac{1}{t^c} \sum_{a \in \mathbb{F}_p} F'_b(a) + \frac{e_c}{t} \\ &= \frac{p}{t^c} + \frac{1}{t^c} \sum_{a \in \mathbb{F}_p} \left(\sum_{S \subset \{0, \dots, c-1\} / \emptyset} \left(\sum_{\substack{(j_\ell), \ell \in S \\ 1 \leq j_\ell \leq t-1}} \omega_{S,j} \left(\prod_{\ell \in S} \chi(a + b\ell)^{j_\ell} \right) \right) \right) + \frac{e_c}{t} \\ &= \frac{p}{t^c} + \frac{1}{t^c} \sum_{S \subset \{0, \dots, c-1\} / \emptyset} \left(\sum_{\substack{(j_\ell), \ell \in S \\ 1 \leq j_\ell \leq t-1}} \omega_{S,j} \left(\sum_{a \in \mathbb{F}_p} \chi \left(\prod_{\ell \in S} (a + b\ell)^{j_\ell} \right) \right) \right) + \frac{e_c}{t}, \end{aligned}$$

which is the desired sum over $a \in \mathbb{F}_p$.

Let us define $f_{S,j}(x) = \prod_{\ell \in S} (x + b\ell)^{j_\ell}$. Notably, $f_{S,j}(x)$ with degree $d = \sum_{\ell \in S} j_\ell$ is not a perfect t -th power since $1 \leq j_\ell \leq t-1$ and $b \neq 0$. Hence, we can apply the Weil's bound, which says for the non-trivial multiplicative character χ of order $t > 1$, if $f \in \mathbb{F}_p[X]$ has d distinct roots and is not a perfect t -th power, then

$$\left| \sum_{x \in \mathbb{F}_p} \chi(f(x)) \right| \leq (d-1)\sqrt{p}.$$

This inequality is an equality if $\deg(f) = 1$, and generally is a strict inequality if $\deg(f) \geq 2$. Applying the Weil's bound to the polynomial $f_{S,j}(x)$, which is not a perfect t -th power, we get

$$\left| \sum_{a \in \mathbb{F}_p} \chi(f_{S,j}(a)) \right| \leq \left(\sum_{\ell \in S} j_\ell - 1 \right) \sqrt{p} \leq (c(t-1) - 1) \sqrt{p}$$

In short, the inner sum over $a \in \mathbb{F}_p$ is bounded by $(c(t-1) - 1)\sqrt{p} < ct\sqrt{p} = O_{c,t}(\sqrt{p})$.

Since the bound of the inner sum is independent of j_ℓ and S , we can extract the term

$$N_p(\omega_0, \dots, \omega_{c-1}) \leq \frac{p}{t^c} + \frac{1}{t^c} O_{c,t}(\sqrt{p}) \left(\sum_{S \subset \{0, \dots, L-1\} / \emptyset} \left(\sum_{\substack{(j_\ell), \ell \in S \\ 1 \leq j_\ell \leq t-1}} \omega_{S,j} \right) \right) + \frac{e_c}{t}$$

Since each $\omega_{S,j}$ satisfies $|\omega_{S,j}| = 1$, we have

$$\begin{aligned} \left| \sum_{S \subset \{0, \dots, c-1\} / \emptyset} \left(\sum_{\substack{(j_\ell), \ell \in S \\ 1 \leq j_\ell \leq t-1}} \omega_{S,j} \right) \right| &\leq \sum_{S \subset \{0, \dots, c-1\} / \emptyset} \left(\sum_{\substack{(j_\ell), \ell \in S \\ 1 \leq j_\ell \leq t-1}} |\omega_{S,j}| \right) \\ &\leq \sum_{k=1}^c \binom{c}{k} (t-1)^k = t^c - 1 \end{aligned}$$

By the triangle inequality, we have

$$\begin{aligned} \left| N_{p,b}(\omega_0, \dots, \omega_{c-1}) - \frac{p}{t^c} \right| &\leq \frac{1}{t^c} O_{c,t}(\sqrt{p})(t^c - 1) + \frac{e_c}{t} \\ &< O_{c,t}(\sqrt{p}) + \frac{c}{t} \end{aligned}$$

where $O_{c,t}(\sqrt{p}) = ct\sqrt{p}$. □

We are now ready to prove the Theorem 2.

Proof. Let X be all possible events under distribution D and U is the uniform distribution on X . For $x \in X$, Lemma 11 shows $D(x) = \frac{1}{t^c} + \frac{ct}{\sqrt{p}} + \frac{c}{pt}$, and the ideal probability is $U(x) = \frac{1}{t^c}$. Hence, $|D(x) - U(x)| \leq \delta$ for each $x \in X$ with $\delta = \frac{ct}{\sqrt{p}}(1 + \epsilon)$ for $\epsilon = \frac{1}{t^2\sqrt{p}} \ll 1$.

For the maximum Rényi divergence denoted as $R_\infty(D \parallel U) = \max_x \frac{D(x)}{U(x)}$, since $U(x) = \frac{1}{t^c}$ is the same for all x , we get

$$\begin{aligned} \frac{D(x)}{U(x)} &= t^c D(x) \\ &= (U(x) + (D(x) - U(x)))t^c \\ &= 1 + (D(x) - U(x))t^c \end{aligned}$$

Hence, we have $\max_x \frac{D(x)}{U(x)} = 1 + \max_x ((D(x) - U(x))t^c) = 1 + t^c \max_x |D(x) - U(x)|$. Based on Lemma 11, $\max_x |D(x) - U(x)| \leq \delta = \frac{ct}{\sqrt{p}}(1 + \epsilon)$, hence

$$\begin{aligned} R_\infty(D \parallel U) &= \max_x \frac{D(x)}{U(x)} \leq 1 + t^c \delta \\ &= 1 + t^c \left(\frac{ct}{\sqrt{p}}(1 + \epsilon) \right) \\ &= 1 + \frac{ct^{c+1}}{\sqrt{p}}(1 + \epsilon) \end{aligned}$$

Since $(a_0, \dots, a_{z-1}) \in \mathbb{F}_p^z$ are chosen uniformly at random, by the multiplicative property of Rényi divergence, we get

$$R_\infty(D^z \parallel U^z) = (R_\infty(D \parallel U))^z \leq \left(1 + \frac{ct^{c+1}}{\sqrt{p}}(1 + \epsilon) \right)^z$$

If $z \cdot \frac{ct^{c+1}}{\sqrt{p}}(1 + \epsilon) \leq 1$ (or equivalently, $\frac{t^c}{\sqrt{p}} \leq \frac{1}{(1+\epsilon)ctz}$), $R_\infty(D^z \parallel U^z) \leq e \leq 2.72$, by applying the standard inequality of $(1 + x)^m \leq e^{mx}$ with $mx \leq 1$. From the probability preservation of Rényi divergence, for any event $x \subseteq X^z$, we have $\Pr_{D^z}[x] \leq R_\infty(D^z \parallel U^z) \cdot \Pr_{U^z}[x] \leq 2.72 \cdot \Pr_{U^z}[x]$.

We now compute the ideal probability $\Pr_{U^z}[x]$. Let $L = z \times c$ and $b \in \mathbb{F}_p^*$ be a fixed non-zero field element. The setup algorithm uniformly random chooses $a_0, \dots, a_{z-1} \in \mathbb{F}_p^z$ and defines $\mathcal{I}' = (\mathcal{I}'_0, \dots, \mathcal{I}'_{z-1}) = ((a_j + b\ell)_{\ell \in [0, c-1]})_{j \in [0, z-1]}$. For a fixed key $K \in \mathbb{F}_p$, let us define for each $j \in [0, z-1]$, $\mathbf{s}_j = (\mathcal{L}_0^t(K + a_j), \dots, \mathcal{L}_0^t(K + a_j + b(\ell-1)))$, where $a_j \xleftarrow{\$} \mathbb{F}_p$. Then, for a fixed pair $(K', \alpha) \in \mathbb{F}_p/\{K\} \times \mathbb{Z}_t$, we define $\mathbf{s}'_j = (\mathcal{L}_0^t(K' + a_j) + \alpha, \dots, \mathcal{L}_0^t(K' + a_j + b(\ell-1)) + \alpha)$. Let $d_H(\mathbf{s}'_j, \mathbf{s}_j) = \#\{0 \leq \ell \leq c : \mathbf{s}'_j[\ell] \neq \mathbf{s}_j[\ell]\}$ be the Hamming distance between two vectors $\mathbf{s}_j$ and $\mathbf{s}'_j$.

Assume the ideal distribution of each element in $\mathbf{s}_j$ and $\mathbf{s}'_j$, the distance $d_H(\mathbf{s}_j, \mathbf{s}'_j)$ is distributed as $\mathfrak{B}(c, 1/t)$. Furthermore, since each a_j is chosen independently from $\mathbb{F}_p$, then for all $j \in [0, z-1]$, we have the probability that

$\sum_{j=0}^{zc-1} d_H(\mathbf{s}_j, \mathbf{s}'_j) \leq \beta zc$ is at most

$$\Pr[\mathfrak{B}(zc, 1/t) > (1 - \beta)zc].$$

Recall that $L = zc$ and since there exists $(p - 1)t$ possible values for (K', α) , the probability that there exists a non-trivial witness for the β -relaxed relation is at most $\Pr[\mathfrak{B}(L, 1/t) > (1 - \beta)L](p - 1)t$, under the assumption of an ideal distribution of $\mathbf{s}_j$ and $\mathbf{s}'_j$. Then, we obtain the bound on the real probability on the distribution of $\sum_{j=0}^{zc-1} d_H(\mathbf{s}_j, \mathbf{s}'_j) \geq \beta L$:

$$\Pr_{D^z}[x] \leq 2.72 \cdot \Pr[\mathfrak{B}(L, 1/t) > (1 - \beta)L](p - 1)t,$$

by applying the argument based on Rényi divergence. $\square$

B.3 Proof of Theorem 3

Proof. Let $\bar{\mathbf{R}} = (r_{e,\ell})_{\ell \in [0, L-1] \setminus \{\bar{\ell}_e\}, e \in [M]}$ be the opening of BAVC. We show that for every $e \in [M]$, the verifier computes $o_e = (K + I_{\bar{\ell}_e})r_e$, where $r_e = \sum_{\ell=0}^{L-1} r_{e,\ell}$:

$$\begin{aligned} o_e &= I_{\bar{\ell}_e} \sum_{\ell=0, \ell \neq \bar{\ell}_e}^{L-1} r_{e,\ell} - \sum_{\ell=0, \ell \neq \bar{\ell}_e}^{L-1} r_{e,\ell} I_{\ell} + \Delta K_e \\ &= I_{\bar{\ell}_e} \sum_{\ell=0, \ell \neq \bar{\ell}_e}^{L-1} r_{e,\ell} - \sum_{\ell=0, \ell \neq \bar{\ell}_e}^{L-1} r_{e,\ell} I_{\ell} + Kr_e + \sum_{\ell=0}^{L-1} I_{\ell} r_{e,\ell} \\ &= Kr_e + I_{\bar{\ell}_e} \sum_{\ell=0, \ell \neq \bar{\ell}_e}^{L-1} r_{e,\ell} + I_{\bar{\ell}_e} r_{e,\bar{\ell}_e} \\ &= Kr_e + I_{\bar{\ell}_e} \sum_{\ell=0}^{L-1} r_{e,\ell} \\ &= Kr_e + I_{\bar{\ell}_e} r_e \\ &= (K + I_{\bar{\ell}_e})r_e \end{aligned}$$

Due to the multiplicative homomorphism of the Legendre and power residue PRFs, we have $\mathcal{L}_0^t(o_e) = \mathcal{L}_0^t(K + I_{\bar{\ell}_e}) + \mathcal{L}_0^t(r_e) = pk_{\bar{\ell}_e} + T_e$. $\square$

B.4 Proof of Theorem 4

Proof. We begin by describe the extractor $\mathcal{E}$ which invokes the BAVC extractor to extract the witness $sk^* = K$ of β -approximate PRF relation.

- **Extractor Description.** For any (quantum) PPT adversary $\mathcal{P}^*$, whenever $\mathcal{P}^*$ queries $\mathcal{H}$, the (quantum) PPT extractor $\mathcal{E}$ forwards the query to $\mathcal{E}_{\text{BAVC}}$ and returns the corresponding response to $\mathcal{P}^*$. After $\mathcal{P}^*$ outputs the transcript $((A, A_\circ), S, Z)$, the extractor $\mathcal{E}$ proceeds as follows:

1. $\mathcal{E}$ parses $A = \text{com}$, $A_o = (T_e, \Delta K_e)_{e \in [M]}$, $S = \{\bar{\ell}_e\}_{e \in [M]}$, and $Z = \text{decom}_S$.
 2. $\mathcal{E}$ runs the BAVC extractor $\mathcal{E}_{\text{BAVC}}$ with the commitment com to obtain the committed vector $\mathbf{R} = (r_{e,0}, \dots, r_{e,L-1})_{e \in [M]}$.
 3. For $e \in [M]$, $\mathcal{E}$ computes the β -approximate witness candidate $K_e^* = (\Delta K_e - \sum_{\ell=0}^{L-1} I_\ell r_{e,\ell}) r_e^{-1}$, where $r_e = \sum_{\ell=0}^{L-1} r_{e,\ell}$.
- **Probability Analysis of ϵ_{sim} .** Since Ext_{BAVC} simulates $\mathcal{H}$ via lazy sampling in the ROM (or using CStO in the QROM), for any $\text{pp} \leftarrow \text{Setup}(1^\lambda)$ and $(sk, pk) \leftarrow \text{KeyGen}(\text{pp})$, it follows that $\epsilon_{\text{sim}} = 0$.
 - **Probability Analysis of ϵ_{ext} .** For $e \in [M]$, $\mathcal{E}$ computes the β -approximate witness candidate $K_e^* = (\Delta K_e - \sum_{\ell=0}^{L-1} I_\ell r_{e,\ell}) r_e^{-1}$, where $r_e = \sum_{\ell=0}^{L-1} r_{e,\ell}$. Suppose the adversary outputs $A_o^* = (\Delta K_e^*, T_e^*)_{e \in [M]}$, we define $I(\Delta K_e^*)$ be the subset of indices in $[0, L-1]$ such that for $j = 0, 1, \dots, L-1$, the equation 5 holds

$$\mathcal{L}_0^t(I_j \sum_{\ell=0, \ell \neq j}^{L-1} r_{e,\ell} - \sum_{\ell=0, \ell \neq j}^{L-1} r_{e,\ell} I_\ell + \Delta K_e^*) = pk_j + T_e^* \quad (5)$$

Therefore, in an accepting transcript, we must have $\bar{\ell}_e \in I(\Delta K_e^*)$ for all $e \in [M]$. Since the challenge indices $(\bar{\ell}_e)_{e \in [M]}$ are sampled uniformly from $[0, L-1]$, the probability of this event is $P = \prod_{e \in [M]} \frac{|I(\Delta K_e^*)|}{L}$. For each extracted candidate witness K_e^* , let us define $\beta_{K_e^*}$ be the Hamming distance of two sequences $\mathcal{L}_{K_e^*}^t(\mathcal{I}) + \mathcal{L}_0^t(r_e) \in \mathbb{Z}_t^L$ and $pk + T_e^* \in \mathbb{Z}_t^L$:

$$\beta_{K_e^*}(Q) = d_H((\mathcal{L}_{K_e^*}^t(\mathcal{I}) + (\mathcal{L}_0^t(r_e)), (T_e^* + pk))),$$

Notably, $\beta_{K_e^*}$ represents the number of $\ell \in [0, L-1]$ such that equation 5 does *not* hold, in other words, $\beta_{K_e^*} = L - |I(\Delta K_e^*)|$. If none of K_e^* is not a β -approximate witness for the statement pk , it follows that $\beta_{K_e^*} > \beta L$ for all $e \in [M]$, according to the Definition 8. Therefore, we can bound for each $e \in [M]$ $|I(\Delta K_e^*)|$ as $|I(\Delta K_e^*)| < (1 - \beta)L$. Combined with the previous argument, we have a bound on the successful cheating probability:

$$P = \prod_{e \in [M]} \frac{|I(\Delta K_e^*)|}{L} < \prod_{e \in [M]} \frac{(1 - \beta)L}{L} = (1 - \beta)^M.$$

For all

$$(b, (A, A_o), S, Z) = [\mathcal{P}^{\mathcal{E}}(pk) \leftrightarrow \mathcal{V}^{\mathcal{E}}(pk)]$$

and

$$sk \leftarrow \mathcal{E}(pk, (A, A_o), S, Z),$$

we define $\text{Event}_1 := \|(pk, sk) \notin R_\lambda\|$ and $\text{Event}_2 := \|b = 1\|$. From the definitions of PoK-OE and extractable binding, it follows directly that

$$\epsilon_{\text{ext}}(\lambda) = \Pr[\text{Event}_1 \wedge \text{Event}_2] \leq \epsilon_{\text{ext}}^{\text{BAVC}}(1^\lambda) + P,$$

where $\epsilon_{\text{ext}}^{\text{BAVC}}(1^\lambda)$ denotes the BAVC extraction error (Definition 11), and P is the bound derived above.

Therefore, when the BAVC protocol satisfies extractable binding in the (Q)ROM, the commit-and-open Σ protocol $\Pi^{\mathcal{H}}$ is a PoK-OE in the (Q)ROM. $\square$

B.5 Proof of Theorem 5

Proof. We begin by describe the simulator which simulates a Σ -protocol transcript on input the challenge S and the public key pk .

- **Simulator Construction.** On input the challenge public key $pk \in L_{\mathcal{R}}$ and any verifier challenge $S = \{\bar{\ell}_1, \dots, \bar{\ell}_M\}$, the simulator $\text{Sim}^{\mathcal{H}}$ does the following:
 1. Sample public parameters $\text{pp} \leftarrow \text{Setup}(1^\lambda)$.
 2. Invoke BAVC algorithms $(\text{decom}, \mathbf{L}) \leftarrow \text{BAVC.TreeGen}^{\text{PRG}}(\text{BAVC.pp})$ and $(\text{com}, \mathbf{R}) \leftarrow \text{BAVC.LeafCom}^{\mathcal{H}}(\mathbf{L}, \text{BAVC.pp})$ to get random messages $\mathbf{R} = (r_{e,0}, \dots, r_{e,L-1})_{e \in [M]}$, where BAVC.pp is included in the public parameters pp .
 3. For each $e \in [M]$, sample $\Delta K_e^* \xleftarrow{\$} \mathbb{F}_p$ and compute

$$T_e^* = \mathcal{L}_0^t \left(I_{\bar{\ell}_e} \sum_{\ell=0, \ell \neq \bar{\ell}_e}^{L-1} r_{e,\ell} - \sum_{\ell=0, \ell \neq \bar{\ell}_e}^{L-1} r_{e,\ell} I_\ell + \Delta K_e^* \right) - pk_{\bar{\ell}_e}.$$

4. Run $\text{decom}_S \leftarrow \text{BAVC.Open}^{\text{PRG}, \mathcal{H}}(\text{decom}, S, \text{BAVC.pp})$.
5. Sim outputs the simulated transcript $((A, A_o^*), S, Z)$ such that $A = \text{com}$, $A_o^* = (T_e^*, \Delta K_e^*)_{e \in [M]}$, and $Z = \text{decom}_S$.

It is immediate that $((A, A_o^*), S, Z)$ is an accepting transcript, as the BAVC commitment and openings are honestly generated, and for each $e \in [M]$, $o_e = I_{\bar{\ell}_e} \sum_{\ell=0, \ell \neq \bar{\ell}_e}^{L-1} r_{e,\ell} - \sum_{\ell=0, \ell \neq \bar{\ell}_e}^{L-1} r_{e,\ell} I_\ell + \Delta K_e^*$, it holds that $T_e^* = \mathcal{L}_0^t(o_e) - pk_{\bar{\ell}_e}$.

- **Hybrid Argument.** We show that the distribution of simulated transcripts $((A, A_o^*), S, Z)$ is indistinguishable from the real distribution of $((A, A_o), S, Z)$ in a hybrid argument, assuming BAVC satisfies selective hiding.
 - **H0 (Real).** Honest prover (knows the witness K) follows Algorithm 3 and outputs a real transcript $((A, A_o), S, Z)$, where

$$A = \text{com}, A_o = (T_e, \Delta K_e)_{e \in [M]}, Z = \text{decom}_S.$$

Here, the prover runs BAVC algorithms honestly, generating messages $\mathbf{R} = (r_{e,0}, \dots, r_{e,L-1})_{e \in [M]}$, then compute

$$r_e = \sum_{\ell=0}^{L-1} r_{e,\ell}, T_e = \mathcal{L}_0^t(r_e), \Delta K_e = K r_e + \sum_{\ell=0}^{L-1} I_\ell r_{e,\ell},$$

for each $e \in [M]$.

- **H1 (Randomizing unopened leaves).** For each $e \in [M]$, replace the unopened message $r_{e, \bar{\ell}_e}$ with an independent uniform sample $r'_{e, \bar{\ell}_e} \xleftarrow{\$} \mathbb{F}_p$. Because the BAVC is hiding, replacing unopened messages with fresh random values yields a distribution for the new transcript $((A, A'_\circ), S, Z)$ which is indistinguishable from $((A, A_\circ), S, Z)$, with distinguishing advantage at most $\text{AdvHD}_A^{\text{BAVC}}$. Concretely, after this substitution, we have

$$r'_e = \sum_{\ell \neq \bar{\ell}_e} r_{e, \ell} + r'_{e, \bar{\ell}_e}, \quad T'_e = \mathcal{L}_0^t(r'_e),$$

and

$$\begin{aligned} \Delta K'_e &= K r'_e + \sum_{\ell \neq \bar{\ell}_e} I_\ell r_{e, \ell} + I_{\bar{\ell}_e} r'_{e, \bar{\ell}_e} \\ &= (K + I_{\bar{\ell}_e}) r_{e, \bar{\ell}_e} + \sum_{\ell \neq \bar{\ell}_e} (K + I_\ell) r_{e, \ell} \end{aligned}$$

Since $r'_{e, \bar{\ell}_e}$ is uniform and independent, r'_e is uniform over $\mathbb{F}_p$. Hence, T'_e is the Legendre or t -th power residue PRF of a uniform input, and $\Delta K'_e$ is also uniform over $\mathbb{F}_p$, provided that $K + I_{\bar{\ell}_e} \neq 0$. Since K is sampled uniformly from $\mathbb{F}_p^*$, $K + I_{\bar{\ell}_e} = 0$ happens with probability at most $\frac{1}{p}$, and across M independent challenges, the probability is at most M/p , which is negligible for an exponentially large p .

- **H2 (Simulator).** Fix $(r_{e, \ell})_{\ell \in [0, L-1] \setminus \{\bar{\ell}_e\}, e \in [M]}$, $(I_\ell)_{\ell \in [0, L-1]}$ and $pk_{\bar{\ell}_e}$, for each $e \in [M]$, replace ΔK_e by an independent uniform $\Delta K_e^* \xleftarrow{\$} \mathbb{F}_p$ and set

$$T_e^* = \mathcal{L}_0^t \left(I_{\bar{\ell}_e} \sum_{\ell=0, \ell \neq \bar{\ell}_e}^{L-1} r_{e, \ell} - \sum_{\ell=0, \ell \neq \bar{\ell}_e}^{L-1} r_{e, \ell} I_\ell + \Delta K_e^* \right) - pk_{\bar{\ell}_e}.$$

Keep the commitment com and the opening $Z = \text{decom}_S$ unchanged, let $((A, A_\circ^*), S, Z)$ denote the resulting transcript. Since ΔK_e^* is uniform and independent, the masked input inside $\mathcal{L}_0^t(\cdot)$ is uniform over $\mathbb{F}_p$. Hence, $(T_e^*, \Delta K_e^*)$ has exactly the same distribution as $(T'_e, \Delta K'_e)$ from H1 for every $e \in [M]$. Therefore, the overall transcript distribution in H2 is identical to that of H1.

□

B.6 Proof of Theorem 6

Proof. Let $\mathcal{A}$ be a PPT adversary against the EUF-KO security of SIG making q queries to $\mathcal{H}_{\text{FS}}$. We first show the construction of a PPT adversary $\mathcal{B}$ which recovers the secret key sk corresponding to the public key pk by invoking both $\mathcal{A}$ and the extractor $\mathcal{E}$. This ensures that (pk, sk) belongs to the relation $\mathcal{R}_{\beta \mathcal{L}^t}$.

- **Construction.** The algorithm $\mathcal{B}$ proceeds as follows:
 1. **Setup Phase.** Upon receiving the public parameters $p, t, L, \mathcal{I}, \mathcal{I}'$ and a statement $pk = \mathcal{L}_K^t(\mathcal{I})$, $\mathcal{B}$ selects an integer M and runs $\text{BAVC.pp} \leftarrow \text{BAVC.Setup}(1^\lambda)$. Let $\text{pp} := (p, t, L, M, \mathcal{I}, \text{BAVC.pp})$. It then runs adversary $\mathcal{A}$ and extractor $\mathcal{E}$, forwarding pp and pk to them.
 2. **RO Simulation Phase.** $\mathcal{B}$ simulates $\mathcal{H}_{\text{FS}}$ using lazy sampling and maintains a query list L_{HFS} . If $\mathcal{A}$ queries $\mathcal{H}$, $\mathcal{B}$ forwards the query to $\mathcal{E}$ and returns the response to $\mathcal{A}$.
 3. **Challenge Phase.** Once $\mathcal{A}$ outputs a forgery (m, σ) , $\mathcal{B}$ parses $\sigma = (\text{salt}, h, \text{ctr}, \text{Decom}_S, (\Delta K_e)_{e \in [M]}, h, Z)$ and computes $S \leftarrow \text{Expand}(h[128])$. It then checks the query list L_{HFS} for an entry of the form

$$((\text{salt}, \text{com}, (T_e)_{e \in [M]}, (\Delta K_e)_{e \in [M]}, m, \text{ctr}), h) \in L_G.$$
 - If no such entry exists, $\mathcal{B}$ aborts.
 - Otherwise, $\mathcal{B}$ extracts $(T_e)_{e \in [M]}$ and sends $((\text{com}, A_o), S, Z)$ to $\mathcal{E}$, where $A_o = (T_e, \Delta K_e)_{e \in [M]}$.
 4. **Output Phase.** Finally, when $\mathcal{E}$ outputs sk , $\mathcal{B}$ outputs sk .
- **Probability Analysis.** We begin by analyzing the simulation of the random oracles. By the properties of random oracles, the simulations of $\mathcal{H}_{\text{FS}}$ and $\mathcal{H}$ are perfect. Next, we turn to the analysis of $\mathcal{B}$'s success probability. To facilitate this, we introduce the following events **Event**, **Event₁**, **Event₂**, and **Event₃**:

- **Event:** $\mathcal{B}$ succeeds (this is the target event we aim to bound).

$$\Pr[\text{Event}] = \Pr[\mathcal{B} \text{ succeeds}].$$

- **Event₁:** (m, σ) is a valid forgery.

$$\Pr[\text{Event}_1] = \text{Adv}_{\text{SIG}, \mathcal{A}^{\mathcal{H}_{\text{FS}}, \mathcal{H}}}^{\text{EUF-KO}}(1^\lambda).$$

- **Event₂:** $\mathcal{B}$ aborts.

$$\Pr[\text{Event}_2] = \Pr[\mathcal{B} \text{ aborts}].$$

- **Event₃:** $((\text{com}, A_o), S, Z)$ is a valid transcript of the protocol $\Pi^{\mathcal{H}}$.

$$\Pr[\text{Event}_3] = \Pr[\Pi.\text{Verify}^{\mathcal{H}}(pk, ((\text{com}, A_o), S, Z), \text{pp}) = 1].$$

We further define $\delta_1 := \Pr[\text{Event}_2 \wedge \text{Event}_1]$ and $\delta_2 := \Pr[\neg \text{Event}_2 \wedge \text{Event}_1 \wedge \neg \text{Event}_3]$. By standard probabilistic reasoning, we obtain

$$\Pr[\text{Event}] = 1 - \Pr[\neg \text{Event}] \geq \Pr[\text{Event}_1] - \epsilon_{\text{ext}} - \delta_1 - \delta_2. \quad (6)$$

We now analyze the upper bounds of δ_1 and δ_2 :

- $\delta_1 = \Pr[\text{Event}_2 \wedge \text{Event}_1]$. This case corresponds to $\mathcal{A}$ producing a valid forgery (m, σ) without querying $\mathcal{H}_{\text{FS}}$ on

$$x = (\text{salt}, \text{com}, (T_e)_{e \in [M]}, (\Delta K_e)_{e \in [M]}, m, \text{ctr}).$$

By the randomness of $\mathcal{H}_{\text{FS}}$, the probability of this event is bounded by the guessing probability of $\mathcal{A}$:

$$\delta_1 \leq \Pr[\text{Event}_1 | \forall (T_e)_{e \in [M]}, (x, h) \notin L_{\text{HFS}}] \leq \frac{1}{2^{2\lambda}}.$$

- $\delta_2 = \Pr[\neg \text{Event}_2 \wedge \text{Event}_1 \wedge \neg \text{Event}_3] \leq \Pr[\neg \text{Event}_3 | \neg \text{Event}_2 \wedge \text{Event}_1]$.
This case corresponds to $\mathcal{B}$ not aborting and (m, σ) being a valid forgery, yet $((\text{com}, A_o), S, Z)$ failing to be a valid proof for $\Pi^{\mathcal{H}}$. Again, by the randomness of $\mathcal{H}_{\text{FS}}$ and the completeness of $\Pi^{\mathcal{H}}$, we obtain:

$$\delta_2 \leq \Pr[\neg \text{Event}_3 | \neg \text{Event}_2 \wedge \text{Event}_1] \leq \frac{q^2}{2^{2\lambda}}.$$

Finally, we conclude that

$$\text{Adv}_{\mathcal{A}^{\mathcal{H}_{\text{FS}}, \mathcal{H}}}^{\text{EUF-KO}}(1^\lambda) \leq \text{Adv}_{\mathcal{B}}^{\mathcal{R}_{\beta \mathcal{L}^t}}(1^\lambda) + \epsilon_{\text{ext}} + \frac{q^2 + 1}{2^{2\lambda}}.$$

This demonstrates that the EUF-KO advantage of $\mathcal{A}$ is tightly bounded by the advantage of $\mathcal{B}$ in breaking the hardness of the relation $\mathcal{R}_{\beta \mathcal{L}^t}$, up to the extractor error ϵ_{ext} and negligible terms. Therefore, under the assumed hardness of $\mathcal{R}_{\beta \mathcal{L}^t}$, the signature scheme SIG (i.e. PEGASUS $^{\mathcal{H}_{\text{FS}}, \mathcal{H}}$) is EUF-KO secure in the ROM if the commit-and-open protocol $\Pi^{\mathcal{H}}$ is a PoK-OE in the ROM. $\square$

B.7 Proof of Theorem 7

Proof. Let $\mathcal{A}$ be a quantum PPT adversary against the EUF-KO security of SIG (i.e., PEGASUS) making q quantum queries to $\mathcal{H}_{\text{FS}}$ and let $\mathcal{E}$ be a quantum PPT extractor, as defined in the notion of PoK-OE.

- **Construction.** We construct a quantum PPT algorithm $\mathcal{B}$. The construction of $\mathcal{B}$ is similar to that in the classical ROM proof, except for **RO Simulation Phase** and **Challenge Phase**.
 1. **RO Simulation Phase.** $\mathcal{B}$ simulates $\mathcal{H}_{\text{FS}}$ via CStO [68]. If $\mathcal{A}$ queries $\mathcal{H}$, then $\mathcal{B}$ forwards the query to $\mathcal{E}$ and returns the response to $\mathcal{A}$.
 2. **Challenge Phase.** Once $\mathcal{A}$ outputs a forgery (m, σ) , $\mathcal{B}$ parses $\sigma = (\text{salt}, h, \text{ctr}, \text{Decom}_S, (\Delta K_e)_{e \in [M]}, h, Z)$ and computes $S \leftarrow \text{Expand}(h[128])$. It then measure the internal state of the compressed oracle to obtain database $D_{\mathcal{H}_{\text{FS}}}$. $\mathcal{B}$ extracts $(T_e)_{e \in [M]}$ s.t.

$$(\text{salt}, \text{com}, (T_e)_{e \in [M]}, (\Delta K_e)_{e \in [M]}, m, \text{ctr}) \in D^{-1}(h).$$

- If no such entry exists, $\mathcal{B}$ aborts.
- Otherwise, $\mathcal{B}$ extracts $(T_e)_{e \in [M]}$ and sends $((\text{com}, A_o), S, Z)$ to $\mathcal{E}$, where $A_o = (T_e, \Delta K_e)_{e \in [M]}$.
- **Probability Analysis.** According to Lemma 3, the simulation of QROs by $\mathcal{B}$ is perfect. We now turn to analyze $\mathcal{B}$'s success probability. To this end, we define the database properties SUC and CL as follows:

$$\text{SUC} := \left\{ \begin{array}{l} \exists \text{ salt, ctr, } m, \text{com, } \{T_e\}_{e \in [M]} \text{ and } (\Delta K_e)_{e \in [M]} \text{ s.t} \\ D : ((\text{salt, ctr, } m, \text{com, } \{T_e\}_{e \in [M]}, (\Delta K_e)_{e \in [M]}), h) \in D \text{ satisfies} \\ \text{Verify}(pk, m, \sigma) = 1 \wedge \Pi.\text{Verify}^D(pk, ((\text{com}, A_o), S, Z), \text{pp}) \neq 1 \end{array} \right\},$$

and

$$\text{CL} := \{D : \exists x \neq x' \text{ s.t. } D(x) = D(x') \neq \perp\}.$$

The condition $D_{H_{\text{FS}}} \in \text{SUC}$ means that $\mathcal{B}$ does not abort and $\mathcal{A}$ produces a valid forgery while $\text{II.Verify}^D(pk, ((\text{com}, A_o), S, Z), \text{pp}) \neq 1$. Similarly, $D_{H_{\text{FS}}} \in \text{CL}$ indicates that there exists a collision in $D_{H_{\text{FS}}}$. We have

$$\begin{aligned} \text{Adv}_{\text{SIG}, \mathcal{A}^{\mathcal{H}_{\text{FS}}, \mathcal{H}}}^{\text{EUF-KO}}(1^\lambda) &\leq \text{Adv}_{\mathcal{B}}^{\mathcal{R}_{\beta \mathcal{L}^t}}(1^\lambda) + \epsilon_{\text{ext}} + \max_{\mathcal{A}} \Pr[D_{H_{\text{FS}}} \in \text{SUC} \cup \text{CL}] + \\ &\quad \Pr[\mathcal{B} \text{ aborts} | \mathcal{A} \text{ succeeds}]. \end{aligned}$$

According to Lemma 4, we can get

$$\Pr[\mathcal{B} \text{ aborts} | \mathcal{A} \text{ succeeds}] \leq \frac{1}{2^{2\lambda}}.$$

We denote

$$\llbracket \perp \xrightarrow{q} \text{SUC} \cup \text{CL} \rrbracket := \sqrt{\max_{\mathcal{A}} \Pr[D_{H_{\text{FS}}} \in \text{SUC} \cup \text{CL}]},$$

and introduce the database property $\text{SZ}_{\leq i} := \{D : \#\{z : D(z) \neq \perp\} \leq i\}$, where q is the number of $\mathcal{A}$'s quantum queries to $\mathcal{H}_{\text{FS}}$. We then analyze upper bounds for $\llbracket \perp \xrightarrow{q} \text{SUC} \cup \text{CL} \rrbracket$. By basic properties of quantum transition capacity and Lemma 5 (Lemma 5.6 in [32]), we obtain

$$\llbracket \perp \xrightarrow{q} \text{SUC} \cup \text{CL} \rrbracket \leq \sum_{i=0}^{q-1} \llbracket \text{SZ}_{\leq i} \setminus \text{SUC} \rightarrow \text{SUC} \rrbracket + \sum_{i=0}^{q-1} \llbracket \text{SZ}_{\leq i} \setminus \text{CL} \rightarrow \text{CL} \rrbracket.$$

- $\llbracket \text{SZ}_{\leq i} \setminus \text{CL} \rightarrow \text{CL} \rrbracket$. By Lemma 1 (Equation (8) in [32]), the collision probability of the database can be bounded as $\llbracket \text{SZ}_{\leq i} \setminus \text{CL} \rightarrow \text{CL} \rrbracket \leq 2e\sqrt{\frac{(i+1)}{2^{2\lambda}}} \leq 2e\sqrt{\frac{q}{2^{2\lambda}}}$.
- $\llbracket \text{SZ}_{\leq i} \setminus \text{SUC} \rightarrow \text{SUC} \rrbracket$. Consider an arbitrary but fixed $D \in \mathcal{D}$ and input $x \in X$, assuming $D(x) = \perp$ (i.e., x is fresh). Define $\text{P} := \text{SZ}_{\leq s} \setminus \text{SUC}$ and $\text{P}' := \text{SUC}$.
 - * Case 1: $D \in \text{SUC}$ and $\perp \in \text{SUC}|_{D|x} = \text{P}'_{D|x}$. Set $\text{L} := \text{P}_{D|x}$. Furthermore,

$$u \in \text{L} \leftrightarrow D[x \rightarrow u] \in \text{P} \rightarrow D[x \rightarrow u] \notin \text{SUC} \rightarrow u = h.$$

Thus,

$$\Pr[y \in \text{L} | y \xleftarrow{\$} Y] \leq \Pr[y = h | y \xleftarrow{\$} Y] = \frac{1}{2^{2\lambda}}.$$

- * Case 2: $D \notin \text{SUC}$. Since $D(x) = \perp$ and $\perp \notin \text{P}'_{D|x}$, we set $\text{L} := \text{P}'_{D|x}$. Then,

$$u \in \text{L} \leftrightarrow D[x \rightarrow u] \in \text{P}' \rightarrow u = h.$$

Hence,

$$\Pr[y \in \text{L} | y \xleftarrow{\$} Y] \leq \Pr[y = h | y \xleftarrow{\$} Y] = \frac{1}{2^{2\lambda}}.$$

By Theorem 1, we obtain the bound

$$\llbracket \text{SZ}_{\leq i} \setminus \text{SUC} \rightarrow \text{SUC} \rrbracket \leq \sqrt{\frac{10}{2^{2\lambda}}}.$$

In conclusion, we derive the following bound:

$$\text{Adv}_{\mathcal{A}^{\mathcal{H}_{\text{FS}}, \mathcal{H}}}^{\text{EUF-KO}}(1^\lambda) \leq \text{Adv}_{\mathcal{B}}^{\mathcal{R}_{\beta \mathcal{L}^t}}(1^\lambda) + \epsilon_{\text{ext}} + \frac{(60q + 10)q^2 + 1}{2^{2\lambda}}.$$

This inequality shows that, under the assumed hardness of $\mathcal{R}_{\beta \mathcal{L}^t}$, the signature scheme $\text{PEGASUS}^{\mathcal{H}_{\text{FS}}, \mathcal{H}}$ is EUF-KO secure in the QROM if the commit-and-open protocol $\Pi^{\mathcal{H}}$ is a PoK-OE in the QROM. $\square$

B.8 Proof of Theorem 8

Proof. Let $\mathcal{A}$ be a PPT adversary against the EUF-CMA security of the signature scheme. We construct a reduction $\mathcal{B}$ against the EUF-KO security by leveraging the SHVZK property of $\Pi^{\mathcal{H}}$.

– **Construction.** Let Sim be the PPT simulator as defined in the SHVZK property. We construct a PPT adversary $\mathcal{B}$ as follows:

1. **Setup Phase.** On input of the public parameters pp and a public key pk , $\mathcal{B}$ runs $\mathcal{A}$ on the same inputs pp and pk .
2. **RO Simulation Phase.** $\mathcal{B}$ simulates the random oracle $\mathcal{H}_{\text{FS}}$ via lazy sampling and maintains the query list $L_{\mathcal{H}_{\text{FS}}}$.
3. **Signing Oracle Simulation Phase.** Upon receiving a signing query on a message m , $\mathcal{B}$ invokes Sim . Concretely, it samples $\text{salt} \xleftarrow{\$} \{0, 1\}^{2\lambda}$ and $h^* \xleftarrow{\$} \{0, 1\}^{2\lambda}$ uniformly at random. Then, it derives a challenge set $S \leftarrow \text{expand}(h[128])$, and invokes the simulator Sim to produce $((\text{com}, A_o), Z)$, where $A_o = (T_e, \Delta K_e)_{e \in [M]}$ and $Z = \text{decom}_S$. Next, it sets

$$\sigma := (\text{salt}, \text{com}, h, \text{ctr}, \text{decom}_S, (\Delta K_e)_{e \in [M]})$$

with $\sigma_1 = (\text{salt}, \text{com}, A_o)$. If $x^* = (\text{salt}, \sigma_1, m, \text{ctr})$ has not been queried before, $\mathcal{B}$ programs the oracle so that $(x^*, h^*) \in L_G$ and then returns σ ; otherwise, it aborts.

4. **Forgery.** When $\mathcal{A}$ outputs a forgery (m, σ) in the EUF-CMA game, $\mathcal{B}$ forwards (m, σ) to the EUF-KO game.

– **Probability Analysis.** The simulation of $\mathcal{H}_{\text{FS}}$ is perfect. For the signing oracle simulation, $\mathcal{B}$ aborts if x has already been queried by $\mathcal{A}$. Since salt is chosen uniformly at random, the probability that $\mathcal{B}$ aborts is at most $\frac{q_s(q_s + q_H)}{2^{2\lambda}}$, where q_H and q_s denote the number of queries made by $\mathcal{A}$ to $\mathcal{H}_{\text{FS}}$ and the signing oracle, respectively. When $\mathcal{B}$ does not abort, by the definition of SHVZK, the simulation of the signing oracle can be distinguished from the real one by $\mathcal{A}$ with probability at most $\text{Adv}_{\Pi^{\mathcal{H}}, \mathcal{A}}^{\text{SHVZK}}(1^\lambda)$. Hence, we obtain

$$\text{Adv}_{\mathcal{A}}^{\text{EUF-CMA}}(1^\lambda) \leq \text{Adv}_{\mathcal{B}}^{\text{EUF-KO}}(1^\lambda) + \text{Adv}_{\Pi^{\mathcal{H}}, \mathcal{A}}^{\text{SHVZK}}(1^\lambda) + \frac{q_s(q_s + q_H)}{2^{2\lambda}}.$$

This inequality shows that if the underlying protocol $\Pi^{\mathcal{H}}$ satisfies SHVZK and $\text{PEGASUS}^{\mathcal{H}_{\text{FS}}, \mathcal{H}}$ is EUF-KO secure, then the signature scheme $\text{PEGASUS}^{\mathcal{H}_{\text{FS}}, \mathcal{H}}$ is also EUF-CMA secure in the ROM. By combining this with Theorem 6, we obtain the claimed result. $\square$

B.9 Proof of Theorem 9

Proof. Define $\mathcal{H}'_{\text{FS}}$ as follows: if $x \neq x^*$, then $\mathcal{H}'_{\text{FS}}(x) = \mathcal{H}_{\text{FS}}(x)$; if $x = x^*$, then $\mathcal{H}'_{\text{FS}}(x) = h^*$. For any quantum PPT adversary $\mathcal{D}$ and quantum PPT extractor $\mathcal{E}$ defined as in Lemma 6, denote

$$P_{\text{left}} := \Pr[b \leftarrow \mathcal{D}^{\mathcal{H}'_{\text{FS}}}(z) : b = 1], \quad P_{\text{right}} := \Pr[b \leftarrow \mathcal{D}^{\mathcal{H}_{\text{FS}}}(z) : b = 1],$$

and

$$P_{\text{guess}} := \Pr[x = x^* \neq \emptyset : x \leftarrow \mathcal{E}^{\mathcal{H}_{\text{FS}}}(z)].$$

According to Lemma 6, we have

$$|P_{\text{left}} - P_{\text{right}}| \leq 2q\sqrt{P_{\text{guess}}},$$

where q denotes the number of the distinguisher's queries to the QRO. Since the **salt** in x^* is chosen uniformly at random, it follows that $P_{\text{guess}} \leq \frac{1}{2^{2\lambda}}$. Putting this together, we obtain

$$\begin{aligned} \text{Adv}_{\mathcal{A}}^{\text{EUF-CMA}}(1^\lambda) &\leq \text{Adv}_{\mathcal{B}}^{\text{EUF-KO}}(1^\lambda) + \text{Adv}_{\Pi^{\mathcal{H}}, \mathcal{A}}^{\text{SHVZK}}(1^\lambda) + 2q_s \cdot (q_s + q_H) \cdot \sqrt{P_{\text{guess}}} \\ &\leq \text{Adv}_{\mathcal{B}}^{\text{EUF-KO}}(1^\lambda) + \text{Adv}_{\Pi^{\mathcal{H}}, \mathcal{A}}^{\text{SHVZK}}(1^\lambda) + \frac{2q_s \cdot (q_s + q_H)}{2^\lambda}, \end{aligned}$$

where q_H and q_S denote the number of queries made by $\mathcal{A}$ to $\mathcal{H}_{\text{FS}}$ and the signing oracle.

This inequality shows that if the underlying protocol $\Pi^{\mathcal{H}}$ satisfies SHVZK and $\text{PEGASUS}^{\mathcal{H}_{\text{FS}}, \mathcal{H}}$ is EUF-KO secure in the QROM, then the signature scheme $\text{PEGASUS}^{\mathcal{H}_{\text{FS}}, \mathcal{H}}$ is also EUF-CMA secure in the QROM. Together with Theorem 7, this completes the proof. $\square$

B.10 Proof of Theorem 10

Proof. Let $\mathcal{A}$ be a PPT adversary against the insider-corruption unforgeability of PEGARING, making q_h^{FS} and q_h^{MT} queries to $\mathcal{G}_{\text{FS}}$ and $\mathcal{G}_{\text{MT}}$, respectively.

– **Construction.** The construction of $\mathcal{B}$ proceeds as follows:

1. **Steup Phase.** Upon receiving the public parameters and a statement $pk^* = \mathcal{L}_K^t(\mathcal{I})$, $\mathcal{B}$ selects correlated parameters and constructs **R-pp**. It then chooses an index $i^* \in [N]$ uniformly at random. For every $j \in [N]$ with $j \neq i^*$, $\mathcal{B}$ generates a key pair $(pk_j, sk_j) \leftarrow \text{R-KeyGen}(\text{R-pp})$, and sets $pk_{i^*} = pk^*$. Let $\mathbb{PK} = \{pk_1, \dots, pk_N\}$ denote the resulting public key list. Finally, $\mathcal{B}$ forwards $(\text{R-pp}, \mathbb{PK})$ to $\mathcal{A}$ and $(\text{R-pp}, pk^*)$ to the extractor $\mathcal{E}$ of PoK-OE.

2. **RO Simulation Phase.** $\mathcal{B}$ simulates the random oracles $\mathcal{G}_{\text{FS}}$ and $\mathcal{G}_{\text{MT}}$ for $\mathcal{A}$ using lazy sampling, while maintaining the corresponding query lists L_G^{FS} and L_G^{MT} . If $\mathcal{A}$ makes a query to $\mathcal{H}$, then $\mathcal{B}$ forwards this query to $\mathcal{E}$ and returns the response back to $\mathcal{A}$.
3. **Corruption Oracle Simulation Phase.** For each query i , if $pk_i \notin \mathbb{PK}$ or $pk_i = pk^*$, then $\mathcal{B}$ aborts. Otherwise, it returns the corresponding sk_i .
4. **Signing Oracle Simulation Phase.** For any query $(\text{msg}, \mathcal{PK}, pk_\pi)$, $\mathcal{B}$ aborts if $pk_\pi \notin \mathbb{PK}$. Furthermore, if $pk_\pi \in \mathcal{PK} \setminus \{pk^*\}$ then $\mathcal{B}$ returns $\sigma \leftarrow \text{R-Sign}(\text{msg}, sk_\pi, \mathcal{PK}, \text{R-pp})$. Otherwise, $\mathcal{B}$ computes the following:
 - (a) $\mathcal{B}$ randomly selects $h_1, \dots, h_n \xleftarrow{\$} \{0, 1\}^\lambda$ and expands each seed to $(\bar{\ell}_e^{(i)})_{e \in [M]} \leftarrow \text{expand}(h_i)$, where $\bar{\ell}_e^{(i)} \in [0, L-1]$.
 - (b) $\mathcal{B}$ computes $(\bar{z}_e, \bar{c}_e) = \left(\left\lfloor \frac{\bar{\ell}_e^{(i)}}{c} \right\rfloor, \bar{\ell}_e^{(i)} \pmod{c} \right)$. Then it computes $\bar{z}_e = \sum_{i=1}^n \bar{z}_e^{(i)} \pmod{z}$, $\bar{c}_e = \sum_{i=1}^n \bar{c}_e^{(i)} \pmod{c}$, and $\bar{\ell}_e = \bar{z}_e c + \bar{c}_e$.
 - (c) $\mathcal{B}$ uniformly randomly selects $\Delta K_e \xleftarrow{\$} \mathbb{F}_p$ and computes

$$o_e = I_{\bar{\ell}_e} \sum_{\ell=0, \ell \neq \bar{\ell}_e}^{L-1} r_{e,\ell} + \sum_{\ell=0, \ell \neq \bar{\ell}_e}^{L-1} r_{e,\ell}, I_\ell + \Delta K_e.$$

- (d) $\mathcal{B}$ randomly choose $b_e \xleftarrow{\$} \{0, 1\}$ and set

$$\Delta K_{e, \bar{z}_e}^{b_e} = \Delta K_e, \Delta K_{e, \bar{z}_e}^{1-b_e} \xleftarrow{\$} \mathbb{F}_p.$$

For other $j \in [0, z-1] \setminus \{\bar{z}_e\}$, $(\Delta K_{e,j}^0, \Delta K_{e,j}^1) \xleftarrow{\$} \mathbb{F}_p^2$.

- (e) $\mathcal{B}$ computes $T_e = \mathcal{L}_0^t(o_e) - \sum_{i=1}^n pk_{\bar{\ell}_e^{(i)}}^{(i)}$.
- (f) $\mathcal{B}$ computes all other first round components following the protocol.
- (g) $\mathcal{B}$ checks whether there exists an h such that

$$((\text{com}, \text{com}_{\text{MT}}, (T_e)^* e \in [M]), h) \in L_G^{\text{FS}}.$$

If such an h exists, $\mathcal{B}$ aborts. Otherwise, it computes $h[128] = \bigoplus_{i=1}^n h_i$, chooses the remaining 128 bits uniformly at random, and sets $L_G^{\text{FS}} \leftarrow L_G^{\text{FS}} \cup ((\text{com}, \text{com}_{\text{MT}}, (T_e)_{e \in [M]}), h)$. This programs the random oracle $\mathcal{G}_{\text{FS}}$ so that, on input $(\text{com}, \text{com}_{\text{MT}}, (T_e)_{e \in [M]})$, it outputs h .

- (h) $\mathcal{B}$ follows the remaining steps to generate σ honestly.

5. **Challenge Phase.** Once $\mathcal{A}$ outputs a forgery $(\text{msg}^*, \sigma^*, \mathcal{PK}^*)$, $\mathcal{B}$ parses

$$\sigma = (\text{salt}, \text{ctr}, \text{decom}_S, \text{path}_I, (\Delta K_e, v_e, d_e)_{e \in [M]}, (h_i)_{i \in [n]}).$$

- (a) For all $i \in [n]$, $\mathcal{B}$ computes the following
 - $(\bar{\ell}_e^{(i)}) \leftarrow \text{expand}(h_i)$,
 - $(\bar{z}_e^{(i)}, \bar{c}_e^{(i)}) = \left(\left\lfloor \frac{\bar{\ell}_e^{(i)}}{c} \right\rfloor, \bar{\ell}_e^{(i)} \pmod{c} \right)$, note that $I_{\bar{\ell}_e^{(i)}} = a_{\bar{z}_e} + b_{\bar{c}_e}$.

- (b) $\mathcal{B}$ computes the mutual challenge $\bar{z}_e = \sum_{i=1}^n \bar{z}_e^{(i)} \pmod{z}$, $\bar{c}_e = \sum_{i=1}^n \bar{c}_e^{(i)} \pmod{c}$.
- (c) $\mathcal{B}$ checks if $pk^* \in \mathcal{PK}$,
 - If $pk^* \notin \mathcal{PK}$, $\mathcal{B}$ aborts.
 - Otherwise, $\mathcal{B}$ finds the index of pk^* in $\mathcal{PK}$ and set the index be π .
- (d) $\mathcal{B}$ checks the query list L_G^{MT} for any entry of the form

$$((\text{salt}, e, \bar{z}_e, b, \Delta K'_e), v_e),$$

where $b \in \{0, 1\}$.

- If no such entry exists, $\mathcal{B}$ aborts.
 - Otherwise, $\mathcal{B}$ extracts $\Delta K'_e$ and computes $r_e^* = \frac{\Delta K_e - \Delta K'_e}{bc}$.
- (e) $\mathcal{B}$ checks the query list L_G^{FS} for any entry of the form

$$((\text{salt}, \sigma_1, \text{msg}, \text{ctr}, \mathcal{PK}), h) \in L_G^{\text{FS}},$$

where $\sigma_1 = (\text{com}, \text{com}_{\text{MT}}, (T_e)_{e \in [M]})$ and $h[:128] = \bigoplus_{i=1}^n h_i$.

- If no such entry exists, $\mathcal{B}$ aborts.
 - Otherwise, $\mathcal{B}$ extracts $(T_e)_{e \in [M]}$ and computes $T_e^* = T_e + \sum_{i=0, i \neq \pi}^n pk_{\bar{\ell}_e^{(i)}}^{(i)}$.
- (f) $\mathcal{B}$ computes $\Delta K_e^* = \Delta K_e - I_{\bar{\ell}_e^{(\pi)}} r_e + I_{\bar{\ell}_e} r_e$ and sends $((A, A_o), S, Z)$ to $\mathcal{E}$, where $A = \text{com}$, $A_o = (T_e^*, \Delta K_e^*)_{e \in [M]}$, $S = (\bar{\ell}_e)_{e \in [M]}$ and $Z = \text{decom}_S$.
- **Output Phase.** Finally, when $\mathcal{E}$ outputs sk^* , $\mathcal{B}$ outputs sk^* .
- **Probability Analysis.** By the properties of random oracles, the simulation of $\mathcal{G}_{\text{FS}}$ and $\mathcal{G}_{\text{MT}}$ is perfect. To facilitate the analysis of $\mathcal{B}$'s success probability, we define the following probabilistic events:
- **Event:** $\mathcal{B}$ succeeds.

$$\Pr[\text{Event}] = \Pr[\mathcal{B} \text{ succeeds}] = \text{Adv}_{\mathcal{B}}^{\mathcal{R}, \beta \mathcal{C}^t}(1^\lambda).$$

- **Event₁:** $\mathcal{B}$ wins the unforgeability game.

$$\Pr[\text{Event}_1] = \text{Adv}_{\mathcal{A}}^{\text{EUF}}(1^\lambda).$$

- **Event₂:** $\mathcal{B}$ aborts during the **Corruption Oracle Simulation Phase**.
- **Event₃:** $\mathcal{B}$ aborts in step (g) of the **Signing Oracle Simulation Phase** 4g.
- **Event₄:** $\mathcal{B}$ aborts in step (c) of the **Challenge Phase** 5c.
- **Event₅:** $\mathcal{A}$ aborts in steps (d) or (e) of the **Challenge Phase** 5d5e.
- **Event₆:** $\mathcal{B}$ does not abort, $\mathcal{A}$ outputs a valid ring-signature forgery, but the extracted Σ -protocol transcript is invalid.

Since the simulation of the corruption oracle is perfect except when $\mathcal{B}$ aborts, and the simulation of the signing oracle is perfect except when $\mathcal{B}$ aborts, $\mathcal{B}$'s simulation for $\mathcal{A}$ is therefore indistinguishable whenever $\neg \text{Event}_2 \wedge \neg \text{Event}_3$. Denote $p_1 = \Pr[\neg \text{Event}_2 \wedge \neg \text{Event}_3]$. It is clear that

$$p_1 = \Pr[\neg \text{Event}_2] \cdot \Pr[\neg \text{Event}_3].$$

We analyze the probabilities of $\Pr[\neg \text{Event}_2]$ and $\Pr[\neg \text{Event}_3]$ as follows:

- $\Pr[\neg \text{Event}_2]$. Since i^* is chosen randomly, when $\mathcal{A}$ makes q_c queries to O_{cor} , we have

$$\Pr[\neg \text{Event}_2] \geq (1 - \frac{1}{N})(1 - \frac{1}{N-1}) \cdots (1 - \frac{N-q_c}{N-q_c} + 1) = \frac{N-q_c}{N}.$$

- $\Pr[\neg \text{Event}_3]$. Due to the randomness of salt , when $\mathcal{A}$ makes q_s queries to O_{sig} , we have

$$\Pr[\neg \text{Event}_2] \geq (1 - \frac{q_s \cdot q_{H^{\text{FS}}}}{2^{2\lambda}}).$$

Then,

$$p_1 \geq (1 - \frac{q_s \cdot (q_s + q_{H^{\text{FS}}})}{2^{2\lambda}}) \cdot \frac{N-q_c}{N}.$$

When $\mathcal{B}$'s simulations are perfect, we analyze $p_2 := \Pr[\text{Event}_4 | \text{Event}_1]$. Note that Event_4 implies $pk^* \notin \mathcal{PK}$, while Event_1 implies $\mathcal{PK} \notin Q$. Therefore,

$$\begin{aligned} p_2 &= (1 - \frac{1}{N-q_c}) \cdot (1 - \frac{1}{N-q_c-1}) \cdots (1 - \frac{1}{N-q_c-n+1}) \\ &= \frac{N-q_c-n}{N-q_c}. \end{aligned}$$

According to results in probability theory, we can get

$$\Pr[\text{Event}] \geq p_1 \cdot (1 - p_2) \Pr[\text{Event}_1] - \Pr[\text{Event}_5] - \Pr[\text{Event}_6] - \epsilon_{sim}.$$

Next, we analyze $\Pr[\text{Event}_5]$ and $\Pr[\text{Event}_6]$:

- $\Pr[\text{Event}_5]$. The node v_e is not found in L_G^{MT} , which happens with probability at most $\frac{1}{2^{2\lambda}}$, and across all M values, the probability is $\frac{M}{2^{2\lambda}}$. The challenge h is not found in L_G^{FS} , which happens with probability at most $\frac{1}{2^{2\lambda}}$. Hence, $\Pr[\text{Event}_5] \leq \frac{1+M}{2^{2\lambda}}$.
- $\Pr[\text{Event}_6]$. It denotes the probability that $\mathcal{B}$ does not abort, $\mathcal{A}$ outputs a valid ring signature forgery, but the extracted Σ -protocol transcript is not a valid proof. By the randomness of $\mathcal{G}_{\text{MT}}$ and $\mathcal{G}_{\text{FS}}$, this happens with probability at most $\frac{(q_{H^{\text{FS}}})^2 + (q_{H^{\text{MT}}})^2}{2^{2\lambda}}$.

Finally, we conclude that

$$\begin{aligned} &\frac{n}{N} (1 - \frac{q_s \cdot (q_s + q_{H^{\text{FS}}})}{2^{2\lambda}}) \cdot \text{Adv}_{\mathcal{A}}^{\text{EUF}}(1^\lambda) \\ &\leq \text{Adv}_{\mathcal{B}}^{\mathcal{R}_{\beta \mathcal{L}^t}}(1^\lambda) + \frac{(q_{H^{\text{FS}}})^2 + (q_{H^{\text{MT}}})^2 + M + 1}{2^{2\lambda}} + \epsilon_{ext}. \end{aligned}$$

□

B.11 Proof of Theorem 11

Proof. The construction of $\mathcal{B}$ is described as follows.

- **RO Simulation Phase.** $\mathcal{B}$ simulates $\mathcal{G}_{\text{FS}}$ and $\mathcal{G}_{\text{MT}}$ using the CStO technique [68].
- **Signing Oracle Simulation Phase.** In the step (g), $\mathcal{B}$ reprograms the $\mathcal{G}_{\text{FS}}$ such that $\mathcal{G}_{\text{FS}}((\text{salt}, \sigma_1, \text{msg}, \text{ctr}, \mathcal{PK})) = h$ rather than aborts.
- **Challenge Phase.** Once $\mathcal{A}$ outputs a forgery, $\mathcal{B}$ measures its internal state to get database D_{FS} and D_{MT} . To simplicity, we denote the database D such that $D(1, \cdot) = D_{\text{FS}}(\cdot)$ and $D(2, \cdot) = D_{\text{MT}}(\cdot)$. Then in step (d) and (e), $\mathcal{B}$ checks the database rather than the query lists.

Let $\mathbf{x} = \{(\text{salt}, e, \bar{z}_e, b, \Delta K'_e)\}_{e \in [M]}$ and $\mathbf{y} = \{v_e\}_{e \in [M]}$. Then, we deonte the database SUC and CL as follows:

$$\text{SUC} := \left\{ \begin{array}{l} \exists \mathbf{x} \text{ s.t. } (1 \| \mathbf{x}, \mathbf{y}) \in D \text{ satisfies} \\ D : \|\mathcal{A} \text{ succeds}\| = 1 \text{ for } D^{-1}(h) \\ \wedge \Pi.\text{Verify}^{\mathcal{H}}(pk^*, ((\text{com}, A_o), S, Z), \text{pp}) \neq 1 \end{array} \right\},$$

and

$$\text{CL} := \{D : \exists x \neq x' \text{ s.t. } D(x) = D(x') \neq \perp\}.$$

Similar to the analysis in the Theorem 10 and Theorem 7, we can get

$$\Pr[\text{Event}] \geq p \cdot \Pr[\text{Event}_1] - \max_{\mathcal{A}} \Pr[D_G \in \text{SUC} \cup \text{CL}] - \Pr[\mathcal{B} \text{ aborts} | \mathcal{A} \text{ succeds}] - \epsilon_{\text{sim}}.^{11}$$

- According to Lemma 6, $p \geq (1 - \frac{q_s \cdot (q_s + q_{H^{\text{FS}}})}{2^\lambda}) \cdot \frac{n}{N}$.
- According to Lemma 1, we have

$$\Pr[\mathcal{B} \text{ aborts} | \mathcal{A} \text{ succeds}] \leq \frac{M+1}{2^{2\lambda}}.$$

- By basic properties of quantum transition capacity in Lemma 5 and Theorem 1, we obtain

$$\sqrt{\max_{\mathcal{A}} \Pr[D_G \in \text{SUC} \cup \text{CL}]} \leq \sum_{i=0}^{q-1} \llbracket \text{SZ}_{\leq i} \setminus \text{SUC} \rightarrow \text{SUC} \rrbracket + \sum_{i=0}^{q-1} \llbracket \text{SZ}_{\leq i} \setminus \text{CL} \rightarrow \text{CL} \rrbracket.$$

Here, $q = q_{H^{\text{FS}}} + q_{H^{\text{MT}}}$.

- $\llbracket \text{SZ}_{\leq i} \setminus \text{CL} \rightarrow \text{CL} \rrbracket$. By Lemma 1 (Equation (8) in [32]), the collision probability of the QRO can be bounded as $\llbracket \text{SZ}_{\leq i} \setminus \text{CL} \rightarrow \text{CL} \rrbracket \leq 2e \sqrt{\frac{(i+1)}{2^{2\lambda}}} \leq 2e \sqrt{\frac{q}{2^{2\lambda}}}$.
- $\llbracket \text{SZ}_{\leq i} \setminus \text{SUC} \rightarrow \text{SUC} \rrbracket$. Consider an arbitrary but fixed $D \in \mathcal{D}$ and input $x \in X$, assuming $D(x) = \perp$ (i.e., x is fresh). Define $\text{P} := \text{SZ}_{\leq s} \setminus \text{SUC}$ and $\text{P}' := \text{SUC}$.
 - * Case 1: $D \in \text{SUC}$ and $\perp \in \text{SUC}_{|D|^x} = \text{P}'_{|D|^x}$. Set $\text{L} := \text{P}_{|D|^x}$. Furthermore,

$$u \in \text{L} \leftrightarrow D[x \rightarrow u] \in \text{P} \rightarrow D[x \rightarrow u] \notin \text{SUC} \rightarrow u = h.$$

Thus,

$$\Pr[y \in \text{L} | y \xleftarrow{\$} Y] \leq \Pr[y = h | y \xleftarrow{\$} Y] \leq \frac{1}{2^{2\lambda}}.$$

¹¹ Here, $\mathcal{B} \text{ aborts}$ means that $\mathcal{B}$ aborts in steps (d) or (e) of the Challenge Phase.

- * Case 2: $D \notin \text{SUC}$ and the prefix of x is 1. Since $D(x) = \perp$ and $\perp \notin P'|_{D|x}$, we set $L := P'_{D|x}$. Then,

$$u \in L \leftrightarrow D[x \rightarrow u] \in P' \rightarrow u = h.$$

Hence,

$$\Pr[y \in L | y \xleftarrow{\$} Y] \leq \Pr[y = h | y \xleftarrow{\$} Y] \leq \frac{1}{2^{2\lambda}}.$$

- * Case 3: $D \notin \text{SUC}$ and the prefix of x is 2. Since $D(x) = \perp$ and $\perp \notin P'|_{D|x}$, we set $L := P'_{D|x}$. Then,

$$u \in L \leftrightarrow D[x \rightarrow u] \in P' \rightarrow u \in \{v_1, \dots, v_e\}.$$

Hence,

$$\Pr[y \in L | y \xleftarrow{\$} Y] \leq \Pr[y \in \{v_1, \dots, v_e\} | y \xleftarrow{\$} Y] \leq \frac{M}{2^{2\lambda}}.$$

By Theorem 1, we obtain the bound

$$\llbracket \text{SZ}_{\leq i} \setminus \text{SUC} \rightarrow \text{SUC} \rrbracket \leq \sqrt{\frac{10M}{2^{2\lambda}}}.$$

In conclusion, we derive the following bound:

$$\begin{aligned} & \frac{n}{N} \left(1 - \frac{q_s \cdot (q_s + q_{H^{\text{FS}}})}{2^\lambda}\right) \cdot \text{Adv}_{\mathcal{A}}^{\text{EUF}}(1^\lambda) \\ & \leq \text{Adv}_{\mathcal{B}}^{\mathcal{R}_{\beta \mathcal{L}^t}}(1^\lambda) + \frac{q^2(60q + 10M + 40\sqrt{M}q^{\frac{1}{2}}) + M + 1}{2^{2\lambda}} + \epsilon_{\text{ext}}, \end{aligned}$$

where $q = q_{H^{\text{FS}}} + q_{H^{\text{MT}}}$. $\square$

B.12 Proof of Theorem 12

Proof. Let hash function $\mathcal{H}_{\text{FS}}$ be modeled as a random oracle. $\mathcal{B}$ runs signature setup to generate R-pp and gives R-pp to $\mathcal{A}$. $\mathcal{B}$ simulates the oracles similarly as in unforgeability game. Except that in anonymity game, $\mathcal{A}$ is given all public-secret key pairs and can insert maliciously generated keys.

Challenge. $\mathcal{A}$ gives $\mathcal{B}$ a message msg^* , a set of public keys $\mathcal{PK}^*$ with n public keys, and two signer indices i_0, i_1 w.r.t. the same ring $\mathcal{PK}^*$. $\mathcal{B}$ computes the signature as follows:

1. $\mathcal{B}$ randomly selects $h_1, \dots, h_n \xleftarrow{\$} \{0, 1\}^\lambda$ and expands each seed to $(\bar{\ell}_e^{(i)})_{e \in [M]} \leftarrow \text{expand}(h_i)$, where $\bar{\ell}_e^{(i)} \in [0, L - 1]$.
2. $\mathcal{B}$ computes $(\bar{z}_e, \bar{c}_e) = \left(\left\lfloor \frac{\bar{\ell}_e^{(i)}}{c} \right\rfloor, \bar{\ell}_e^{(i)} \pmod{c}\right)$. Then it computes $\bar{z}_e = \sum_{i=1}^n \bar{z}_e^{(i)}$, $\bar{c}_e = \sum_{i=1}^n \bar{c}_e^{(i)}$, and $\bar{\ell}_e = \bar{z}_e c + \bar{c}_e$.

3. $\mathcal{B}$ randomly selects $\Delta K_e^* \xleftarrow{\$} \mathbb{F}_p$ and computes $o_e^* = I_{\bar{\ell}_e} \sum_{\ell=0, \ell \neq \bar{\ell}_e}^{L-1} r_{e,\ell} + \sum_{\ell=0, \ell \neq \bar{\ell}_e}^{L-1} r_{e,\ell}, I_{\ell} + \Delta K_e^*$.
4. $\mathcal{B}$ generates $b_e \xleftarrow{\$} \{0, 1\}$ and set $\Delta K_{e, \bar{z}_e}^{b_e} = \Delta K_e^*$ where $\Delta K_{e, \bar{z}_e}^{1-b_e} \xleftarrow{\$} \mathbb{F}_p$. For other $j \in [0, z-1] \setminus \{\bar{z}_e\}$, $(\Delta K_{e,j}^{b_e}, \Delta K_{e,j}^{1-b_e}) \xleftarrow{\$} \mathbb{F}_p^2$.
5. $\mathcal{B}$ computes $T_e^* = \mathcal{L}_0^t(o_e^*) - \sum_{i=1}^n pk_{\bar{\ell}_e^{(i)}}^{(i)}$.
6. $\mathcal{B}$ computes all other first round components following the protocol.
7. Finally, $\mathcal{B}$ programs the random oracle $\mathcal{H}$, which on input $(\text{com}, \text{com}_{\text{MT}}, (T_e^*)_{e \in [M]})$, outputs h such that $h[128] = \bigoplus_{i=1}^n h_i$ and the last 128 bits are randomly chosen by $\mathcal{B}$.
8. $\mathcal{B}$ follows the remaining steps to generate σ honestly.

Then, $\mathcal{B}$ returns the simulated signature σ to $\mathcal{A}$ and randomly picks $\text{bit} \xleftarrow{\$} \{0, 1\}$.

Hybrid Argument. It is obvious that the simulated signature passes the verification, we omit the detail of the simulation correctness. We show that the distribution of simulated ring signature is indistinguishable from the real distribution in a hybrid argument, assume BAVC is hiding.

- **H0(Real)** Honest prover (knows K w.r.t. $pk_\pi \in \mathcal{PK}$) follows Algorithm 5 and 6 to output a real signature

$$\sigma = (\text{salt}, \text{ctr}, \text{decom}_S, \text{path}_I, (\Delta K_e, v_e, d_e)_{e \in [M]}, (h_i)_{i \in [n]}).$$

Here, the signer runs BAVC and Merkle tree commitment honestly such that

$$r_e = \sum_{\ell=0}^{L-1} r_{e,\ell}, \quad T_e = \mathcal{L}_0^t(r_e) - \sum_{i=1, i \neq \pi}^n pk_{\bar{\ell}_e^{(i)}}^{(i)}$$

and

$$\Delta K_e = (K - I_{\bar{\ell}_e} + I_{\bar{\ell}_e^{(\pi)}})r_e + \sum_{\ell=0}^{L-1} I_{\ell} r_{e,\ell},$$

$$v_e = \mathcal{H}(\text{salt}, e, \bar{z}_e, 0/1, \Delta K_e \pm bcr_e).$$

- **H1(Randomizing unopened leaves for BAVC).** For each $e \in [M]$, replace the unopened message $r_{e, \bar{\ell}_e}$ with an independent, uniformly sampled $r'_{e, \bar{\ell}_e} \xleftarrow{\$} \mathbb{F}_p$. Because BAVC is hiding, replacing unopened message with fresh random values yields a distribution for the new signature that is indistinguishable from the real signature, with distinguishing advantage at most $\text{AdvHD}_{\mathcal{A}}^{\text{BAVC}}$. Concretely, after the substitution, we have

$$r'_e = \sum_{\ell \neq \bar{\ell}_e} r_{e,\ell} + r_{e, \bar{\ell}_e}, \quad T'_e = \mathcal{L}_0^t(r'_e) - \sum_{i=1, i \neq \pi}^n pk_{\bar{\ell}_e^{(i)}}^{(i)}$$

and

$$\Delta K'_e = (K - I_{\bar{\ell}_e} + I_{\bar{\ell}_e^{(\pi)}})r'_e + \sum_{\ell \neq \bar{\ell}_e} I_\ell r_{e,\ell} + I_{\bar{\ell}_e} r_{e,\bar{\ell}_e},$$

$$v_e = \mathcal{H}_{\text{MT}}(\text{salt}, e, \bar{z}_e, 0/1, \Delta K'_e \pm bcr'_e)$$

Since $r'_{e,\bar{\ell}_e}$ is uniform and independent, r'_e is uniform over $\mathbb{F}_p$. Hence, T'_e is the Legendre or t -th power residue PRF of a uniform input (minus a fixed constant value). For all $j \in [0, z-1]$, since we can write $\Delta K_{e,j}^{b'} = A_{e,j}^b r'_{e,\bar{\ell}_e} + B_{e,j}$ for $b \in \{0, 1\}$. As long as $A_{e,j}^b \neq 0$, which happens with probability at most $2ML/p$, $\Delta K_{e,j}^{0'}$ and $\Delta K_{e,j}^{1'}$ are all uniformly random from the adversary's view.

- **H2(Simulator)**. Replacing ΔK_e with independent uniformly random ΔK_e^* and set $T_e^* = \mathcal{L}_0^t(o_e^*) - \sum_{i=1}^n pk_{\bar{\ell}_e^{(i)}}$ as in simulation does not change the distribution of signature from H2. Furthermore, since the value committed in v_e and other Merkle tree nodes are uniformly random in H2, the simulation is a perfect hop from H1.

Output $\mathcal{A}$ outputs a guess bit' . Observe that bit is not used in the generation of σ , hence $\mathcal{A}$ wins with probability $1/2$.

Probability Analysis. We now analyze the probability that $\mathcal{B}$ aborts in the above simulation. For q_h queries made to $\mathcal{H}$ and q_s queries made to signing oracle, $\mathcal{B}$ aborts if the simulated σ_1 was queried. Due to the uniformly random salt of length 2λ bit, for each signing oracle query, the probability that $\mathcal{A}$ has queried σ_1 is at most $\frac{q_h + q_s}{2^{2\lambda}}$. Then, for all q_s queries, the aborting probability is at most $\frac{q_s(q_s + q_h)}{2^{2\lambda}}$. $\square$